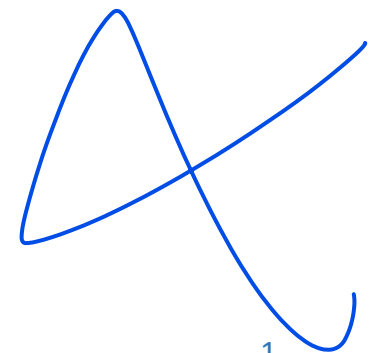


Graph-II



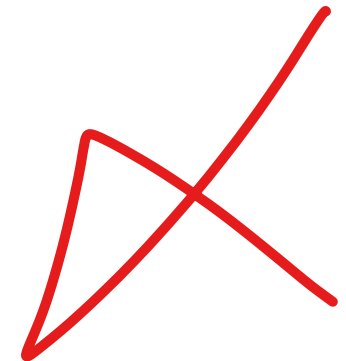
TOPICS

Breadth First Search (BFS)

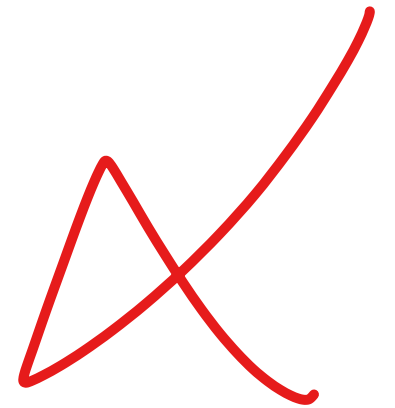
Level Order Traversal 

Depth First Search (DFS)

~~Topological Sorting~~

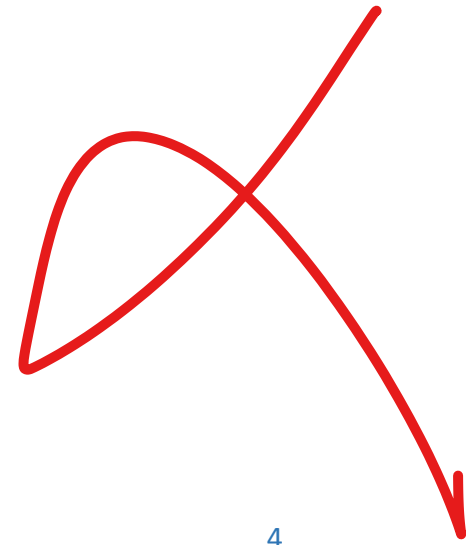


BREADTH FIRST SEARCH

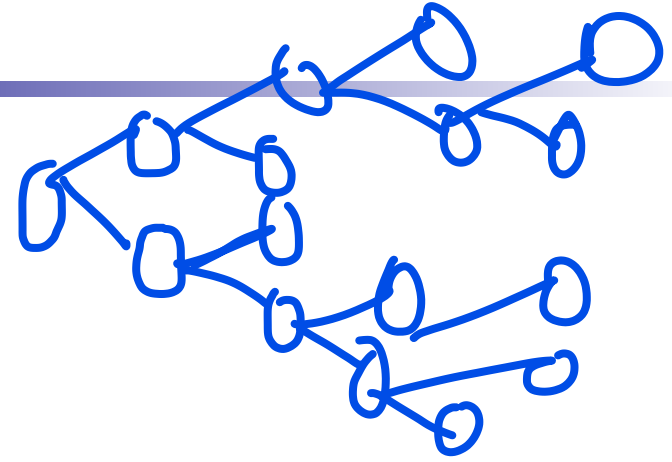


Graph Searching

- Given: a graph $G = (V, E)$, **directed or undirected**.
- Goal: **methodically explore (or visit)** every vertex.
- There are **two standard graph traversal techniques**:
 - Breadth-First Search (BFS)
 - Depth-First Search (DFS)
- You will be requiring these two algorithms in the future too:
 - Learning algorithms like Backtracking
 - In Courses like Artificial Intelligence, DSA – II, etc.
 - Developing software

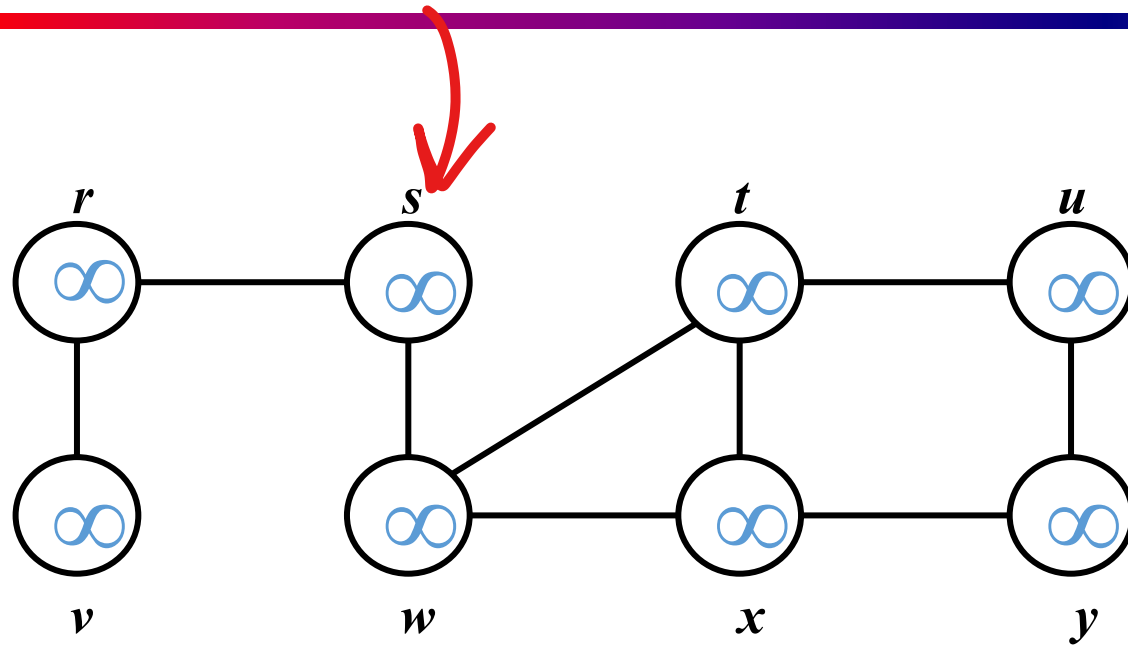


Breadth-First Search



- Building a tree over the graph (**parent array**)
 - We will pick a *source vertex* to be the root
 - We will find (“discover”) its **children**, then their children, etc.
- Associating **colors** (**colors array**)
 - White vertices have **not been discovered** : All vertices start out white
 - Grey vertices are discovered **but not fully explored** : They may be adjacent to white vertices. **Enqueuing** will be done for these vertices.
 - Black vertices are **discovered** and **fully explored**: They are adjacent only to black and grey vertices
- Keeping track of distance from the root (**distance array**)
 - Helps in determining how far a vertex is from the root
- Using a **queue** (array based queue/ linked list based queue)
 - Will be used for guiding the exploration

BFS Simulation



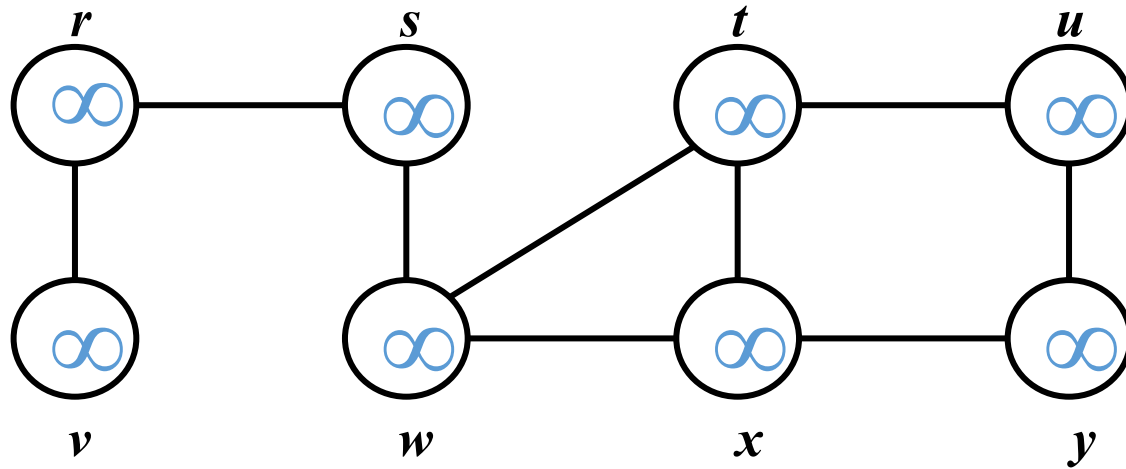
We can use arrays like before for this even though the vertices are represented by characters (use the ASCII values)

properties	r	s	t	u	v	w	x	Y
d	inf	inf	inf	inf	inf	inf	inf	inf
P	-1	-1	-1	-1	-1	-1	-1	-1
c	w	w	w	w	w	w	w	W

Q

- Initial vertex condition
 - all the vertices will be at infinite distance from the source (source not picked yet).
 - All the vertices will be colored w (not been discovered)
 - They will have no parent
- An empty Queue

BFS Simulation

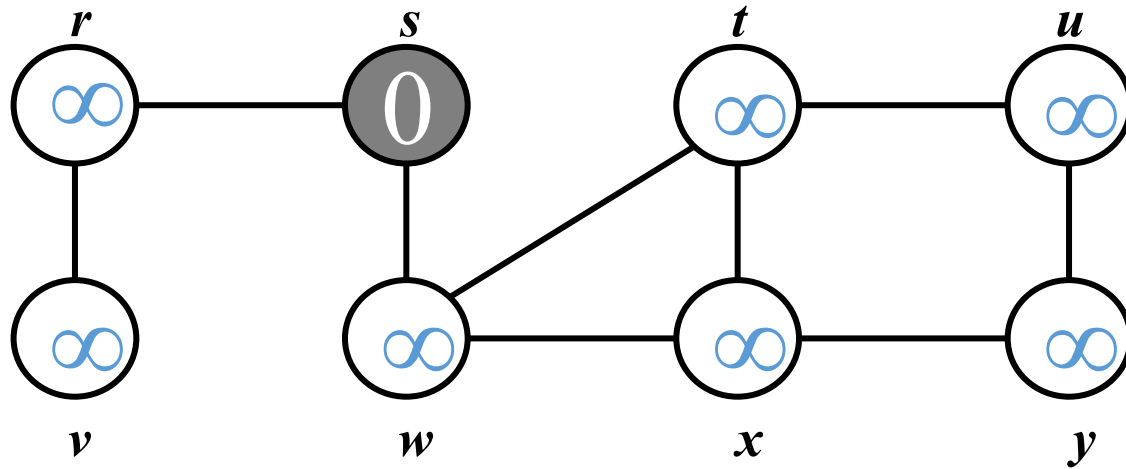


properties	r	s	t	u	v	w	x	Y
d	inf	inf	inf	inf	inf	inf	inf	inf
P	-1	-1	-1	-1	-1	-1	-1	-1
c	w	w	w	w	w	w	w	W

Q

- Let us take s as the source for this example.
 - We will now update its attributes

BFS Simulation

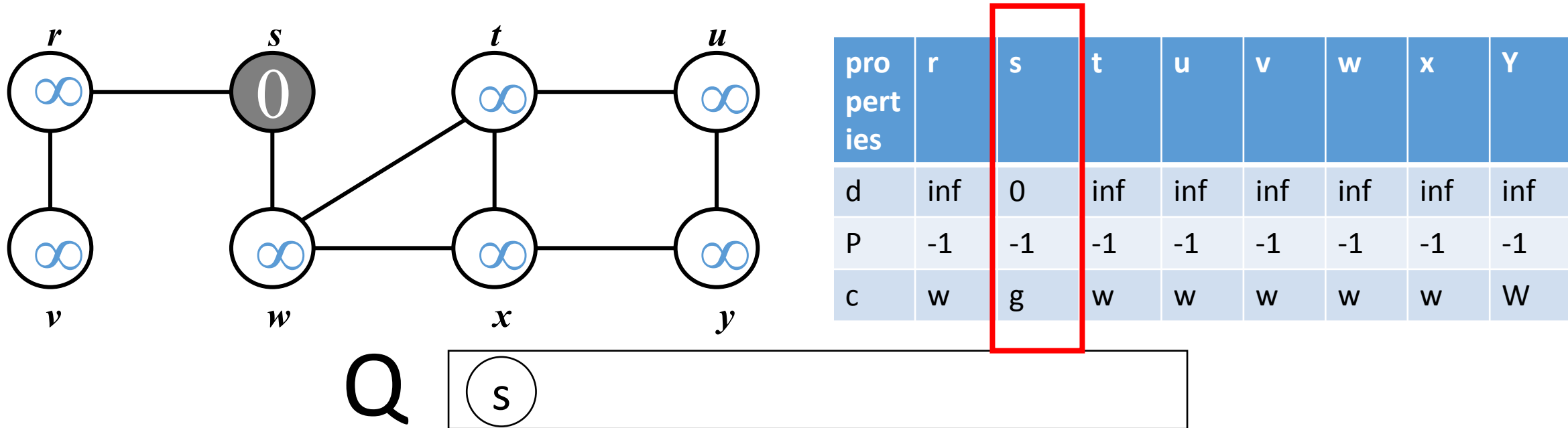


properties	r	s	t	u	v	w	x	Y
d	inf	0	inf	inf	inf	inf	inf	inf
P	-1	-1	-1	-1	-1	-1	-1	-1
c	w	g	w	w	w	w	w	W



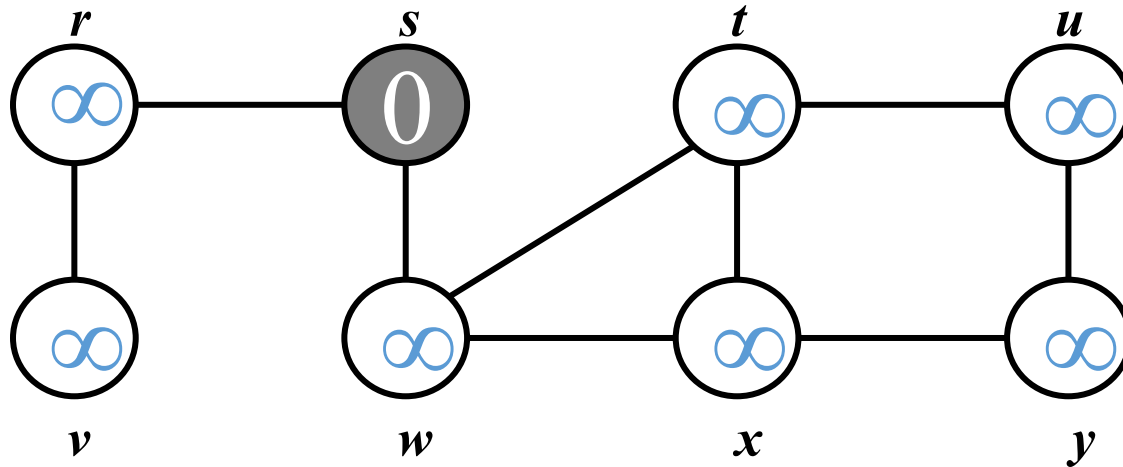
- Let us take s as the source for this example.
 - We will now update its attributes
 - Distance (from source) = 0 (as s itself is the source)
 - Parent = -1 (no parent)
 - Color=gray (discovered) . We are placing gray/discovered vertices in the queue

BFS Simulation

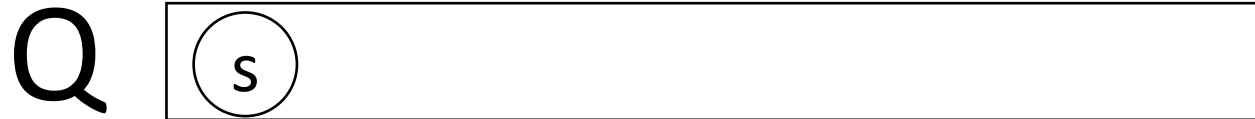


- Let us take s as the source for this example.
 - We will now update its attributes
 - Distance (from source) = 0 (as s itself is the source)
 - Parent = -1 (no parent)
 - Color=gray (discovered) . We are placing gray/discovered vertices in the queue

BFS Simulation

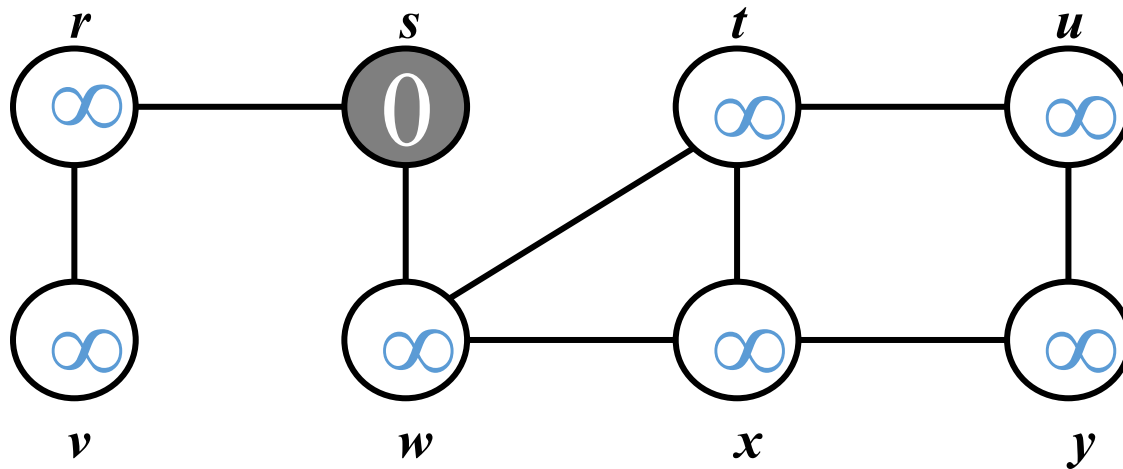


properties	r	s	t	u	v	w	x	Y
d	inf	0	inf	inf	inf	inf	inf	inf
P	-1	-1	-1	-1	-1	-1	-1	-1
c	w	g	w	w	w	w	w	W



- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



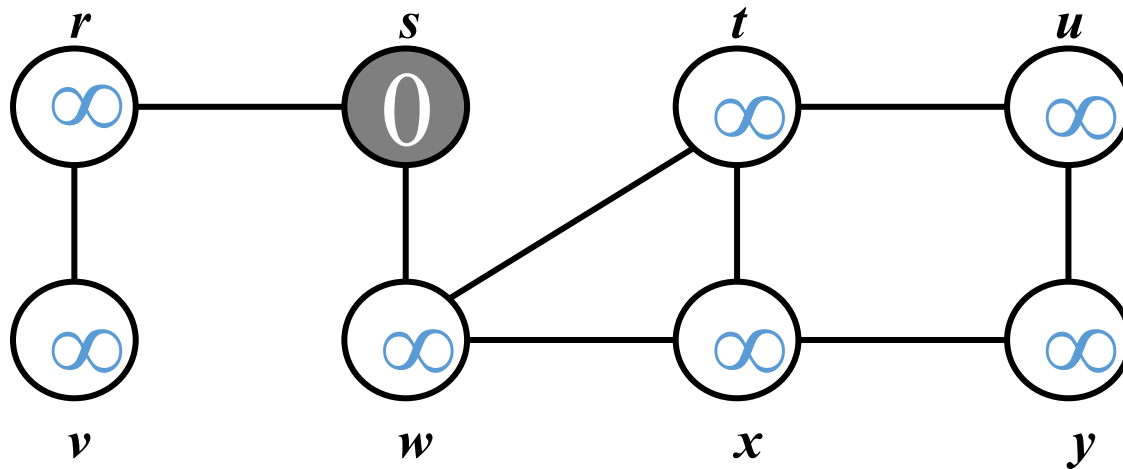
properties	r	s	t	u	v	w	x	Y
d	inf	0	inf	inf	inf	inf	inf	inf
P	-1	-1	-1	-1	-1	-1	-1	-1
c	w	g	w	w	w	w	w	W

S

Q

- Now keep on doing the following until the queue is empty
 - ➔ • Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



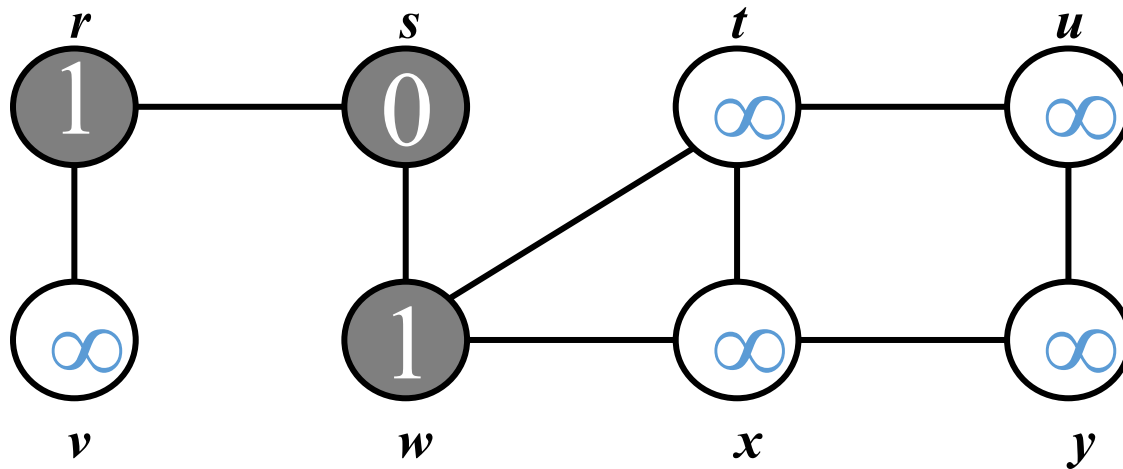
properties	r	s	t	u	v	w	x	Y
d	inf	0	inf	inf	inf	inf	inf	inf
P	-1	-1	-1	-1	-1	-1	-1	-1
c	w	g	w	w	w	w	w	W

S

Q

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - ➔ • Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



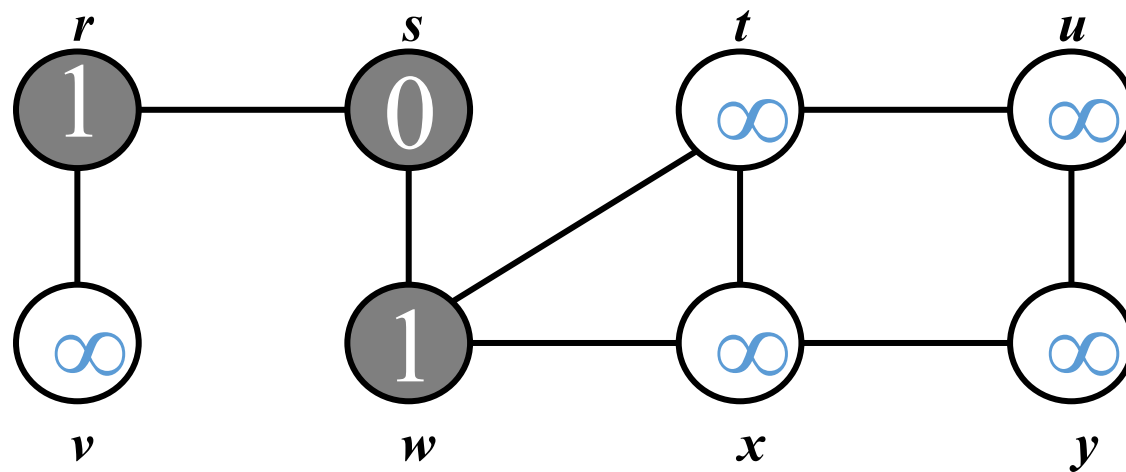
properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	inf	1	inf	inf
P	s	-1	-1	-1	-1	s	-1	-1
c	g	g	w	w	w	g	w	W

S

Q

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

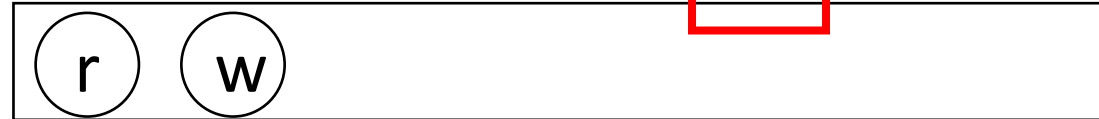
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	inf	1	inf	inf
P	s	-1	-1	-1	-1	s	-1	-1
c	g	g	w	w	w	g	w	W

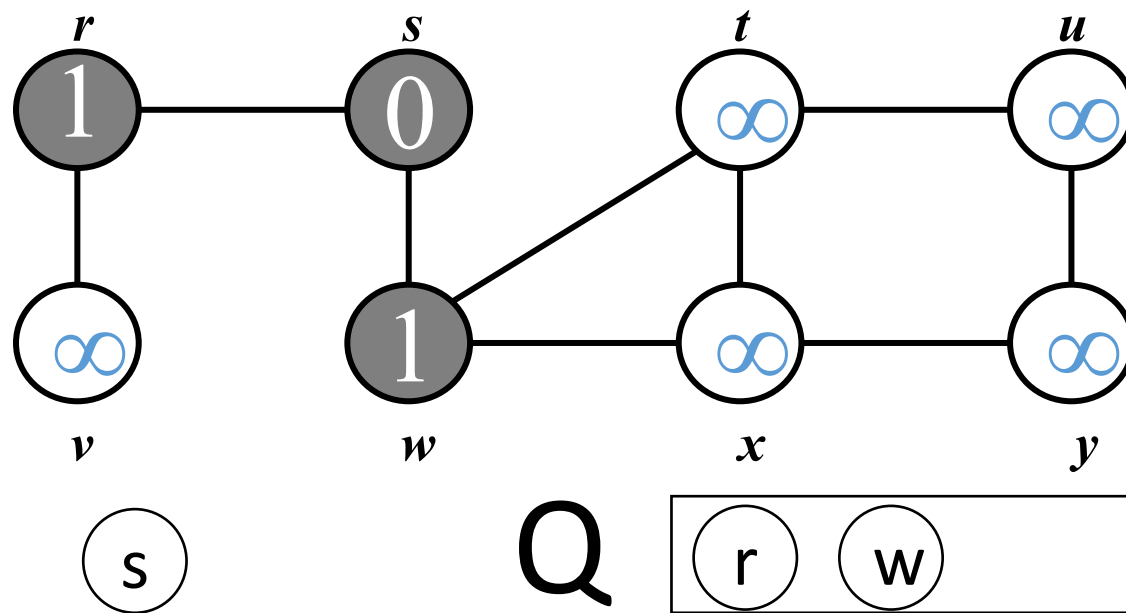
S

Q



- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

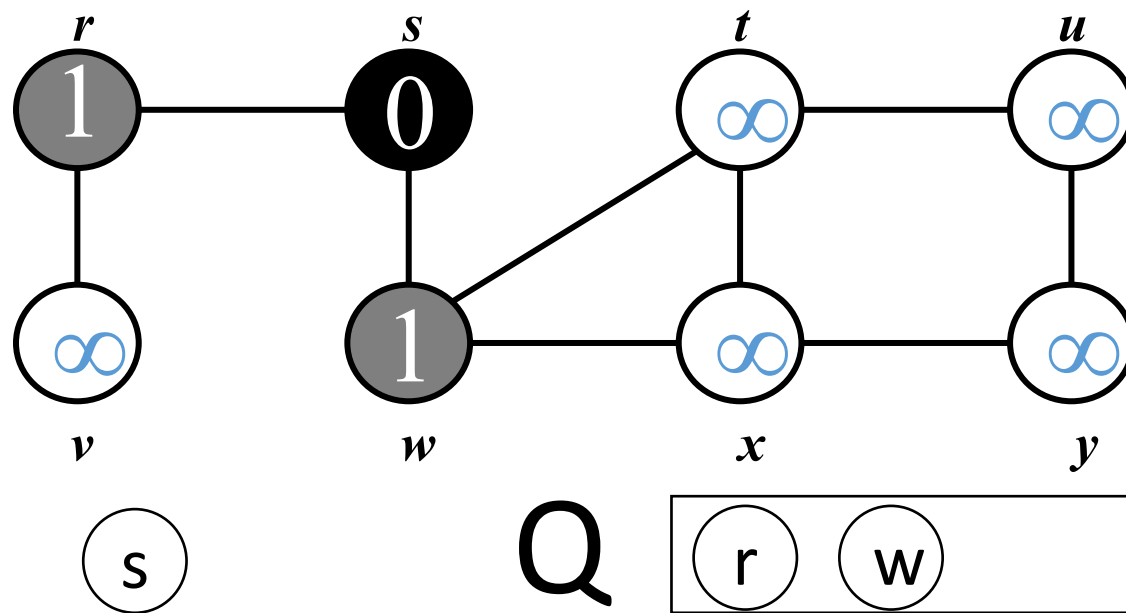
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	inf	inf	inf	1	inf	inf
P	s	-1	-1	-1	-1	s	-1	-1
c	g	g	w	w	w	g	w	W

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

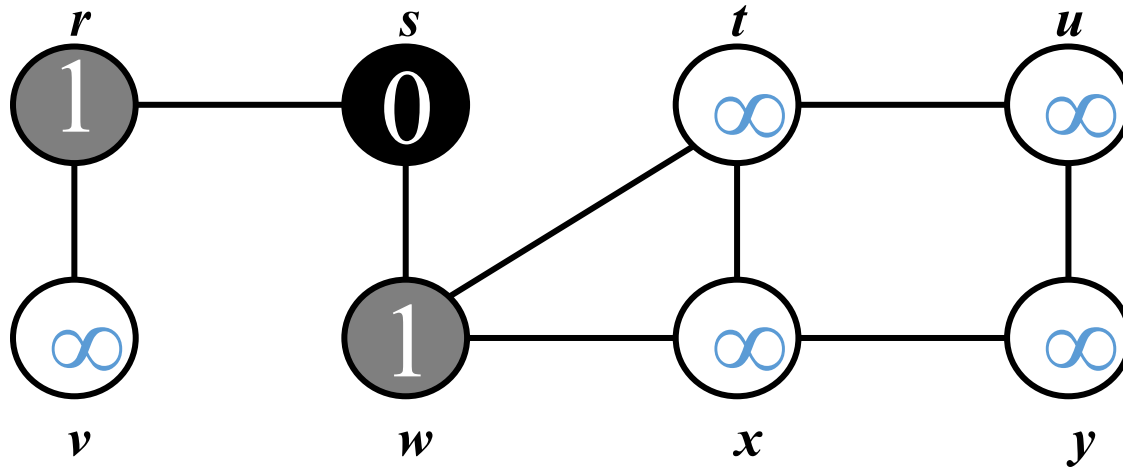


properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	inf	1	inf	inf
P	s	-1	-1	-1	-1	S	-1	-1
c	g	g	w	w	w	g	w	W

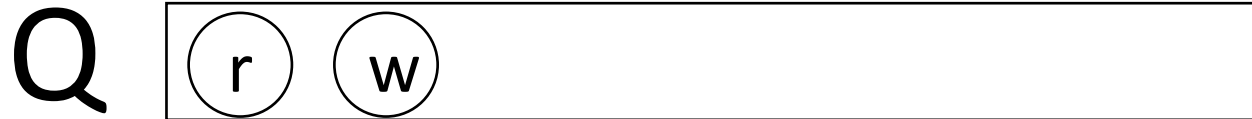
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- Mark the taken out element as explored (Black – all its children has been discovered)



BFS Simulation

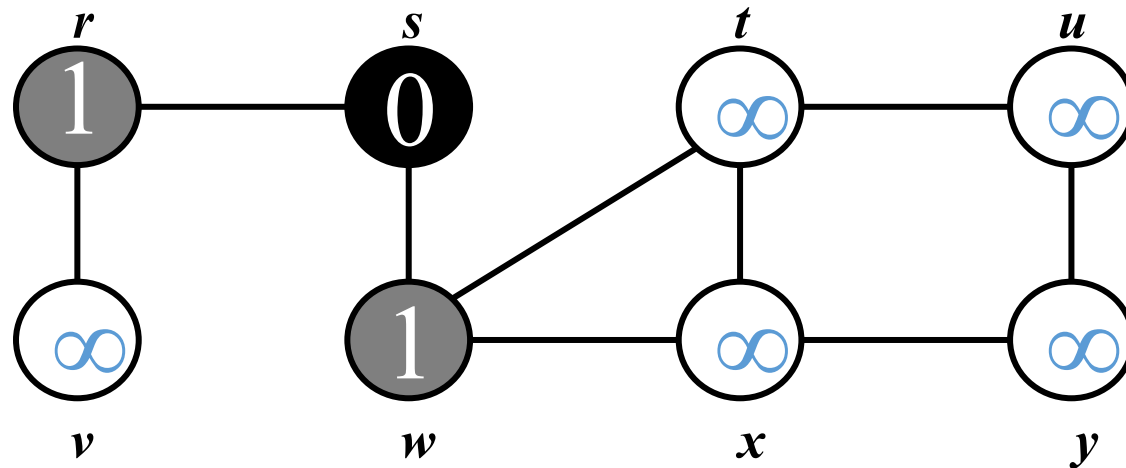


properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	inf	1	inf	inf
P	S	-1	-1	-1	-1	S	-1	-1
c	g	b	w	w	w	g	w	W



- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- ➔ • Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

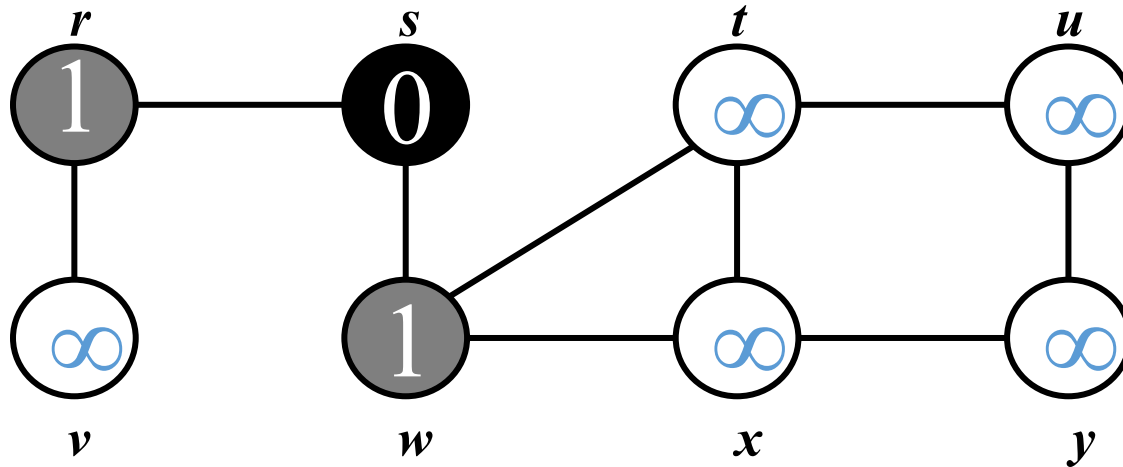


properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	inf	1	inf	inf
P	s	-1	-1	-1	-1	s	-1	-1
c	g	b	w	w	w	g	w	W



- ➔
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	inf	1	inf	inf
P	s	-1	-1	-1	-1	s	-1	-1
c	g	b	w	w	w	g	w	W

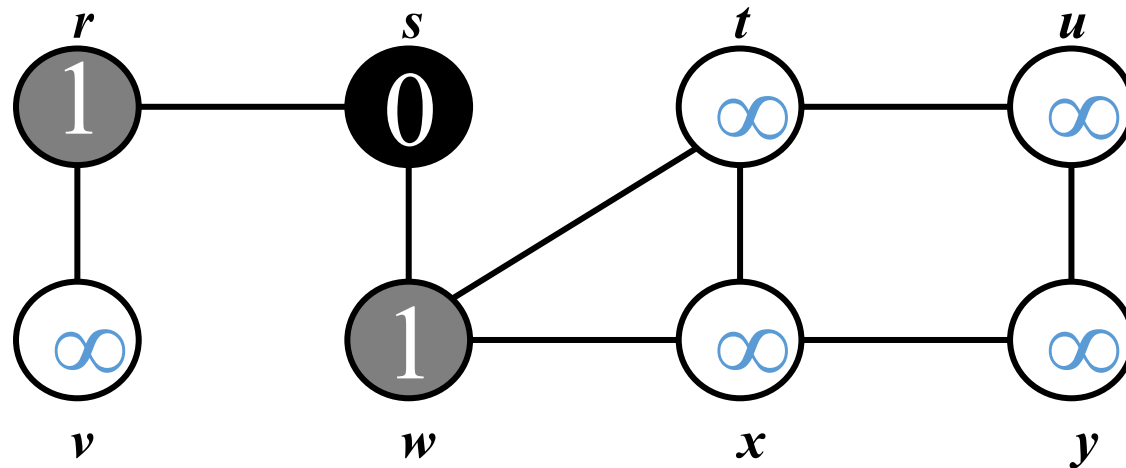
r

Q

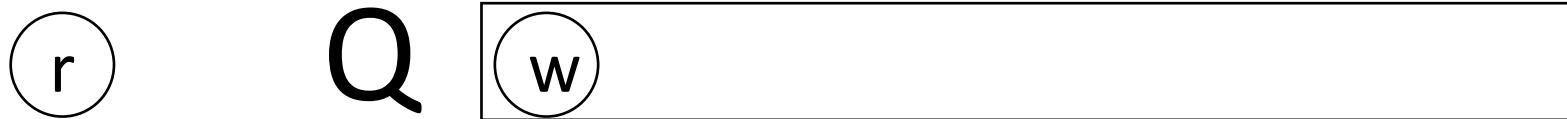


- Now keep on doing the following until the queue is empty
- ➔
- Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

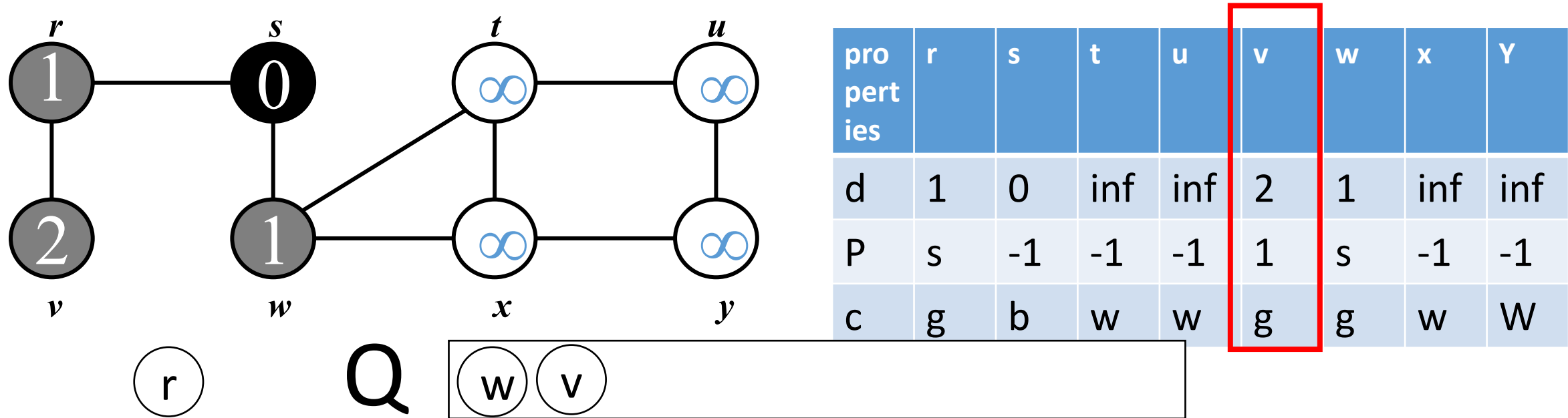


properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	inf	1	inf	inf
P	s	-1	-1	-1	-1	s	-1	-1
c	g	b	w	w	w	g	w	W



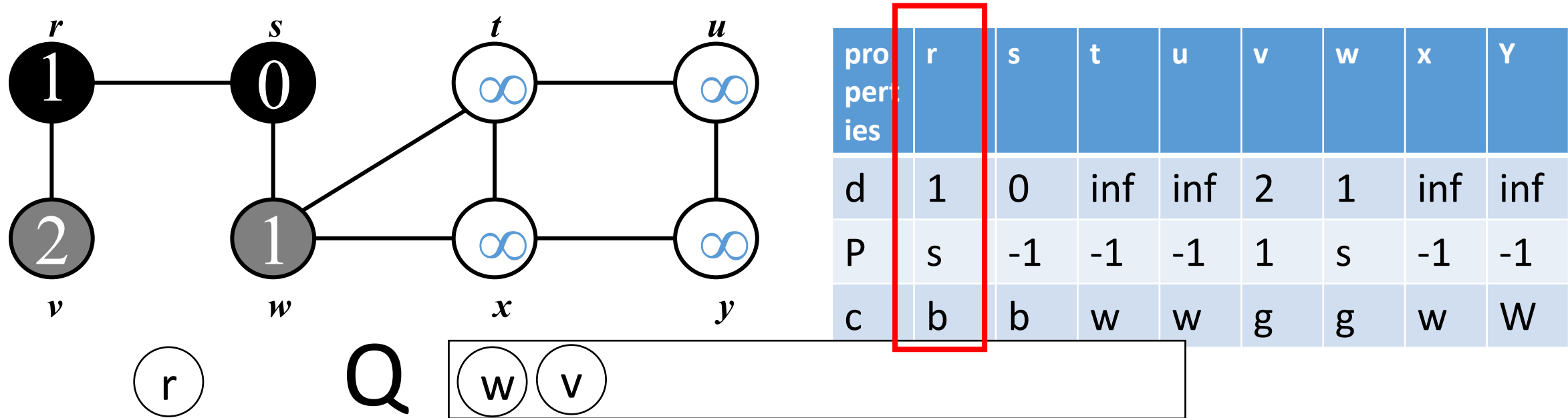
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



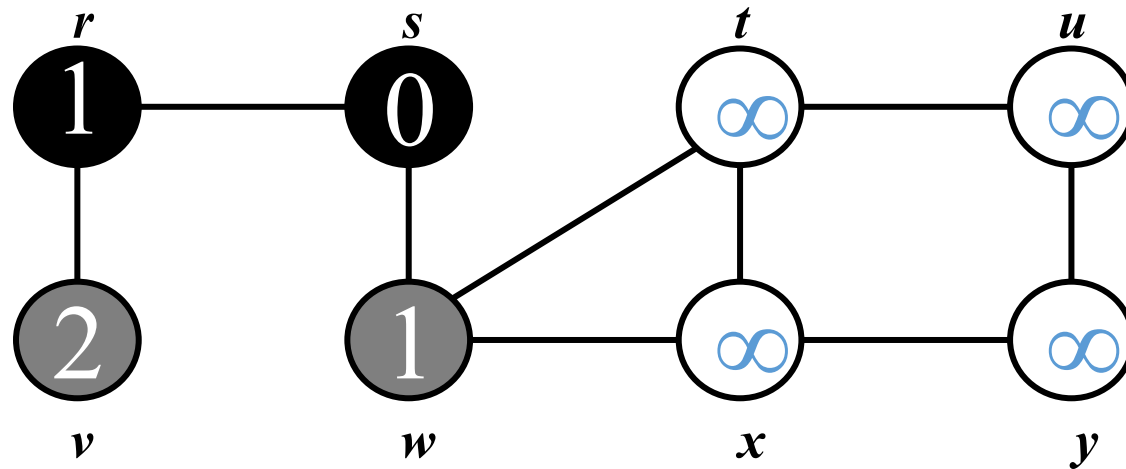
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

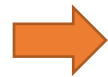
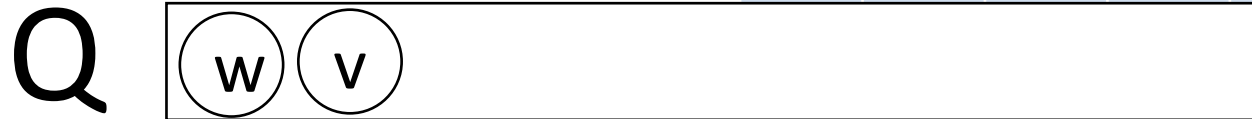


- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- ➔ • Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

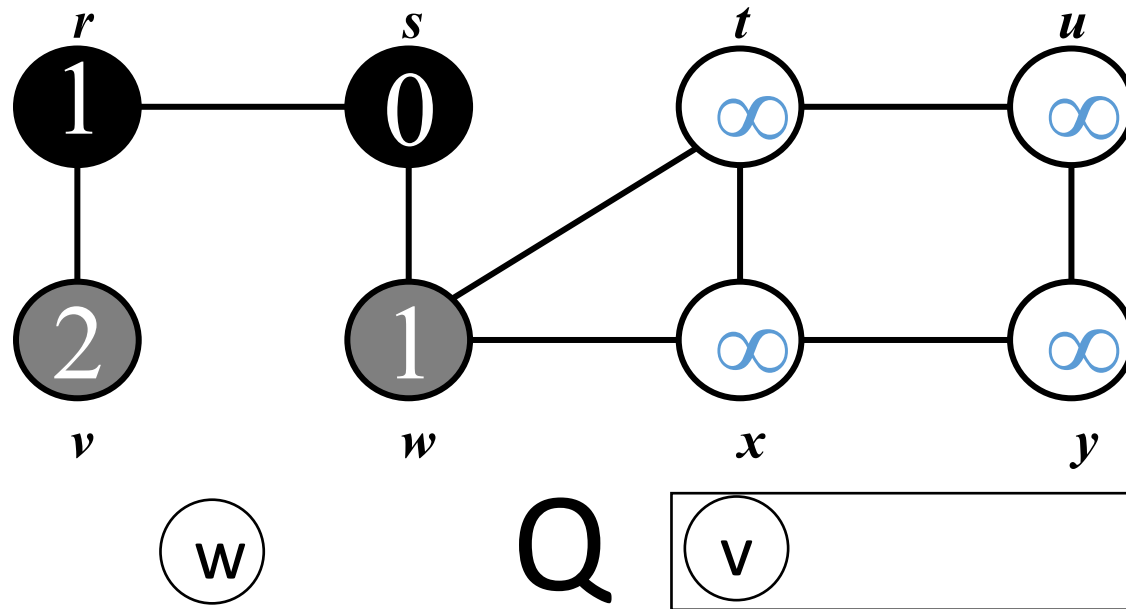


properties	r	s	t	u	v	w	x	Y
d	1	0	inf	inf	2	1	inf	inf
P	s	-1	-1	-1	1	s	-1	-1
c	b	b	w	w	g	g	w	W



- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

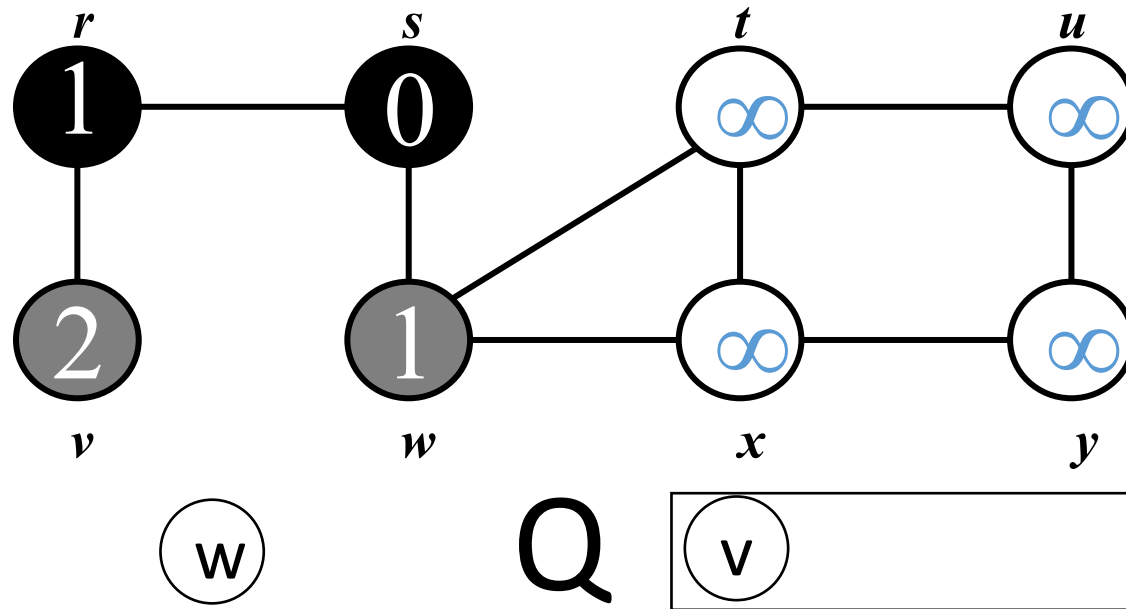
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	inf	inf	2	1	inf	inf
P	s	-1	-1	-1	1	s	-1	-1
c	b	b	w	w	g	g	w	W

- Now keep on doing the following until the queue is empty
 - ➔ • Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

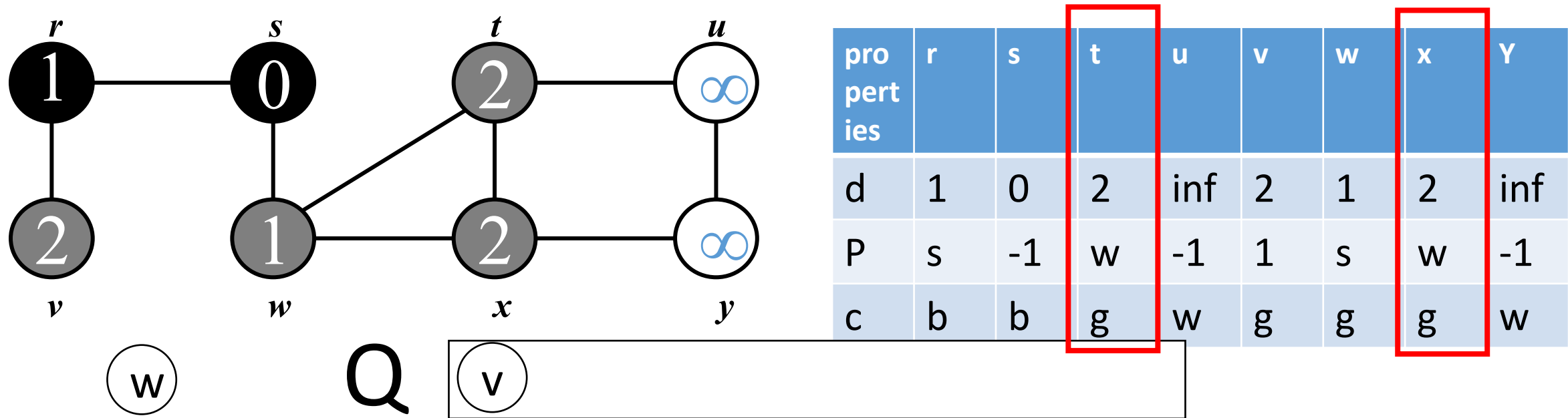
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	inf	inf	2	1	inf	inf
P	s	-1	-1	-1	1	s	-1	-1
c	b	b	w	w	g	g	w	W

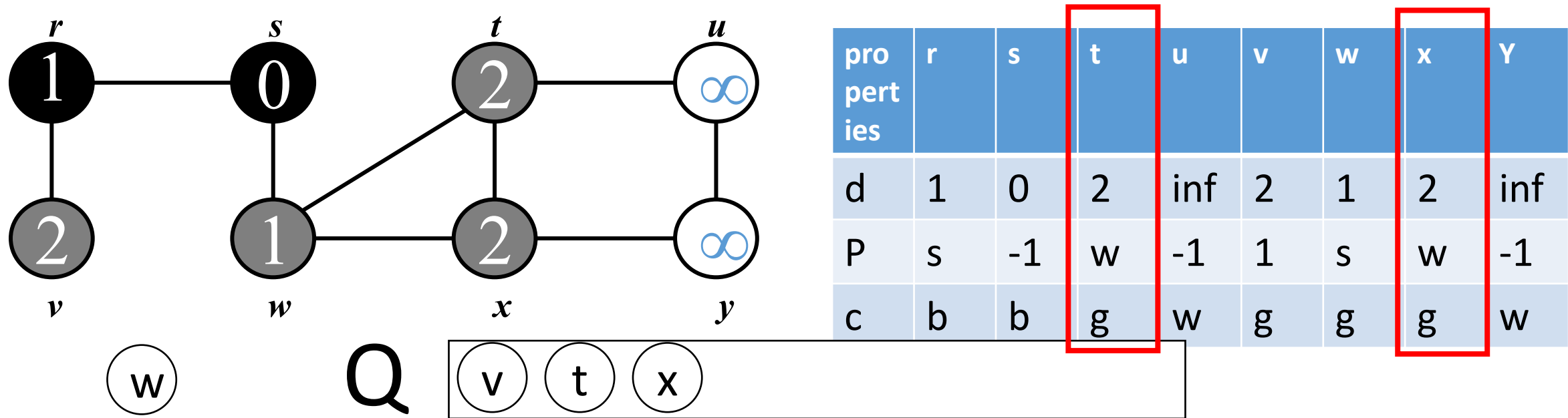
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - ➔ • Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



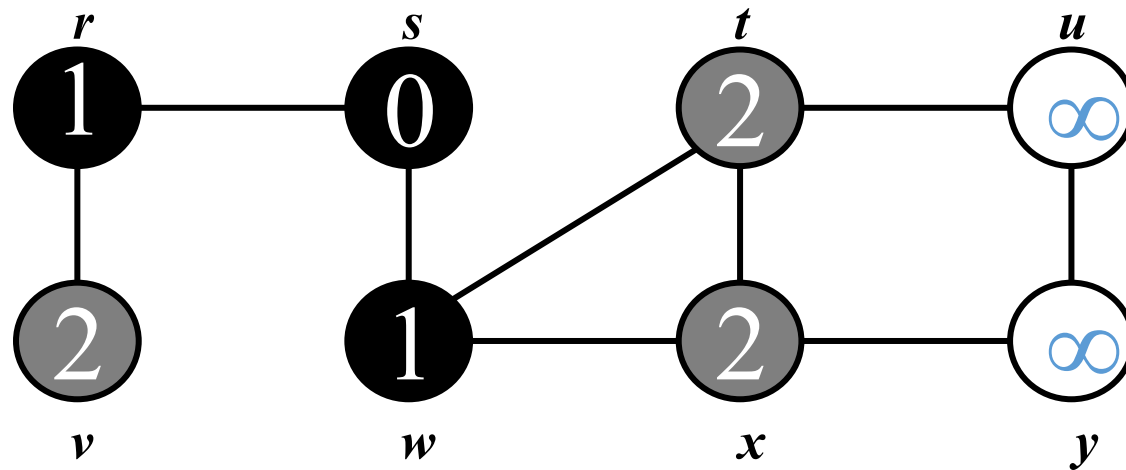
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

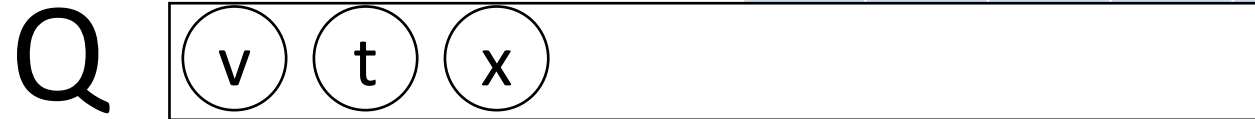


- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

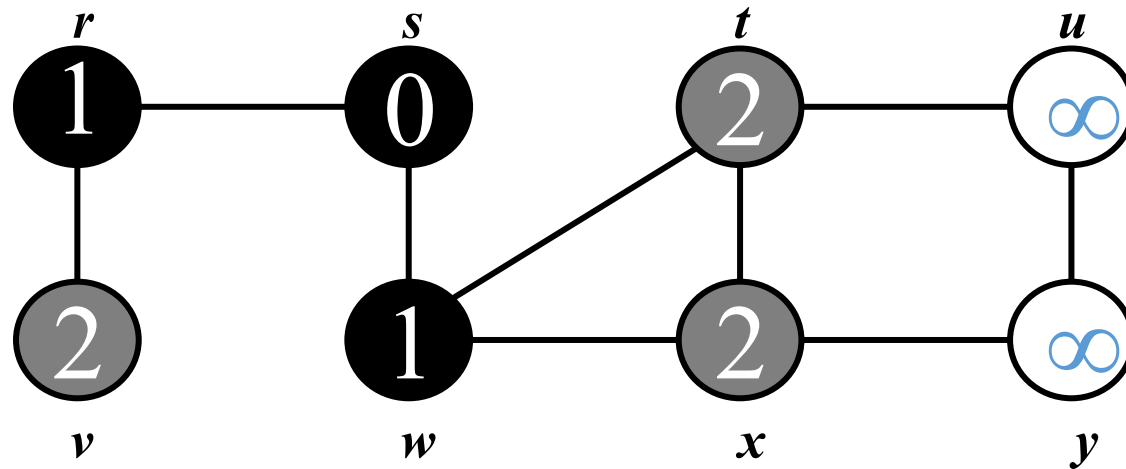


properties	r	s	t	u	v	w	x	y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	g	b	g	w



- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



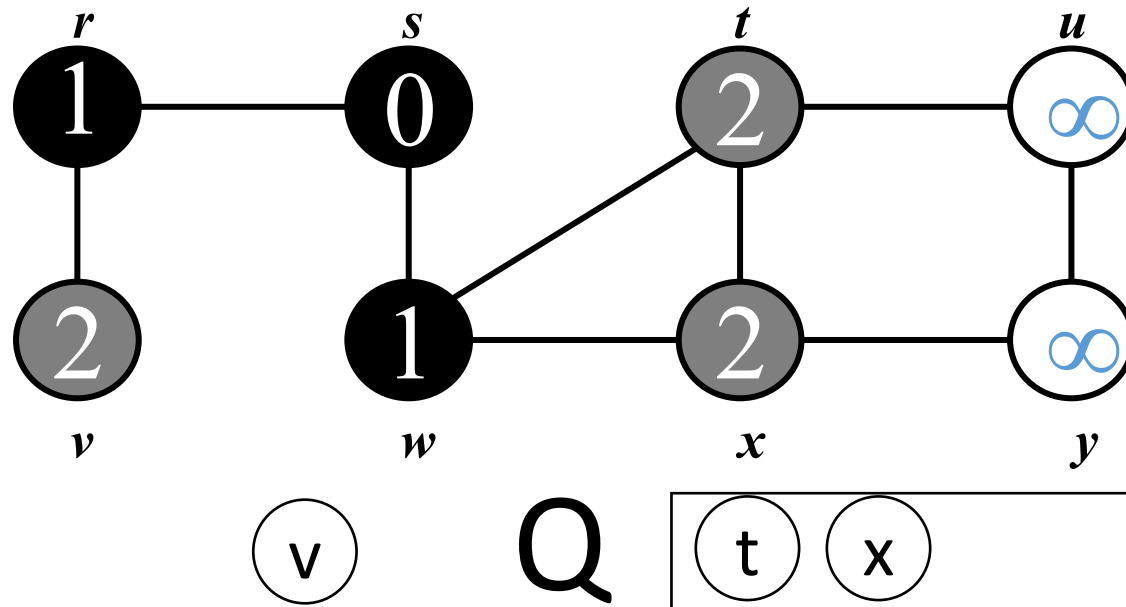
properties	r	s	t	u	v	w	x	Y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	g	b	g	w



Now keep on doing the following until the queue is empty

- Take out the topmost element
- Visit each of its neighbors
- If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- Mark the taken out element as explored (Black – all its children has been discovered)

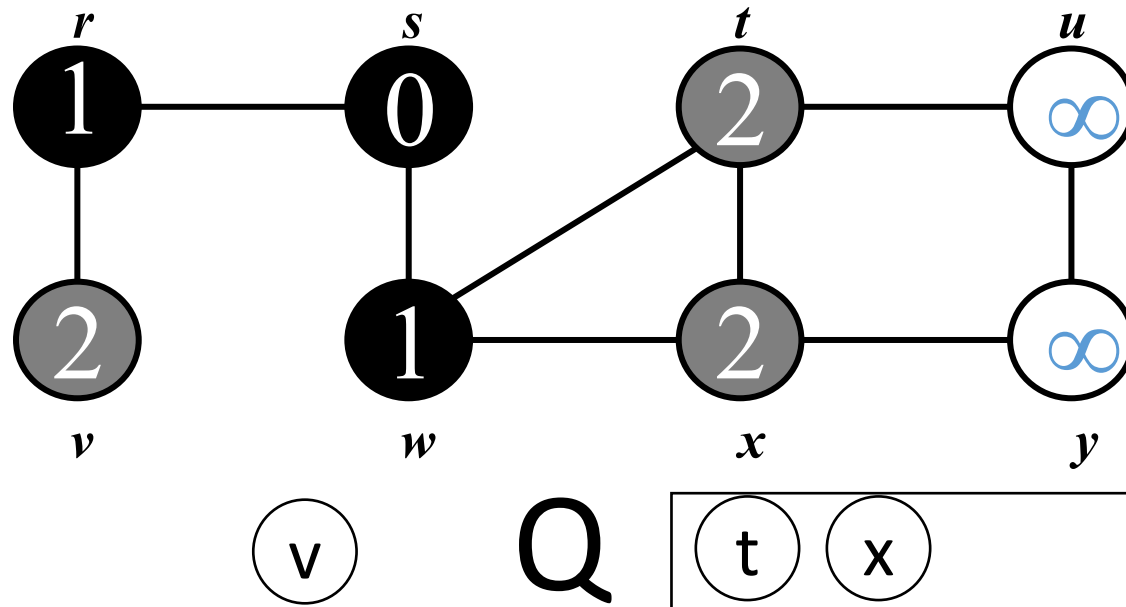
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	g	b	g	w

- Now keep on doing the following until the queue is empty
 - ➔ • Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

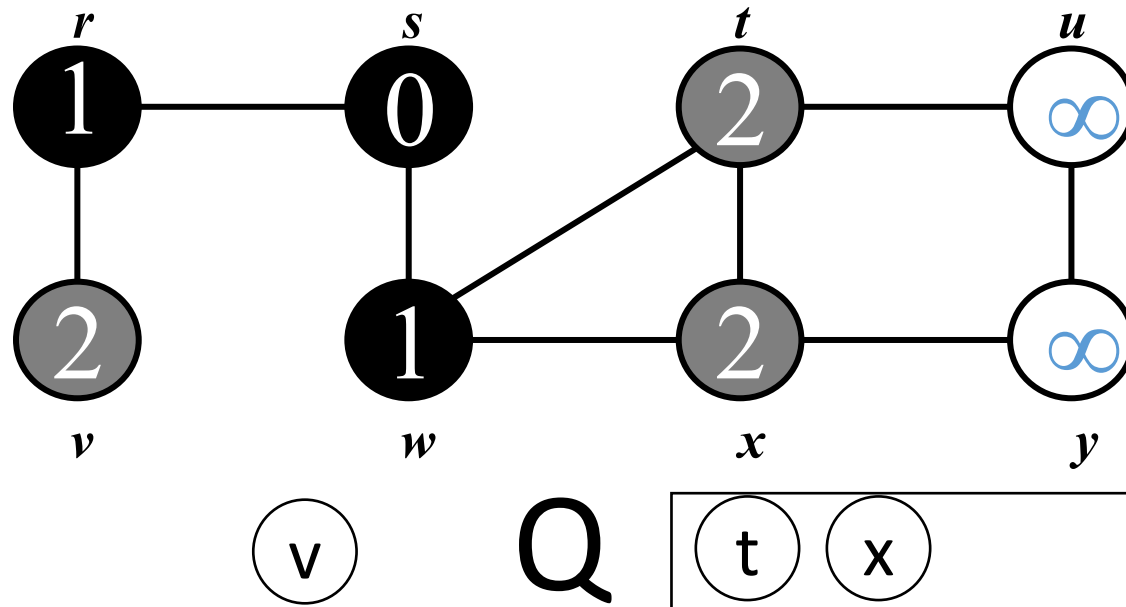
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	g	b	g	w

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - ➔ • Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

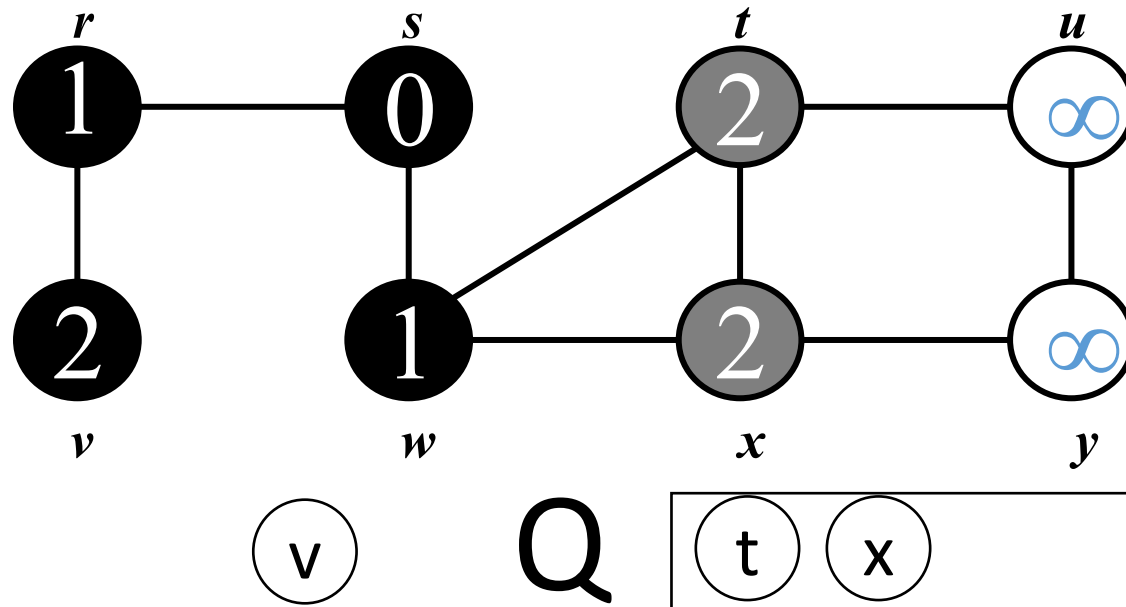
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	g	b	g	w

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

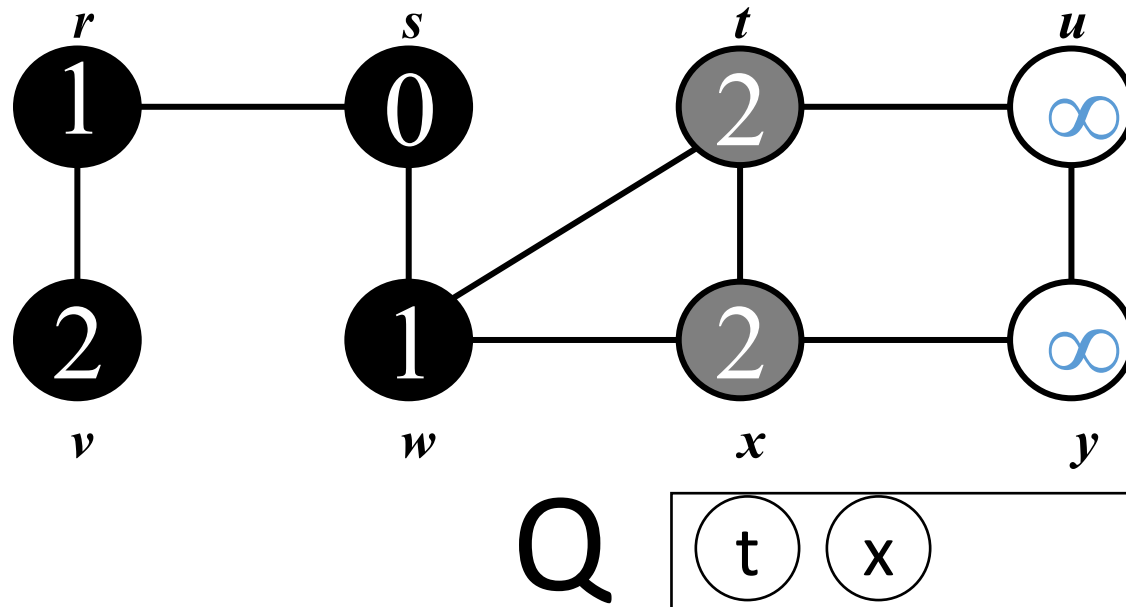
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	b	b	g	w

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- ➔ • Mark the taken out element as explored (Black – all its children has been discovered)

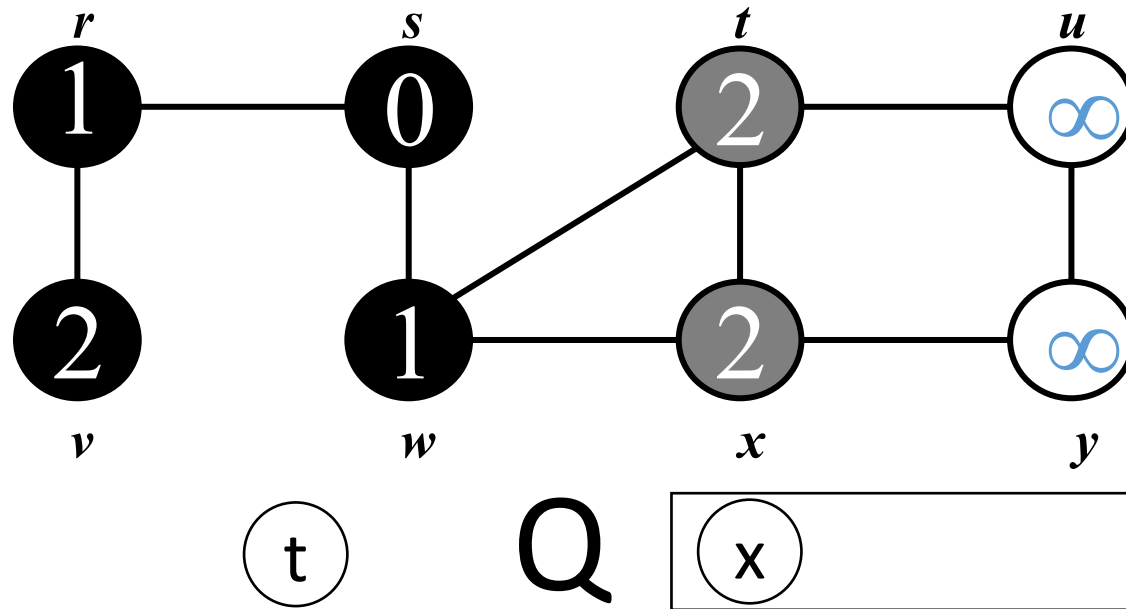
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	b	b	g	w

- ➔
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

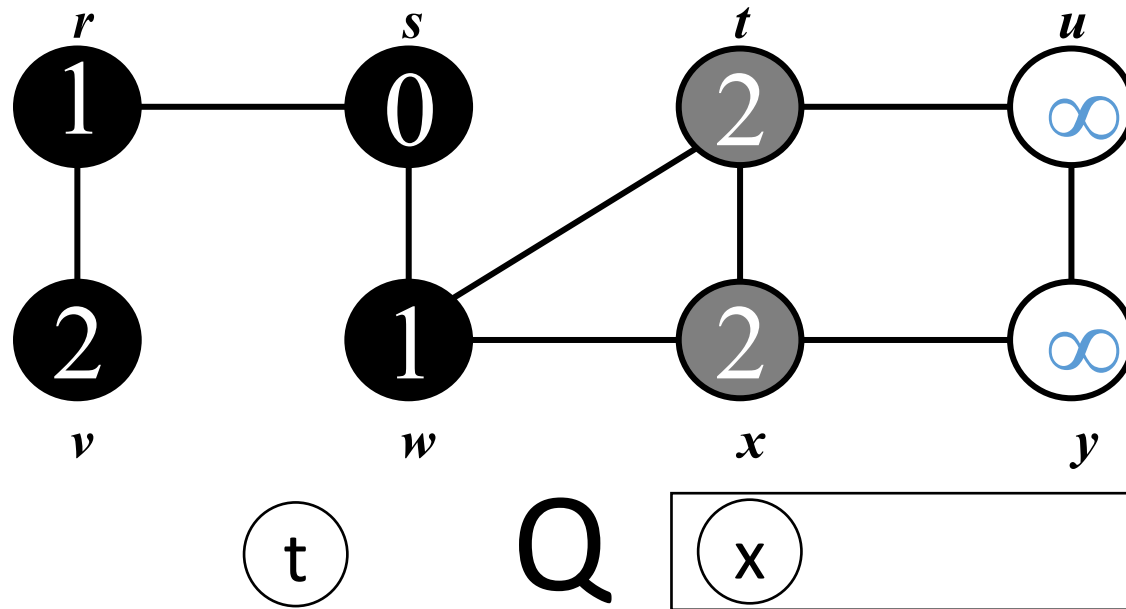
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	b	b	g	w

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

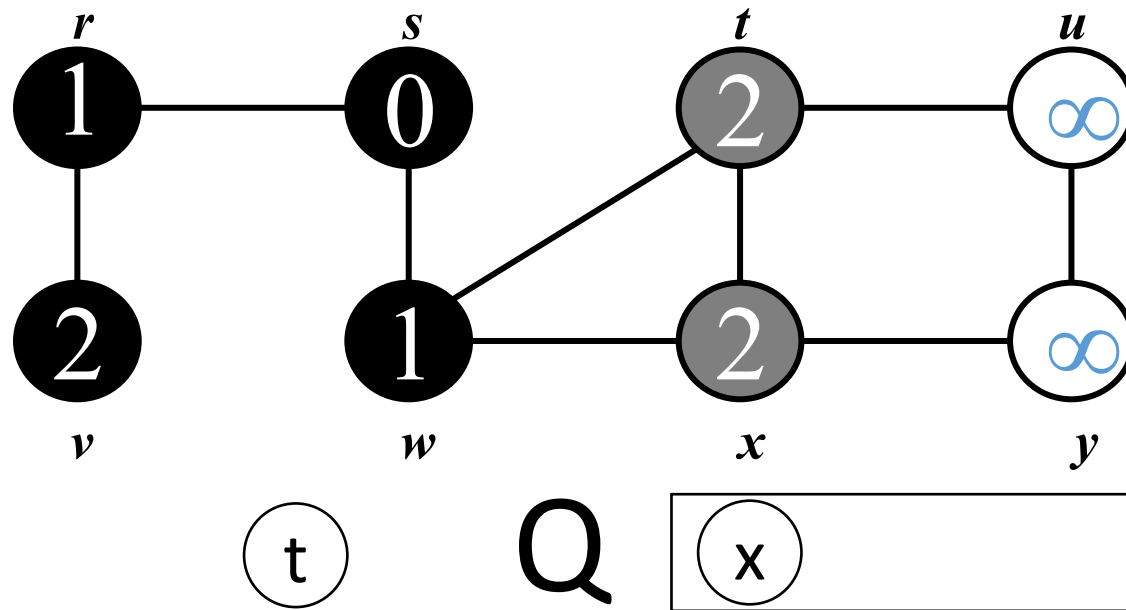
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	b	b	g	w

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

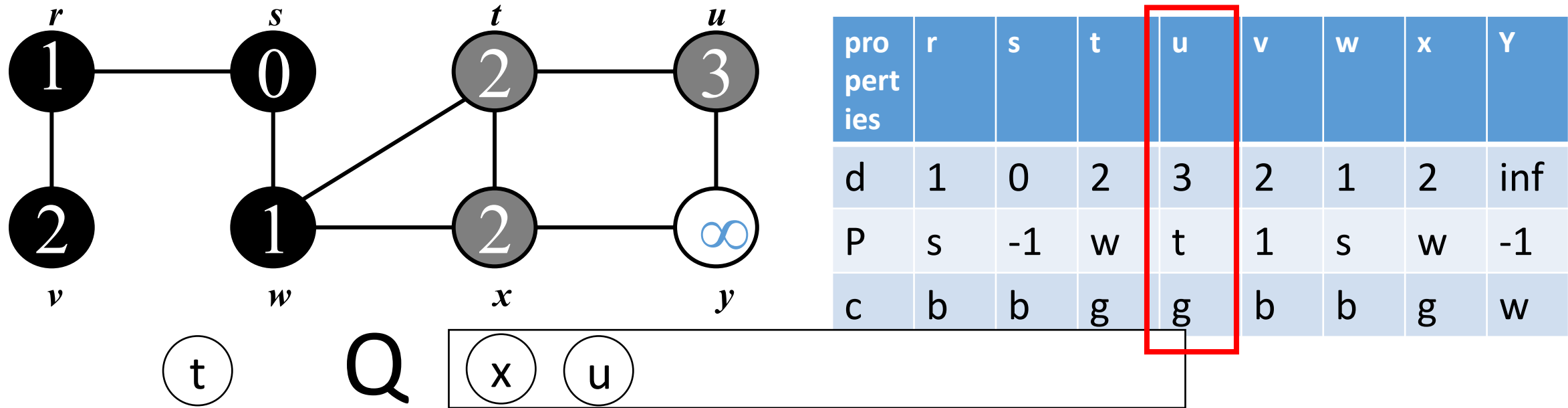
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	2	inf	2	1	2	inf
P	s	-1	w	-1	1	s	w	-1
c	b	b	g	w	b	b	g	w

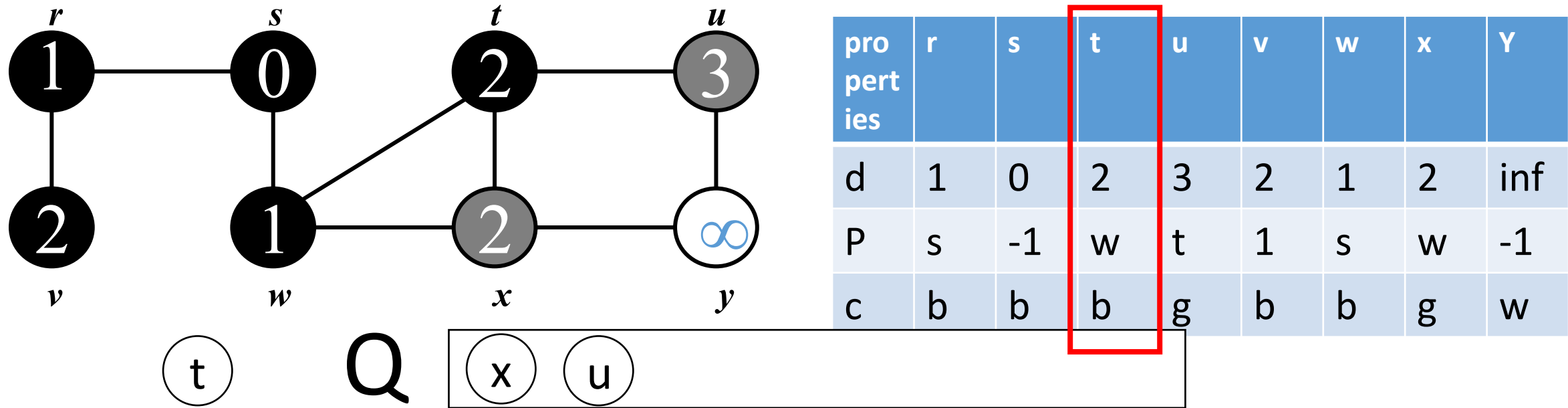
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



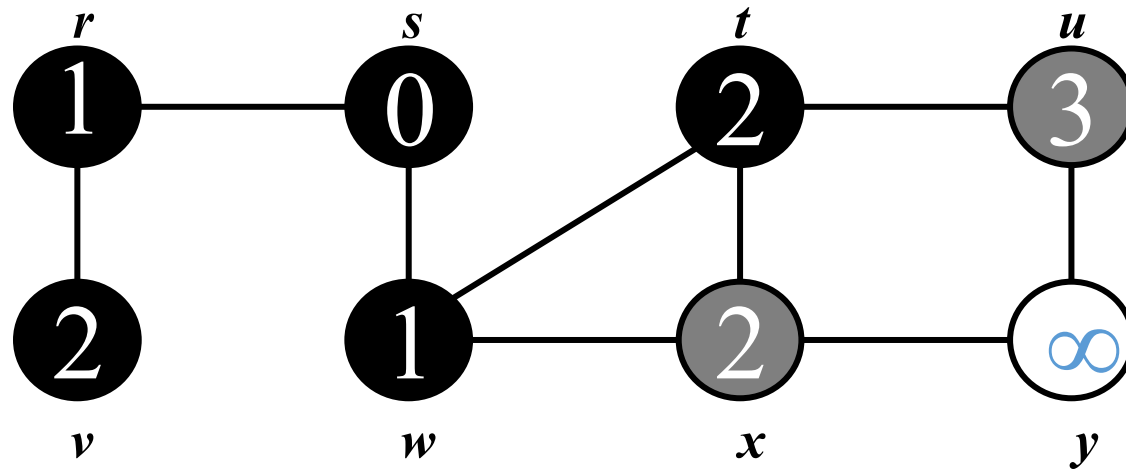
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

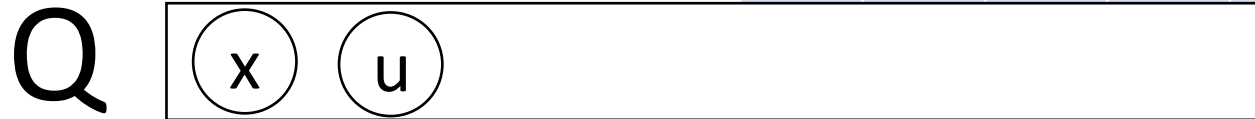


- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- ➔ • Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

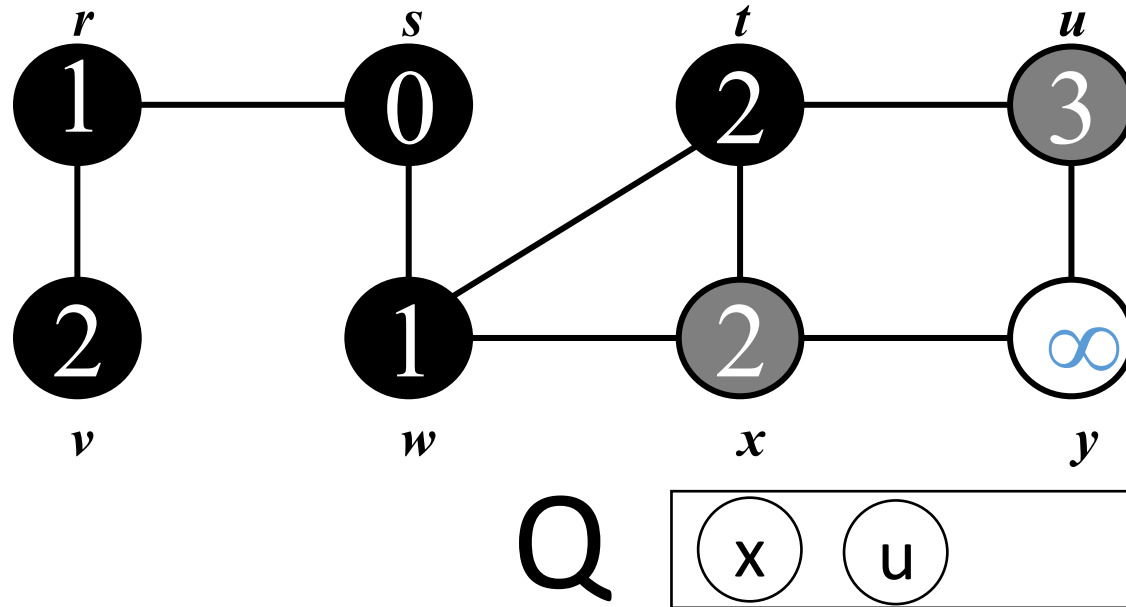


properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	inf
P	s	-1	w	t	1	s	w	-1
c	b	b	b	g	b	b	g	w



- ➔
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

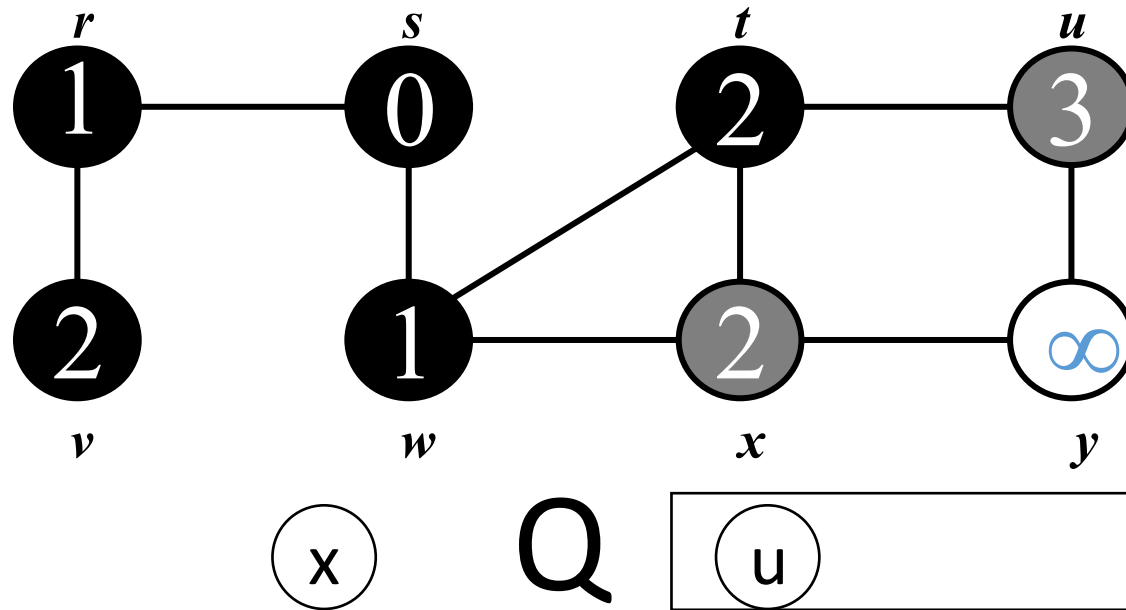
BFS Simulation



properties	r	s	t	u	v	w	x	y
d	1	0	2	3	2	1	2	inf
P	s	-1	w	t	1	s	w	-1
c	b	b	b	g	b	b	g	w

- Now keep on doing the following until the queue is empty
 - ➔ • Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

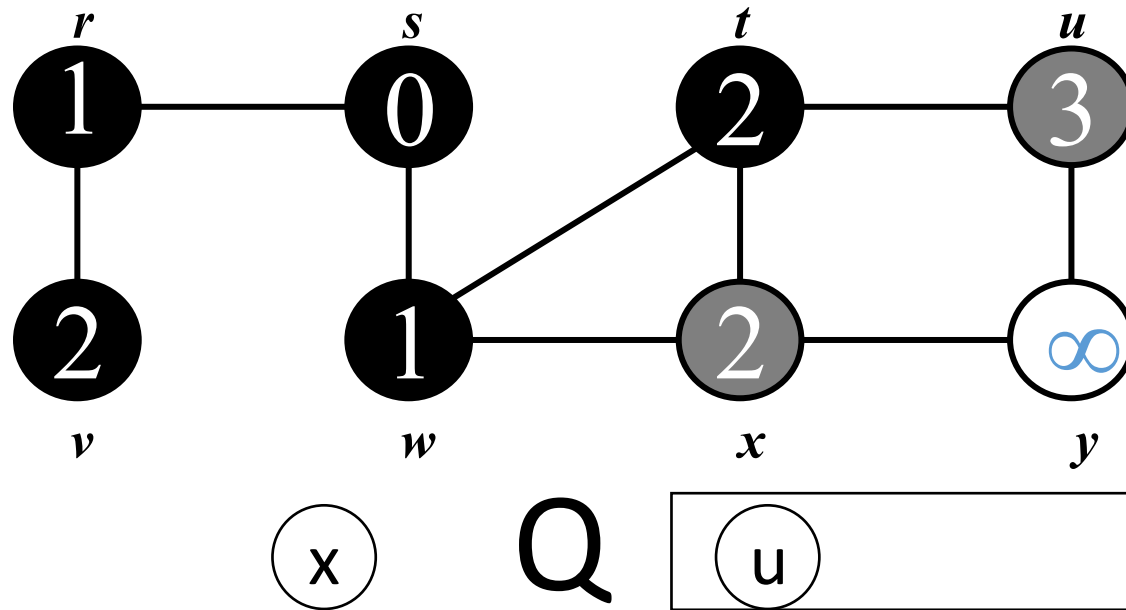
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	inf
P	s	-1	w	t	1	s	w	-1
c	b	b	b	g	b	b	g	w

- Now keep on doing the following until the queue is empty
 - ➔ • Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

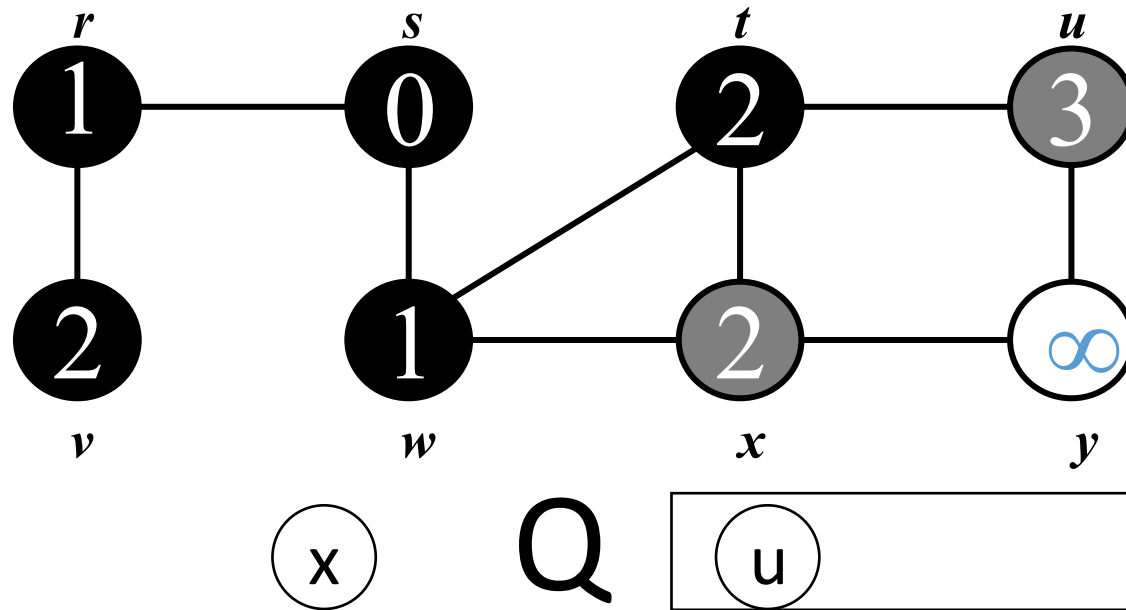
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	inf
P	s	-1	w	t	1	s	w	-1
c	b	b	b	g	b	b	g	w

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - ➔ • Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

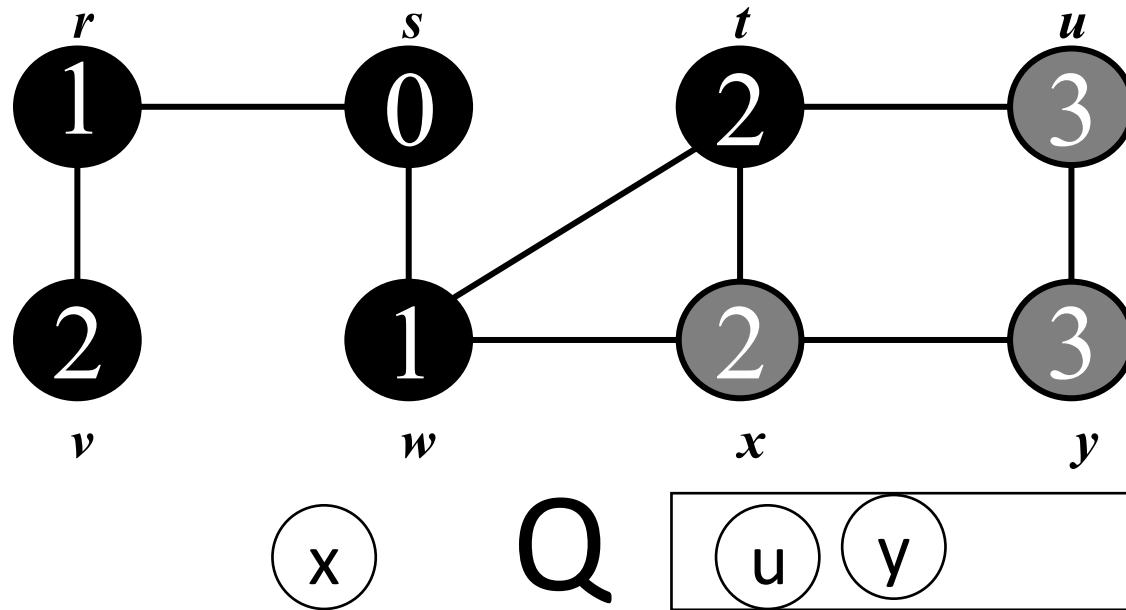
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	inf
P	s	-1	w	t	1	s	w	-1
c	b	b	b	g	b	b	g	w

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

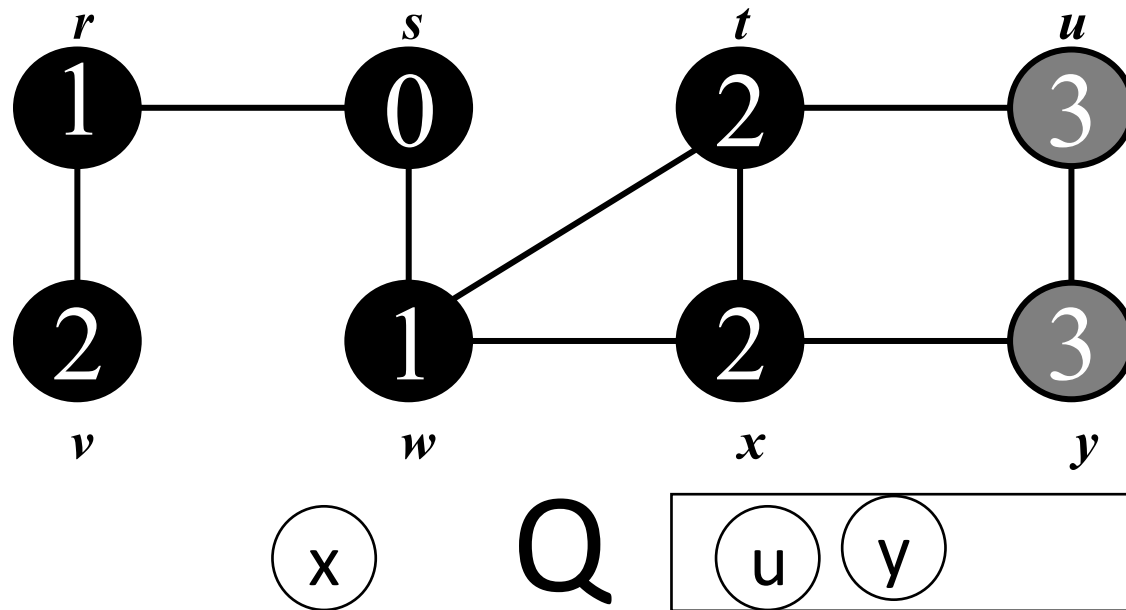
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	g	b	b	g	g

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

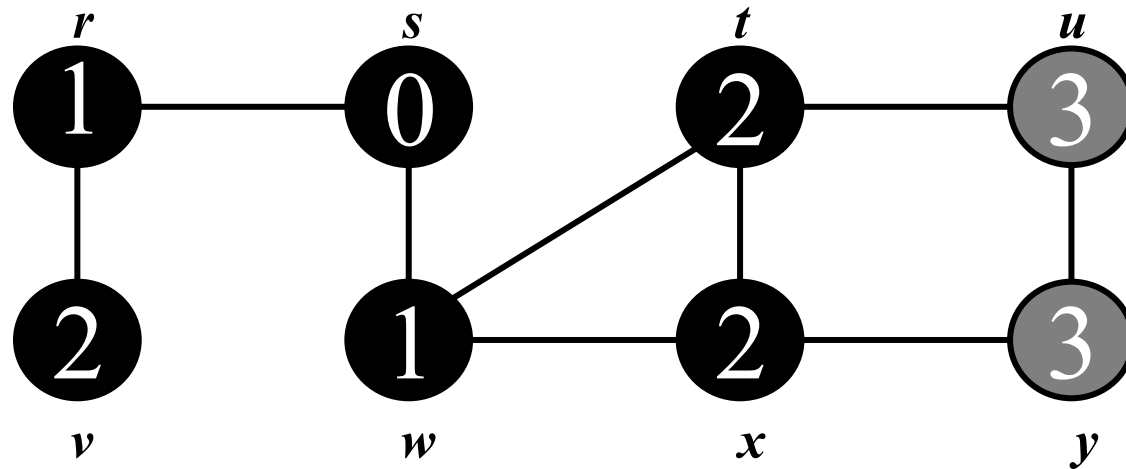
BFS Simulation



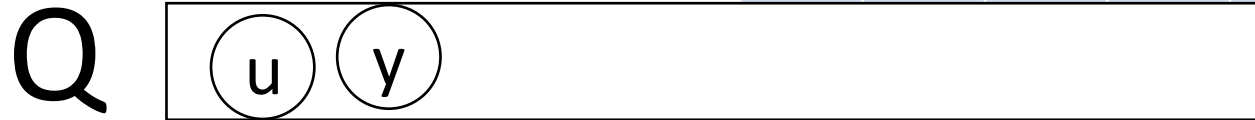
properties	r	s	t	u	v	w	x	y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	g	b	b	b	g

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- ➔ • Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

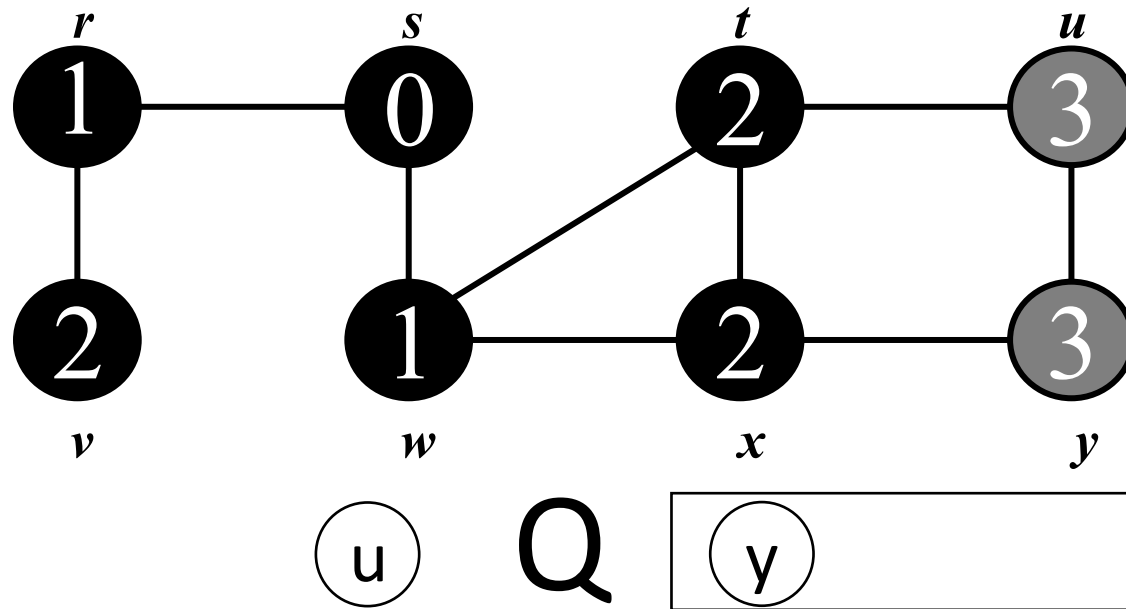


properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	g	b	b	b	g



- ➔
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

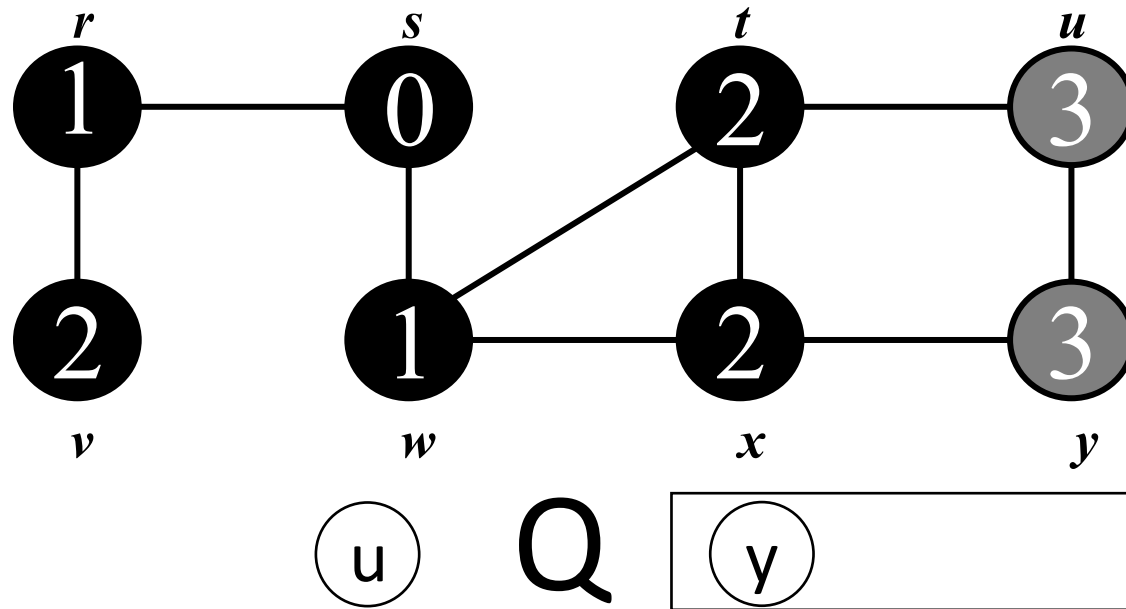
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	g	b	b	b	g

- Now keep on doing the following until the queue is empty
 - ➔ • Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

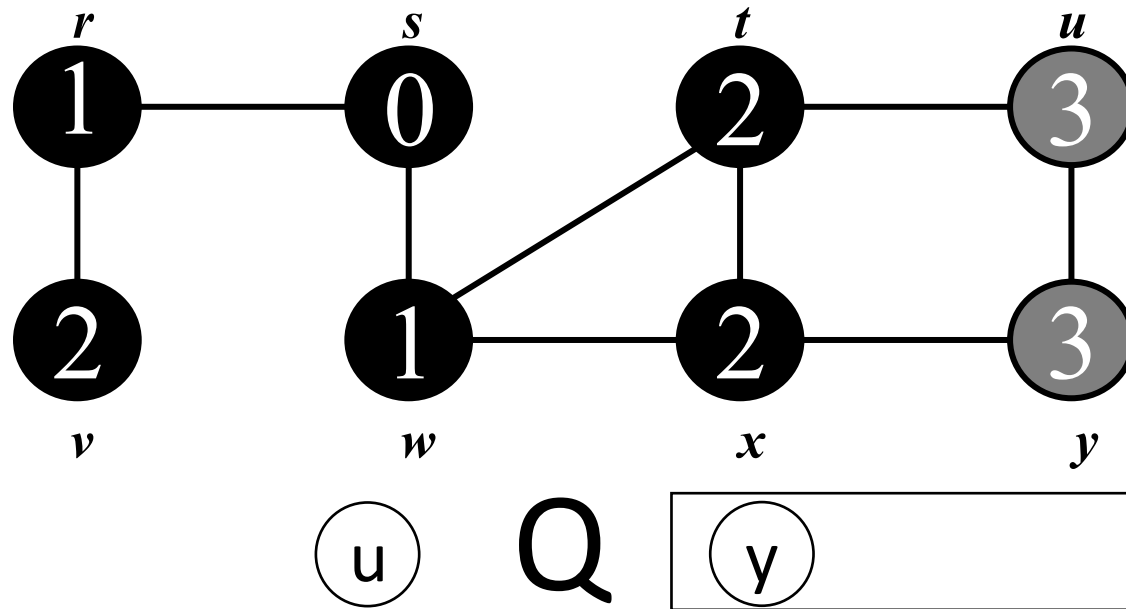
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	g	b	b	b	g

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

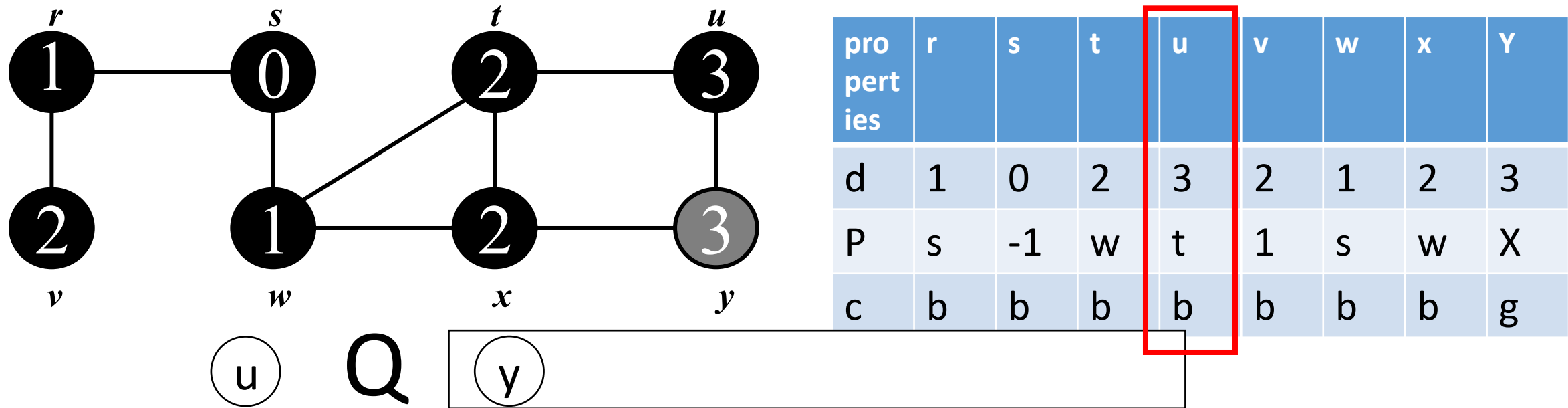
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	g	b	b	b	g

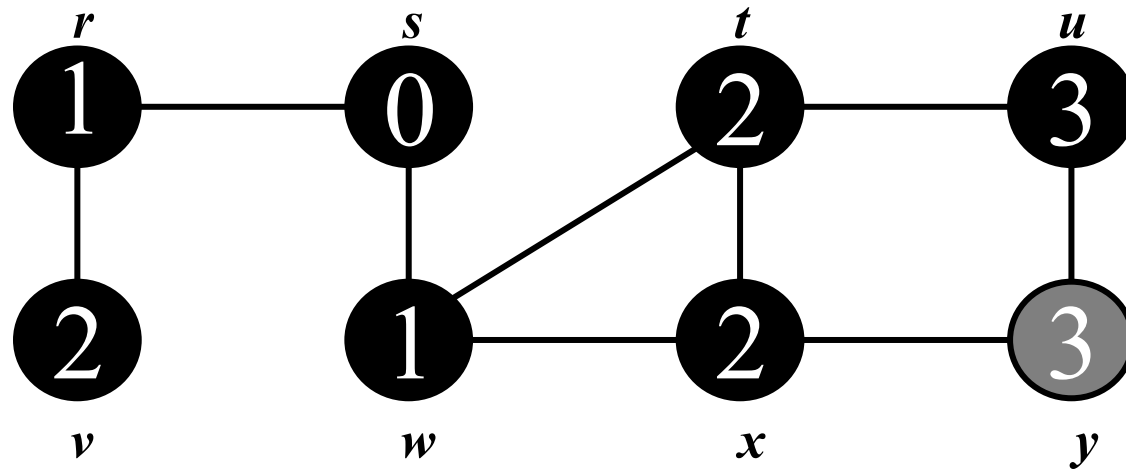
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - ➔ • If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

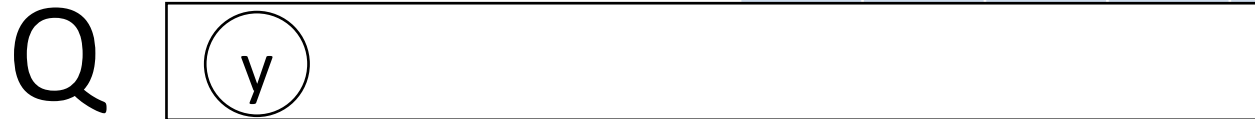


- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- ➡ • Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

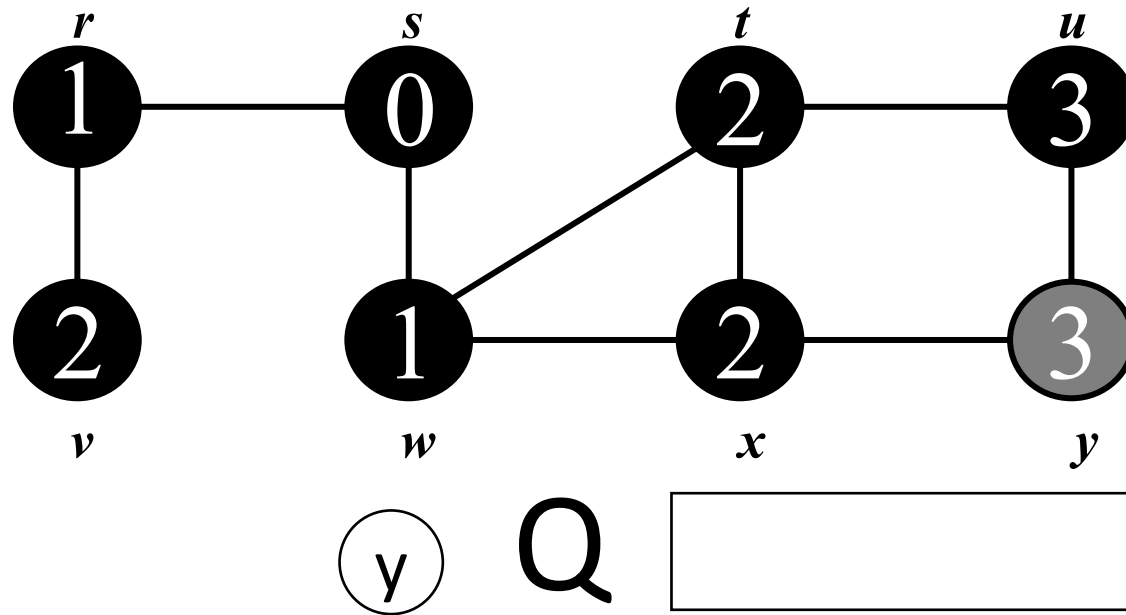


properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	b	b	b	b	g



- ➔
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

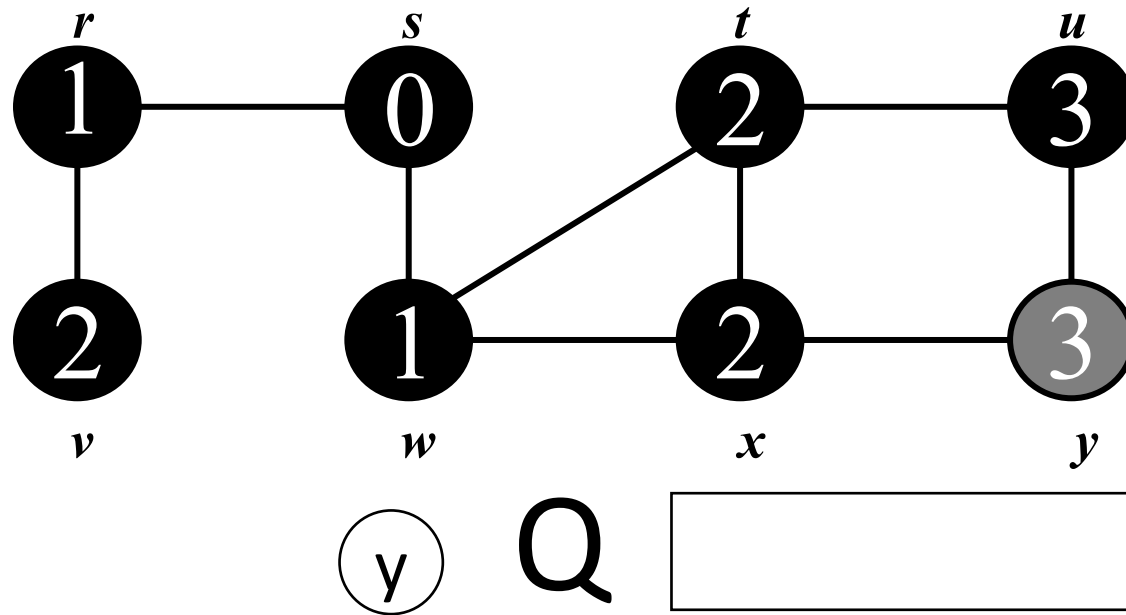
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	b	b	b	b	g

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

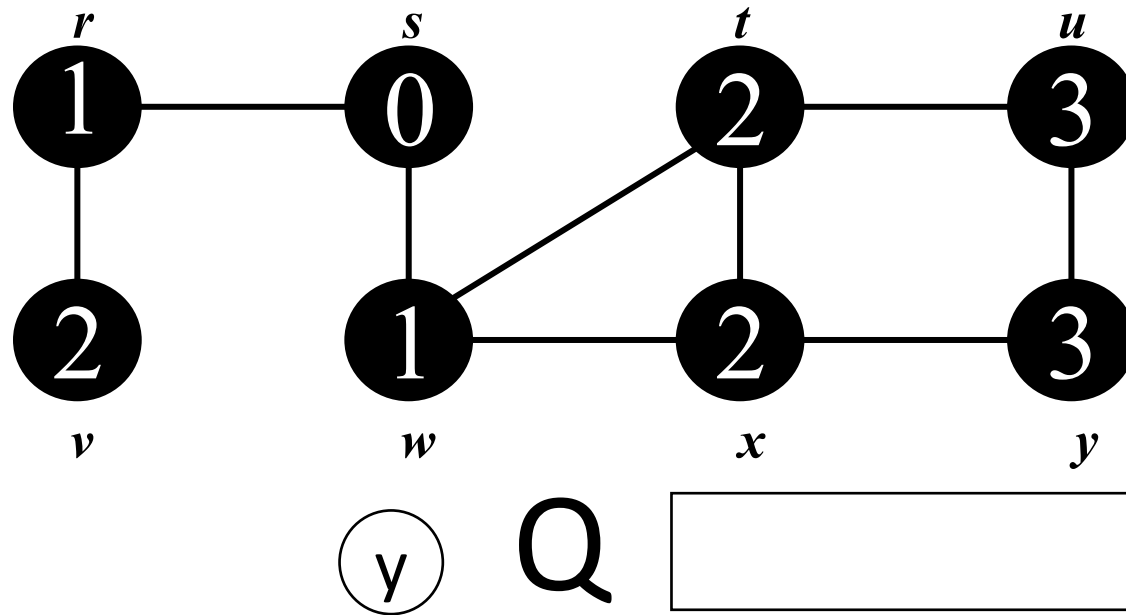
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	b	b	b	b	g

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - ➔ • Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

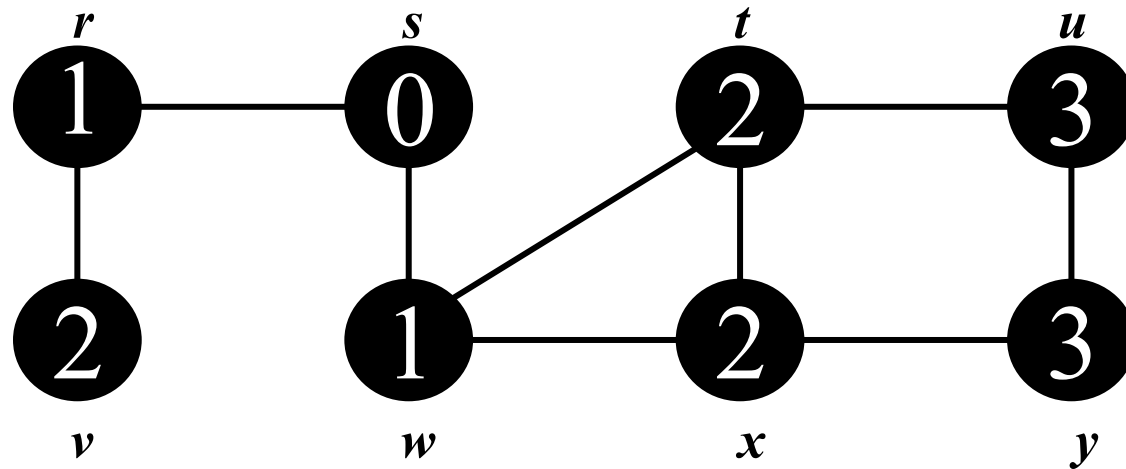
BFS Simulation



properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	b	b	b	b	g

- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
- ➔ • Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation

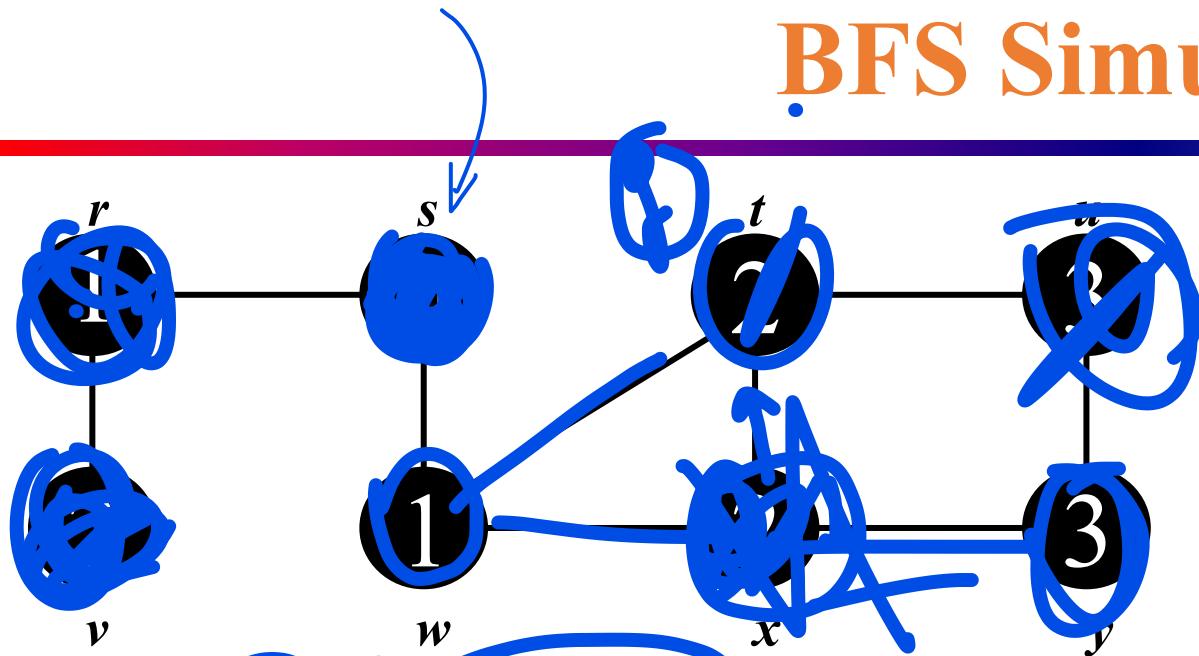


properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	b	b	b	b	g

Q

- ➔
- Now keep on doing the following until the queue is empty
 - Take out the topmost element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)

BFS Simulation



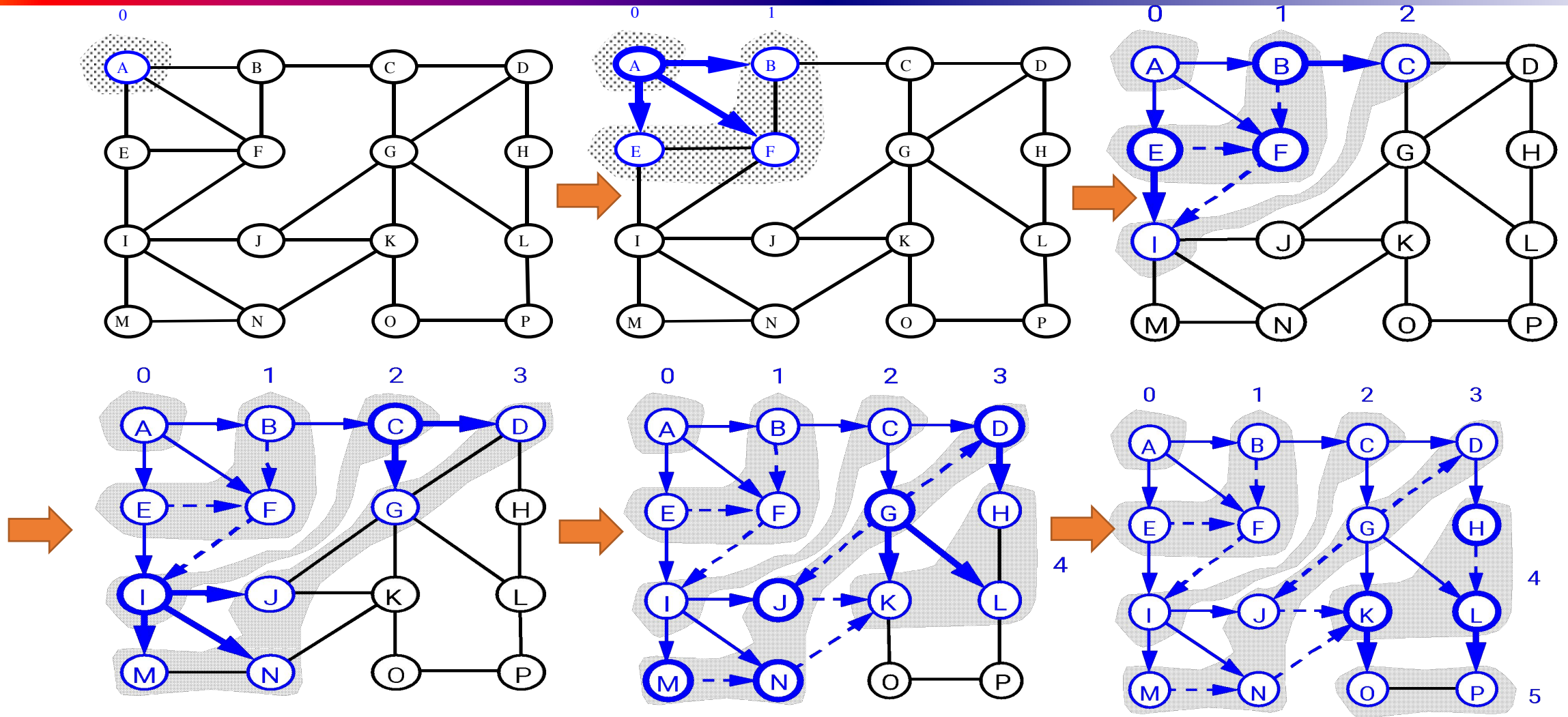
properties	r	s	t	u	v	w	x	Y
d	1	0	2	3	2	1	2	3
P	s	-1	w	t	1	s	w	X
c	b	b	b	b	b	b	b	g

Q

- Now keep on doing the following until the queue is empty
 - Take out the ~~topmost~~ element
 - Visit each of its neighbors
 - If the neighbors are undiscovered (white)
 - Mark them as discovered (color them with GRAY color and place them in queue)
 - Update their distance, parent.
 - Mark the taken out element as explored (Black – all its children has been discovered)



BFS Discovery Pattern



Breadth-First Search: Properties

- BFS calculates the *shortest-path along with its distance* to the **source node**
 - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v , or ∞ if v not reachable from s
- BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G
 - Thus can use BFS to calculate shortest path from one vertex to another in **$O(V+E)$** time in an unweighted tree

Let us Formulate the BFS Pseudocode

BFS(G, s)

1 **for** each vertex $u \in V[G] - \{s\}$

2 **do** $color[u] \leftarrow \text{WHITE}$

3 $d[u] \leftarrow \infty$

4 $\pi[u] \leftarrow \text{NIL}$

5 $color[s] \leftarrow \text{GRAY}$

6 $d[s] \leftarrow 0$

7 $\pi[s] \leftarrow \text{NIL}$

8 $Q \leftarrow \emptyset$

9 $\text{ENQUEUE}(Q, s)$

10 **while** $Q \neq \emptyset$

11 **do** $u \leftarrow \text{DEQUEUE}(Q)$

12 **for** each $v \in \text{Adj}[u]$

13 **do if** $color[v] = \text{WHITE}$

14 **then** $color[v] \leftarrow \text{GRAY}$

15 $d[v] \leftarrow d[u] + 1$

16 $\pi[v] \leftarrow u$

17 $\text{ENQUEUE}(Q, v)$

18 $color[u] \leftarrow \text{BLACK}$

$s = \text{Root}$

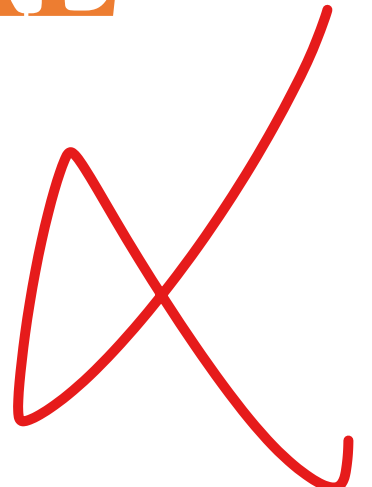
making all nodes properly null
distance

Color

Parent

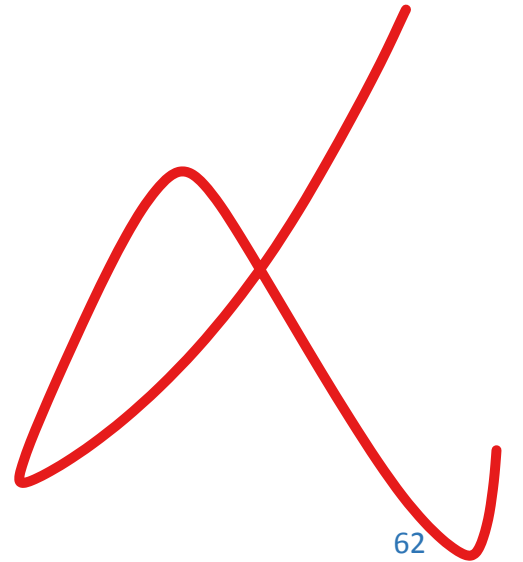
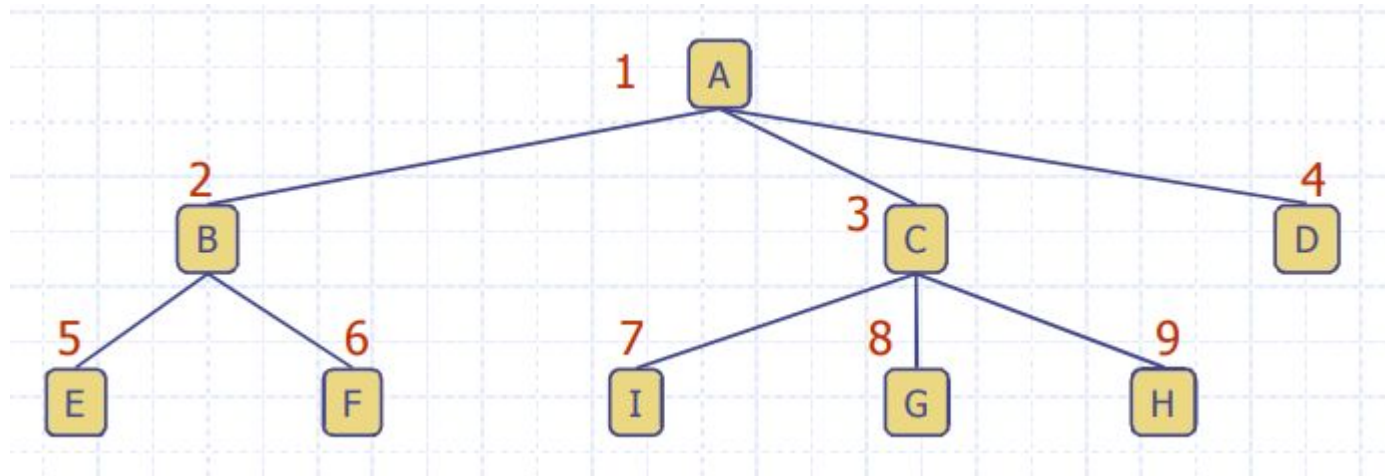
Queue

LEVEL ORDER TRAVERSAL

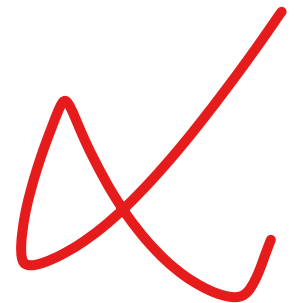


Level order traversal

- Run BFS from the root
- Print the node when you color it black



DEPTH FIRST SEARCH



Depth First Search

- We can simply replace the **queue** with a **stack**.
- However, this time we want to work with two other attributes:
 - Discovery Time
 - Finish Time
- Hence we are using the recursive version.
 - You will understand this after the simulation.

BFS(G, s)

```
1  for each vertex  $u \in V[G] - \{s\}$ 
2      do  $color[u] \leftarrow WHITE$ 
3          $d[u] \leftarrow \infty$ 
4          $\pi[u] \leftarrow NIL$ 
5   $color[s] \leftarrow GRAY$ 
6   $d[s] \leftarrow 0$ 
7   $\pi[s] \leftarrow NIL$ 
8   $Q \leftarrow \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11     do  $u \leftarrow DEQUEUE(Q)$ 
12        for each  $v \in Adj[u]$ 
13            do if  $color[v] = WHITE$ 
14                then  $color[v] \leftarrow GRAY$ 
15                    $d[v] \leftarrow d[u] + 1$ 
16                    $\pi[v] \leftarrow u$ 
17                   ENQUEUE( $Q, v$ )
18      $color[u] \leftarrow BLACK$ 
```


Depth-First Search

- **Depth-first search is another strategy for exploring a graph**
 - Explore “deeper” in the graph whenever possible
 - Edges are explored out of the **most recently discovered** vertex v that still has unexplored edges
 - When all of v 's edges have been explored, **backtrack** to the vertex from which v was discovered (**Recursion is used here**)
- **Vertex Colors:**
 - Vertices initially colored white
 - Then colored grey when discovered
 - Then black when finished
- **Times**
 - Discovery Time (d)
 - Returning Time / Exploration Completion Time (f)
- **Running time**
 - running time of DFS = $O(V+E)$

DFS Simulation

- Consider two functions:

```
DFS (G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(v);
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```

DFS Simulation

- Consider two functions:

```
DFS (G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(u);
    }
}
```

1. Initialization

2. Ensures that visiting done for all vertices

```
DFS_Visit(u)
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(v);
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```

DFS Simulation

- Consider two functions:

```
DFS (G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(u);
    }
}
```

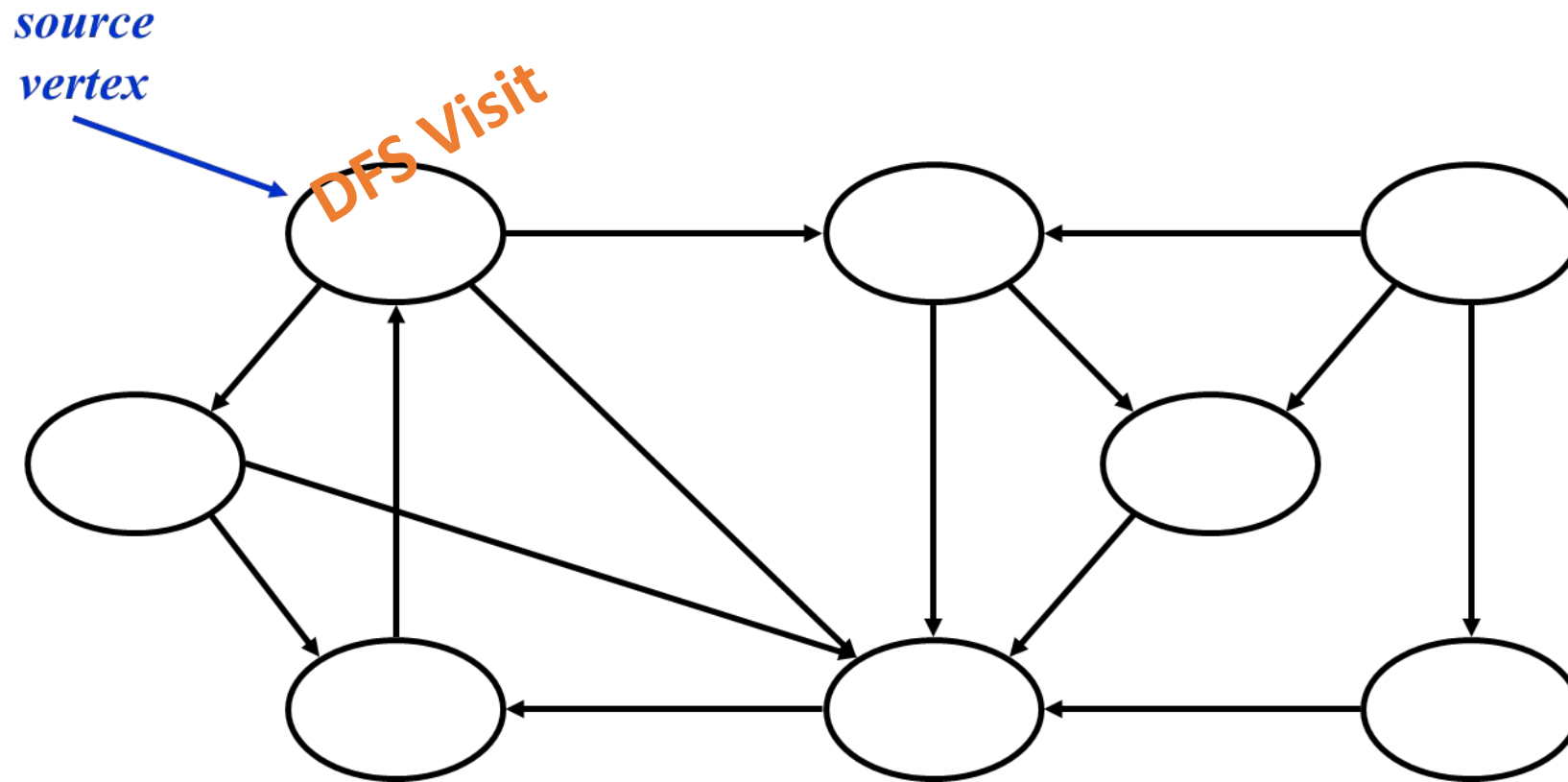
1. Initialization

2. Ensures that visiting done for all vertices

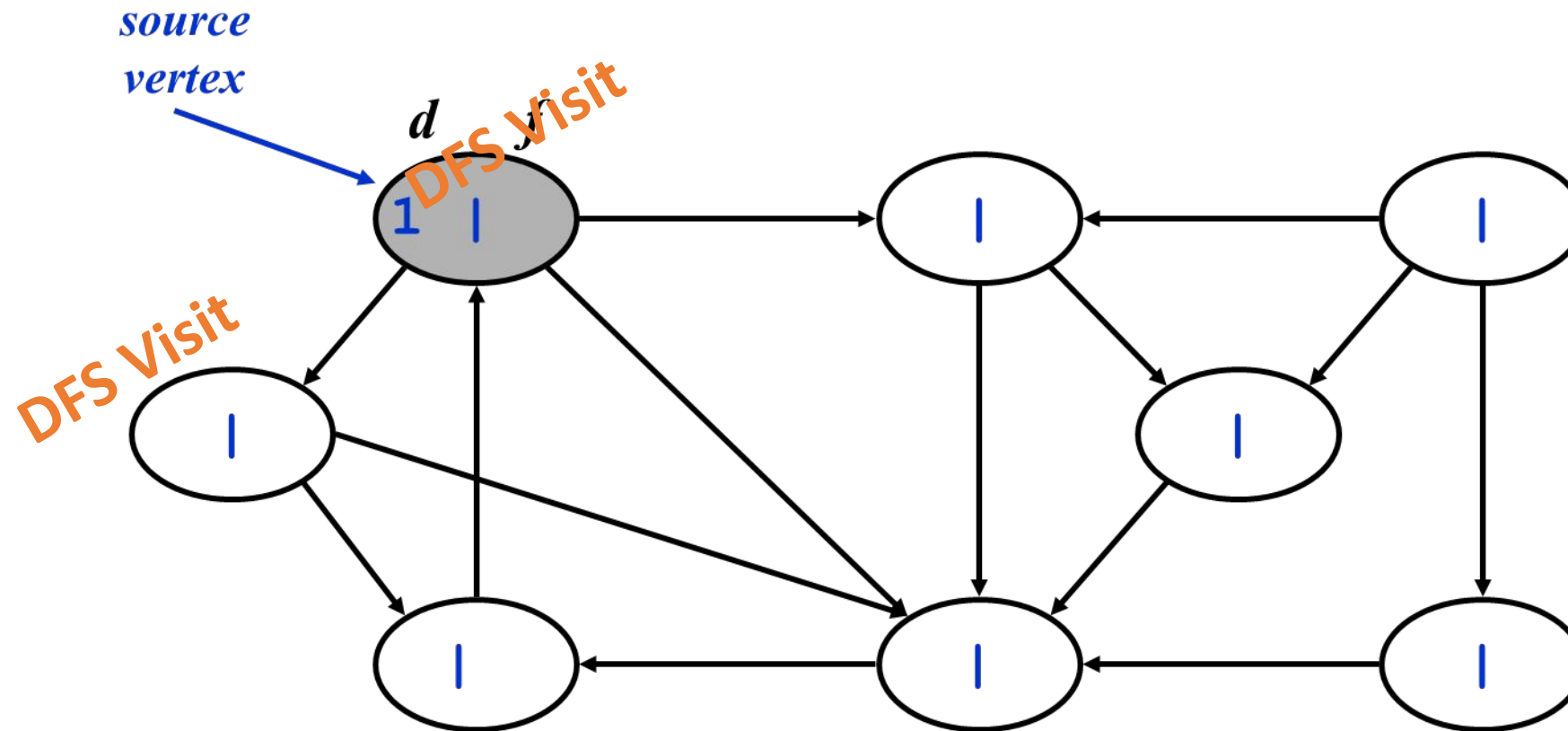
```
DFS_Visit(u)
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(v);
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```

1. Visits a vertex
2. Updates its color and discovery time
3. **Recursively** visits the white neighbors
4. Updates the color and closure time

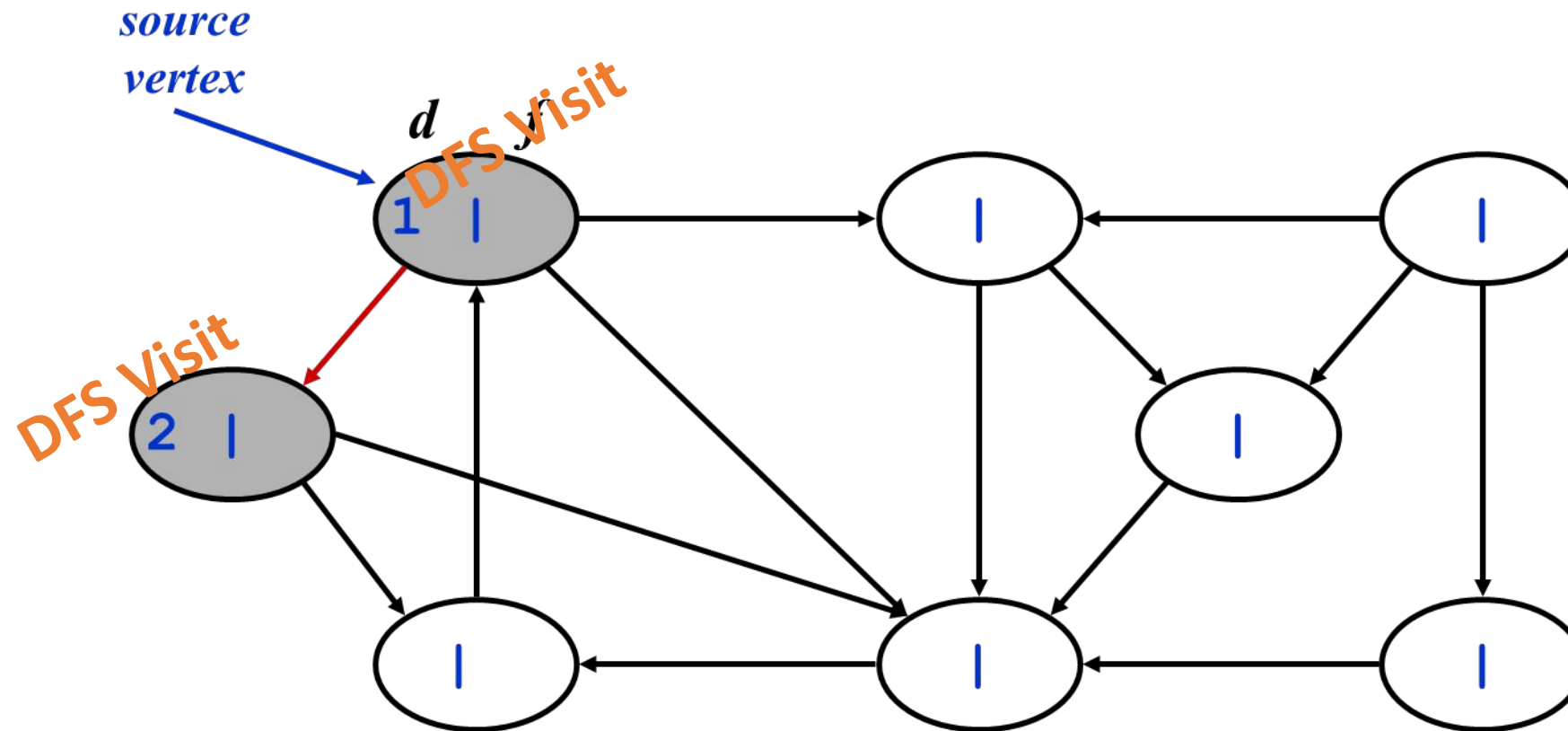
DFS Simulation



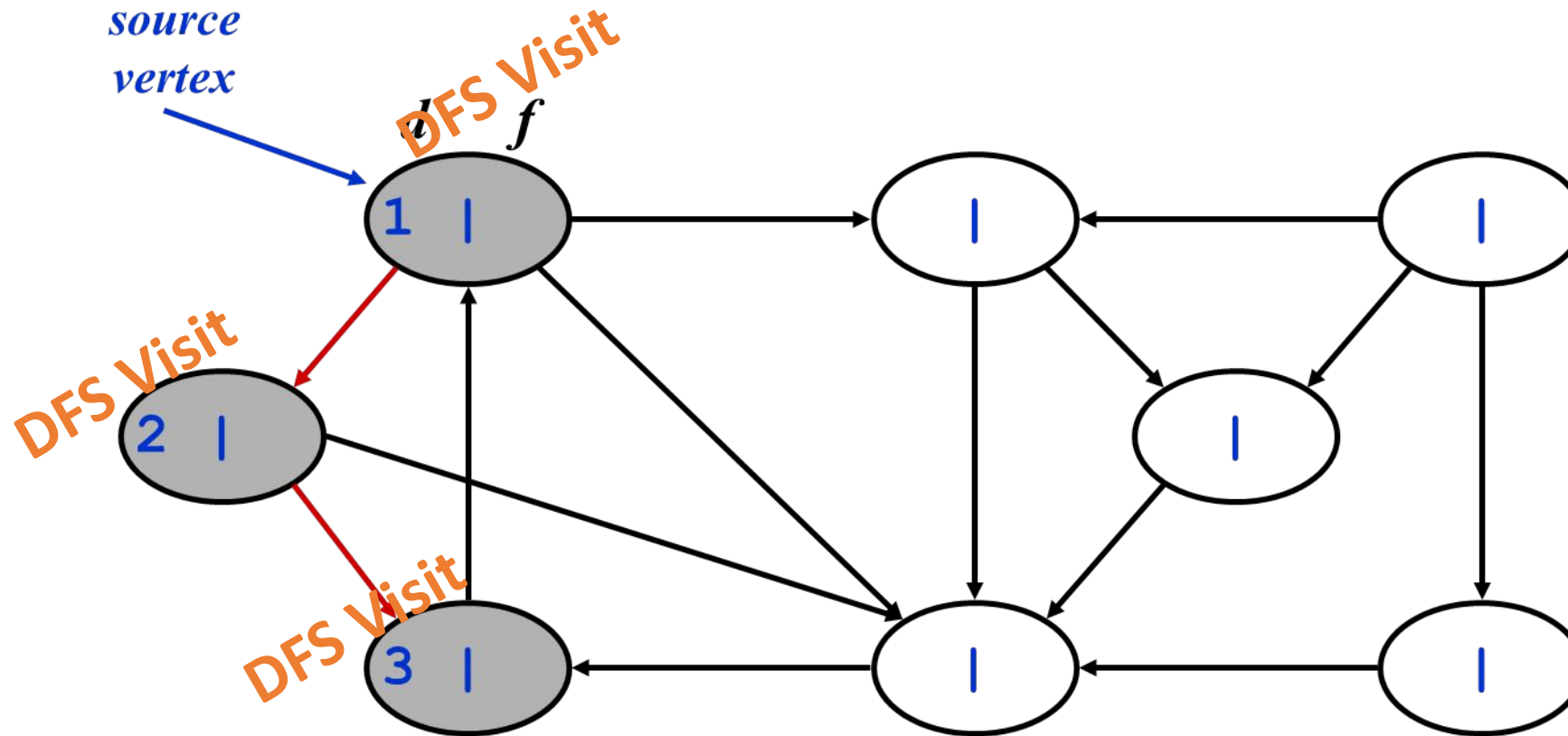
DFS Simulation



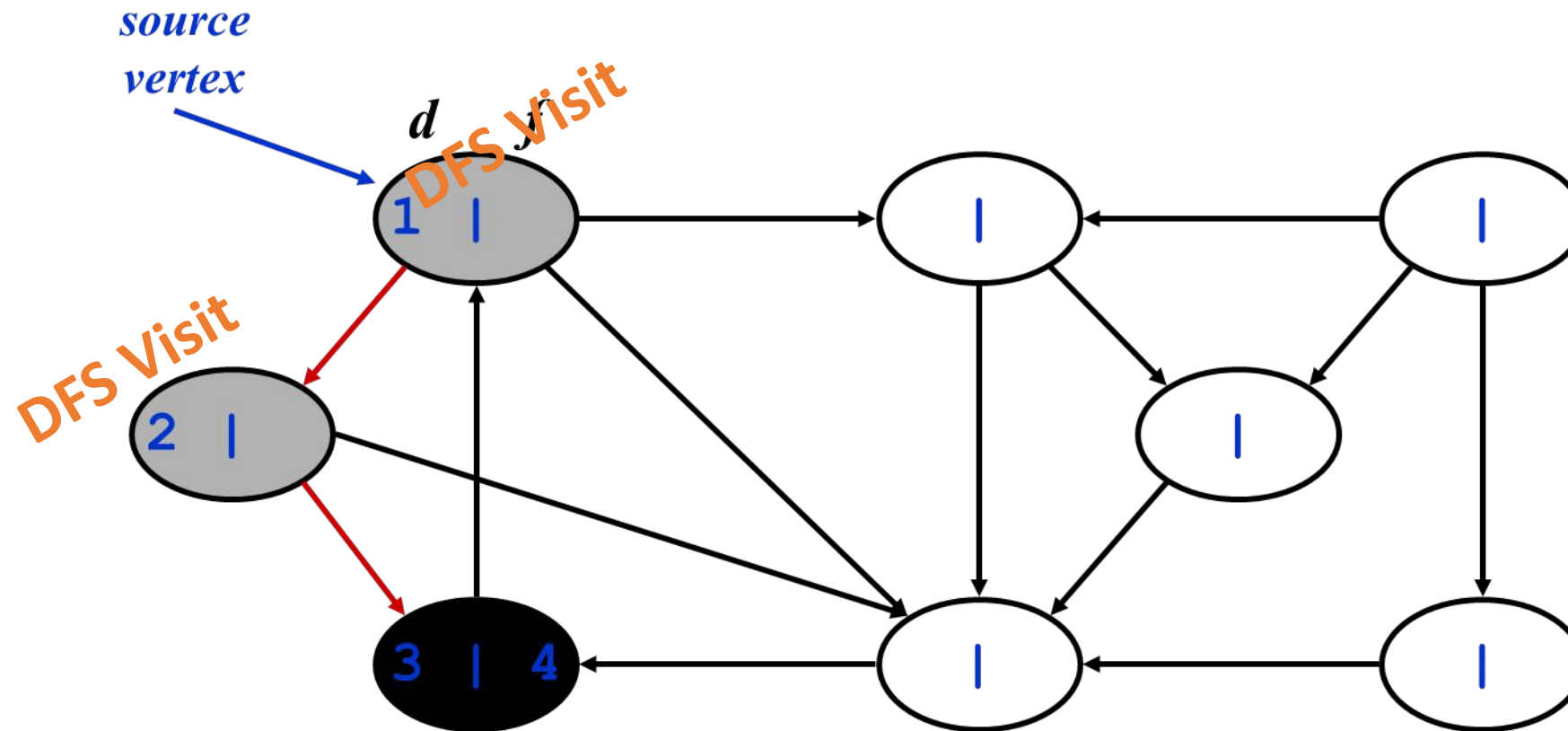
DFS Simulation



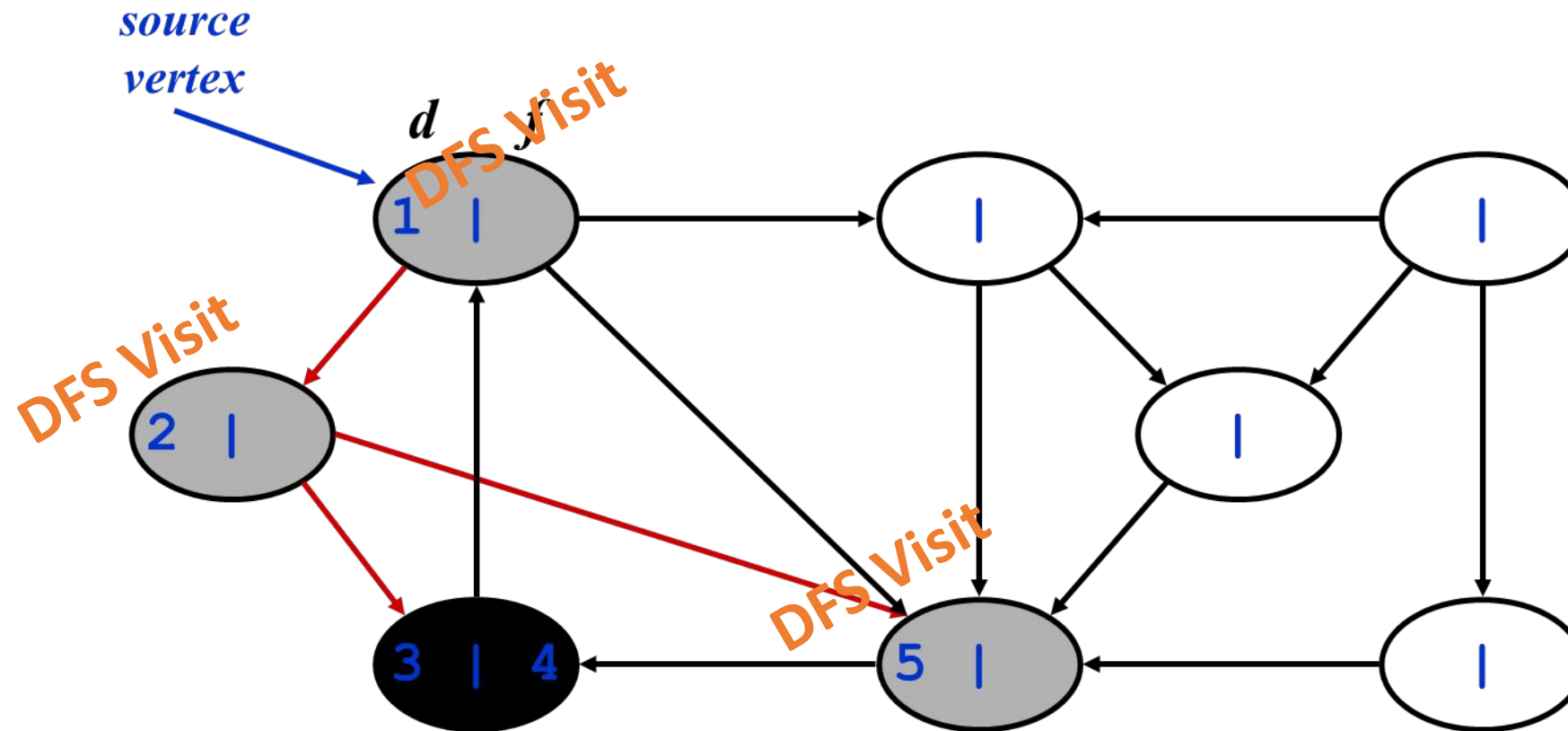
DFS Simulation



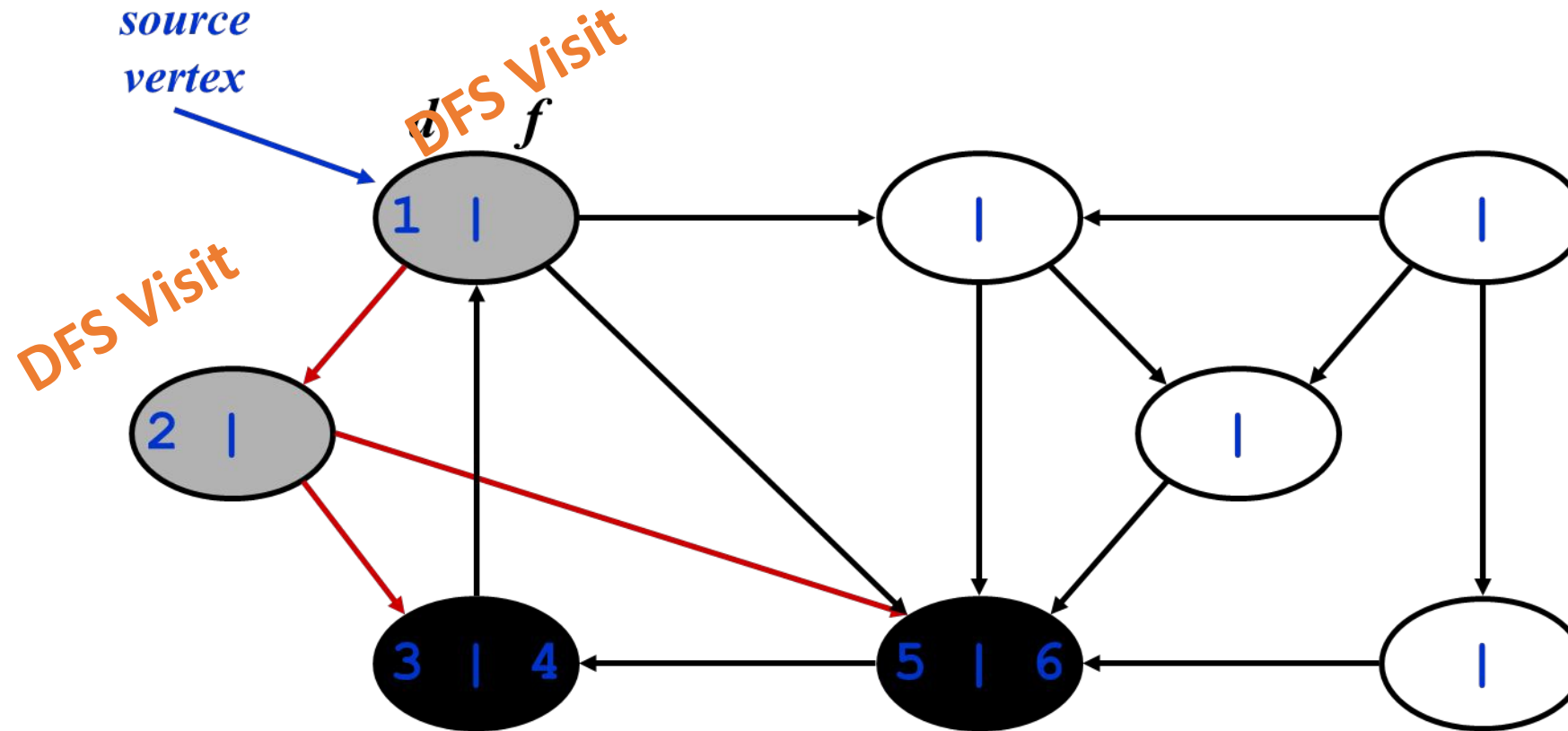
DFS Simulation



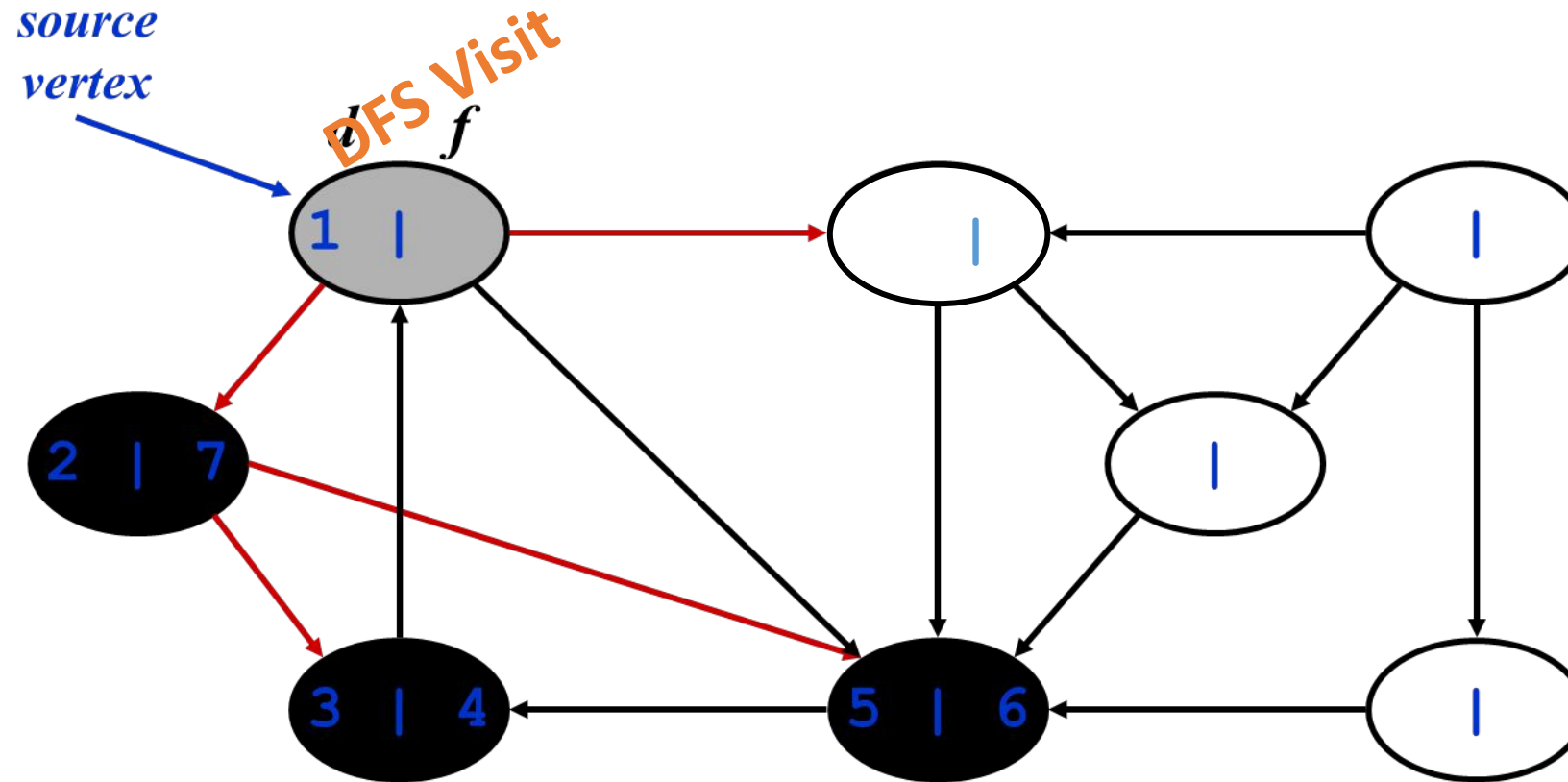
DFS Simulation



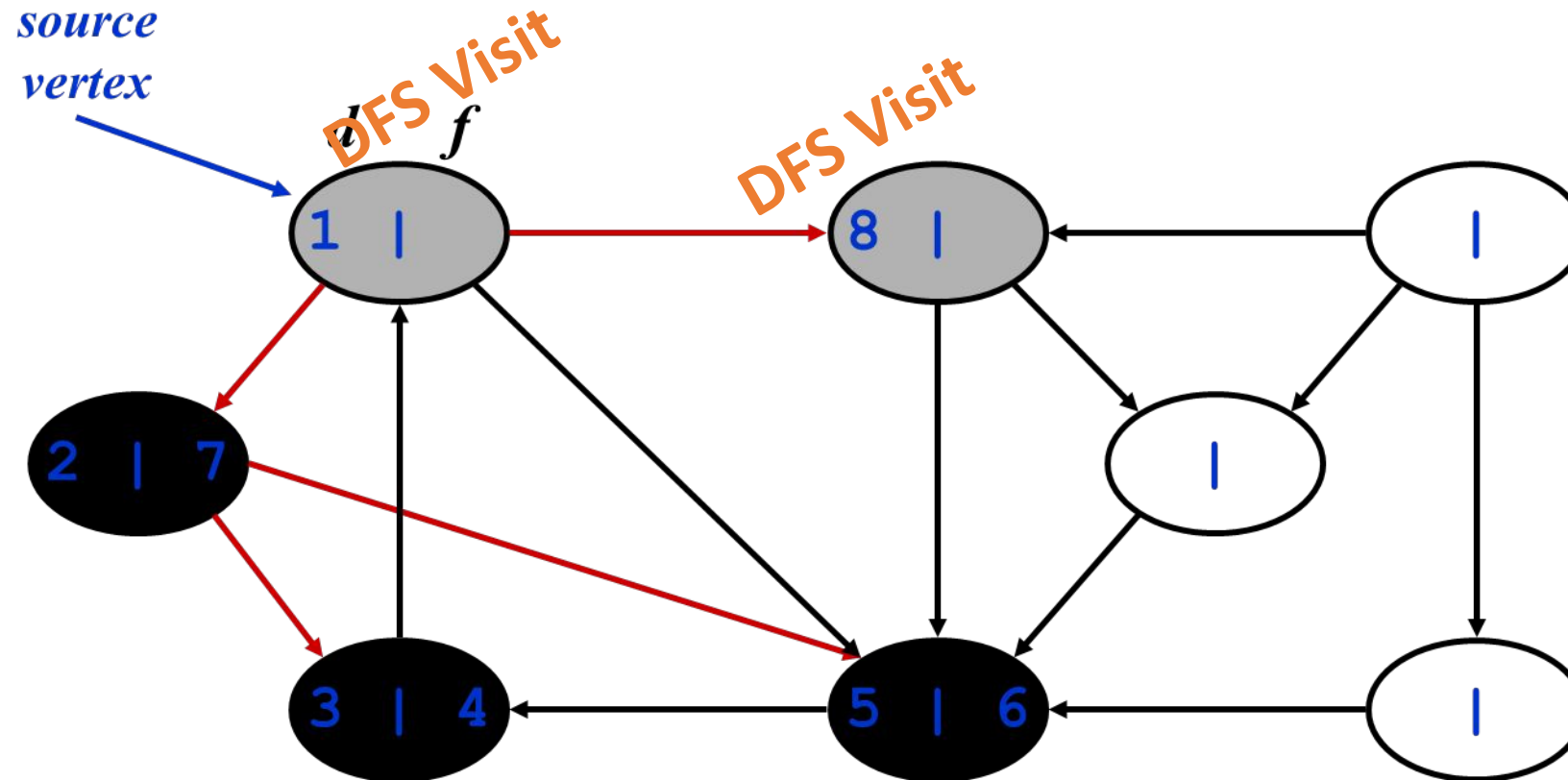
DFS Simulation



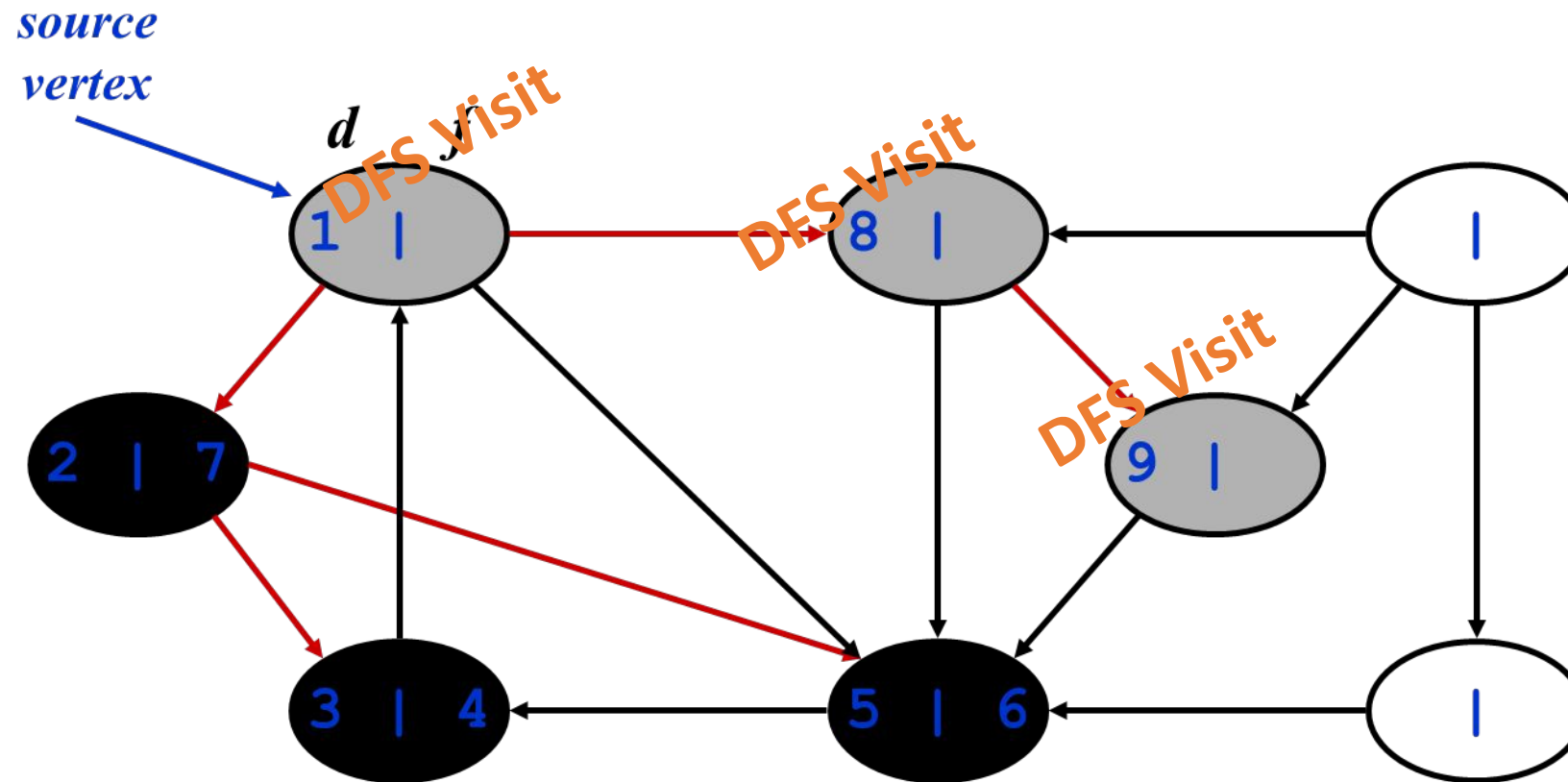
DFS Simulation



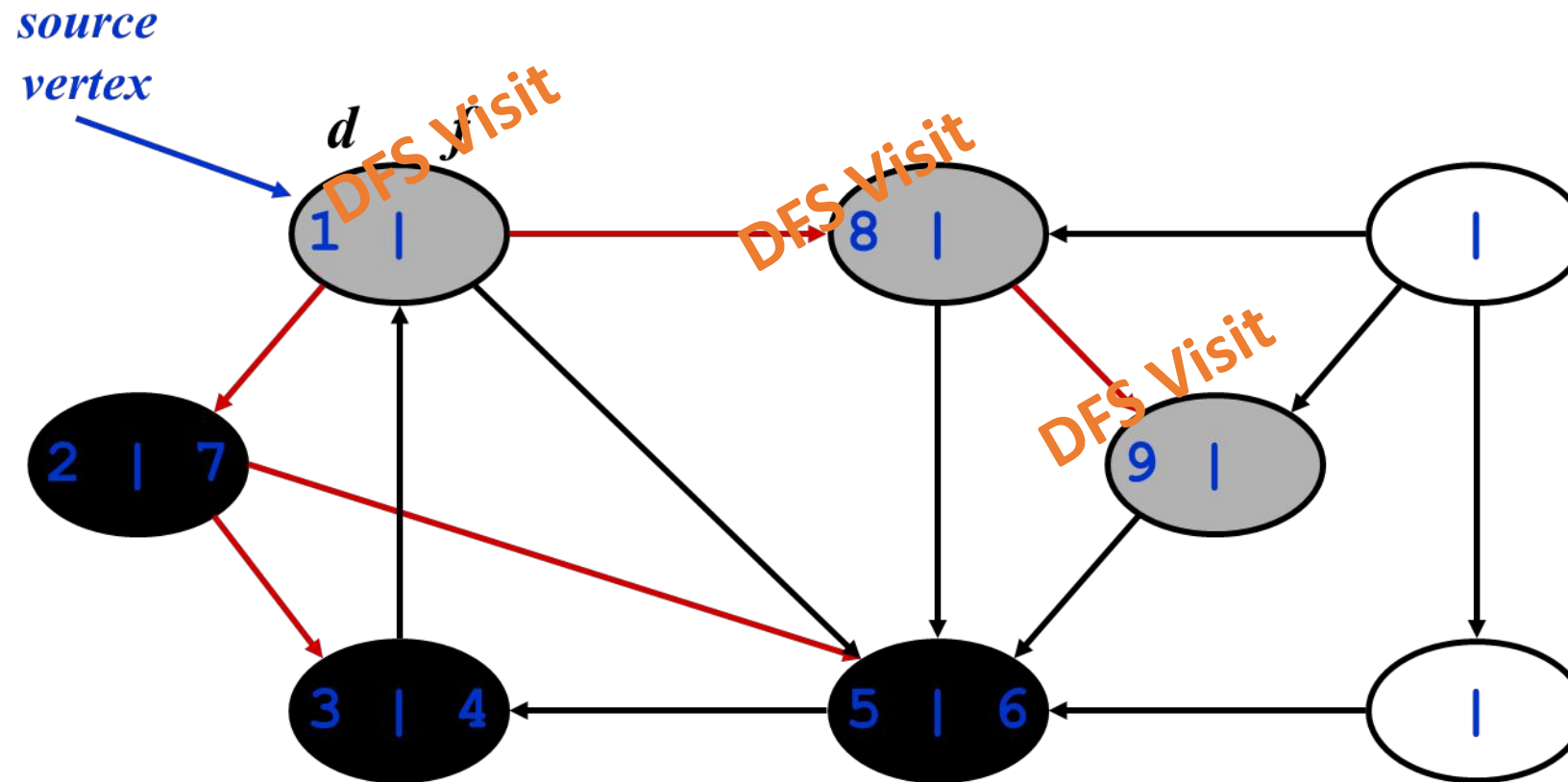
DFS Simulation



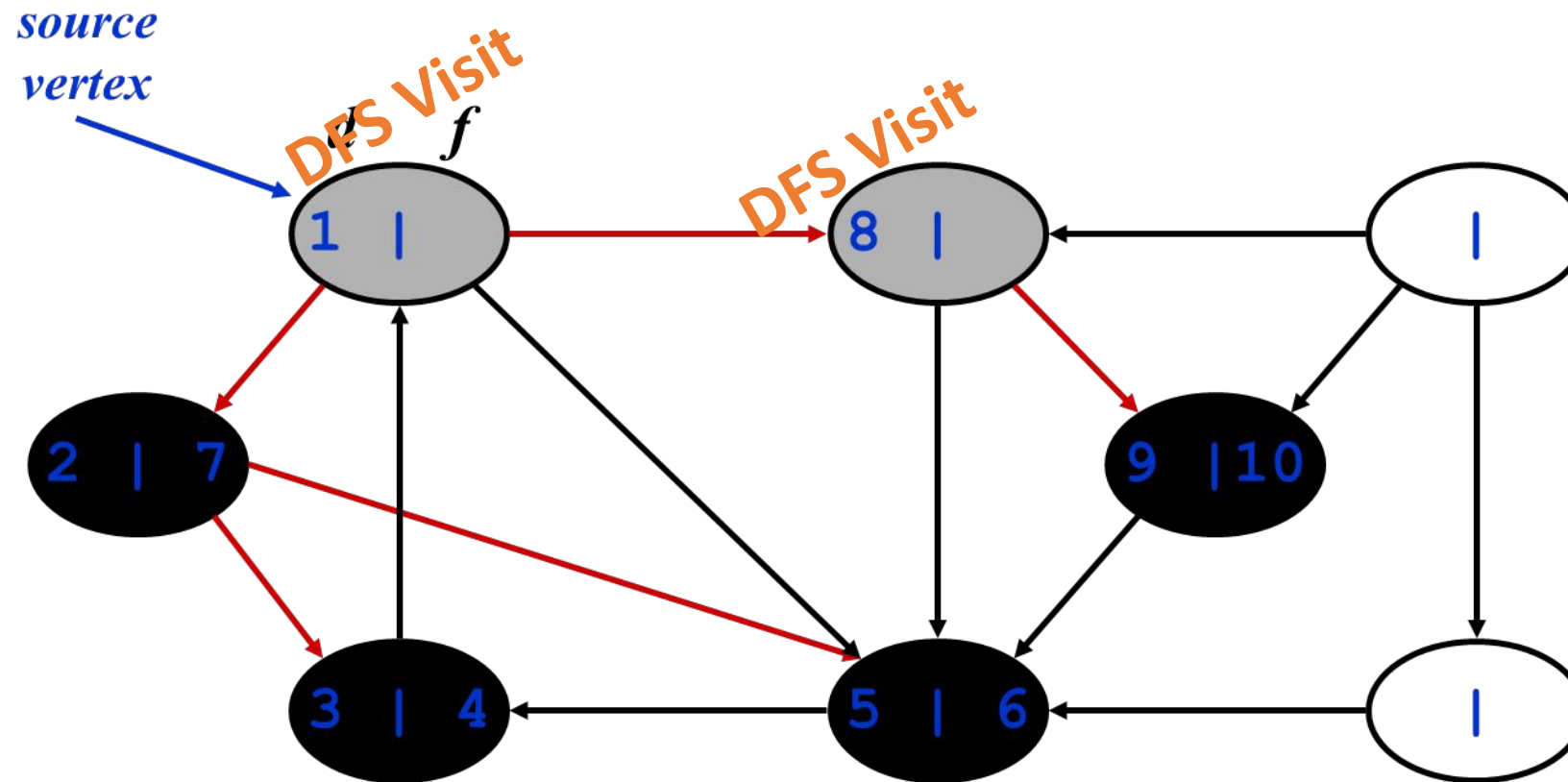
DFS Simulation



DFS Simulation

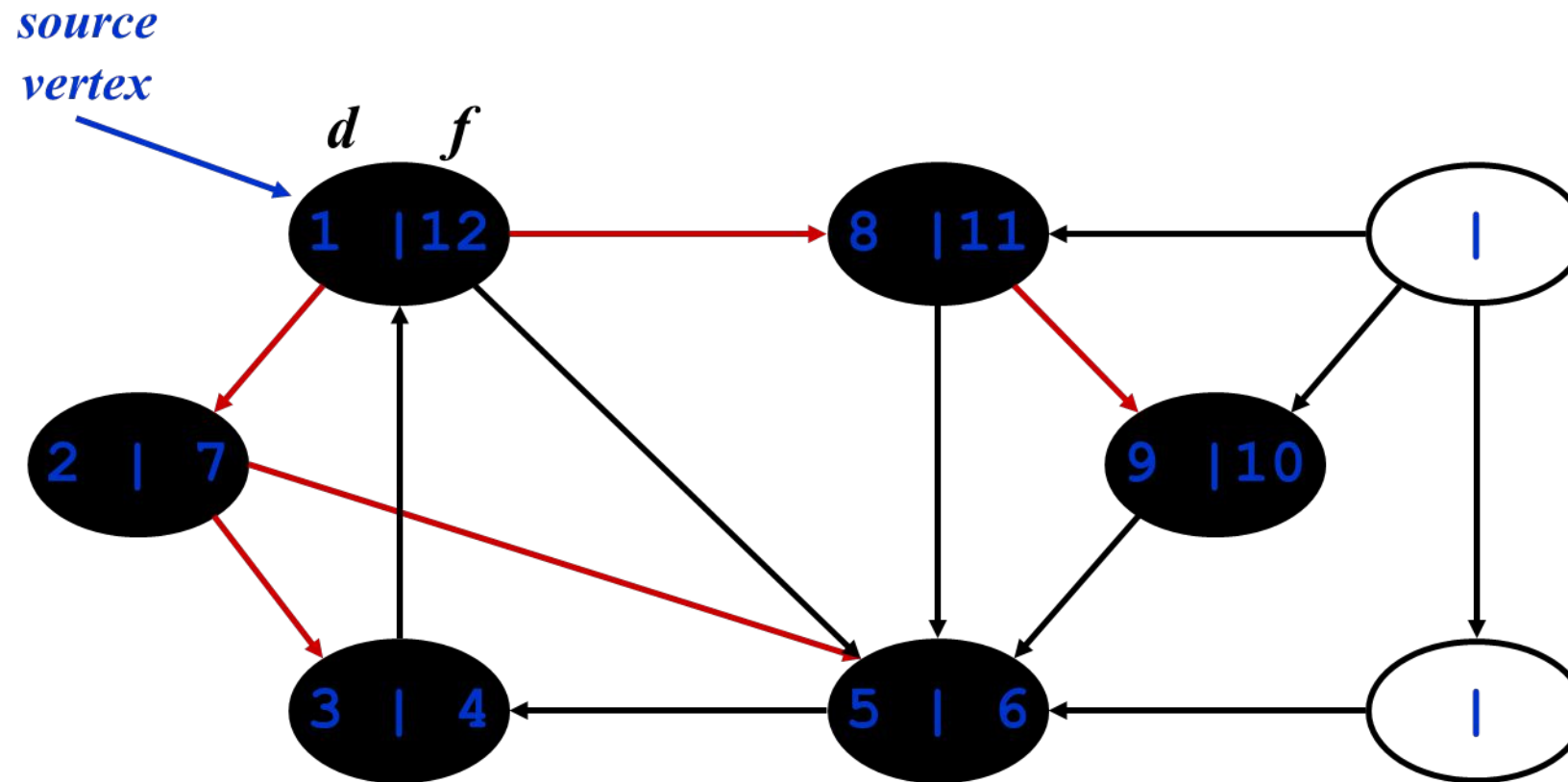


DFS Simulation





DFS Simulation



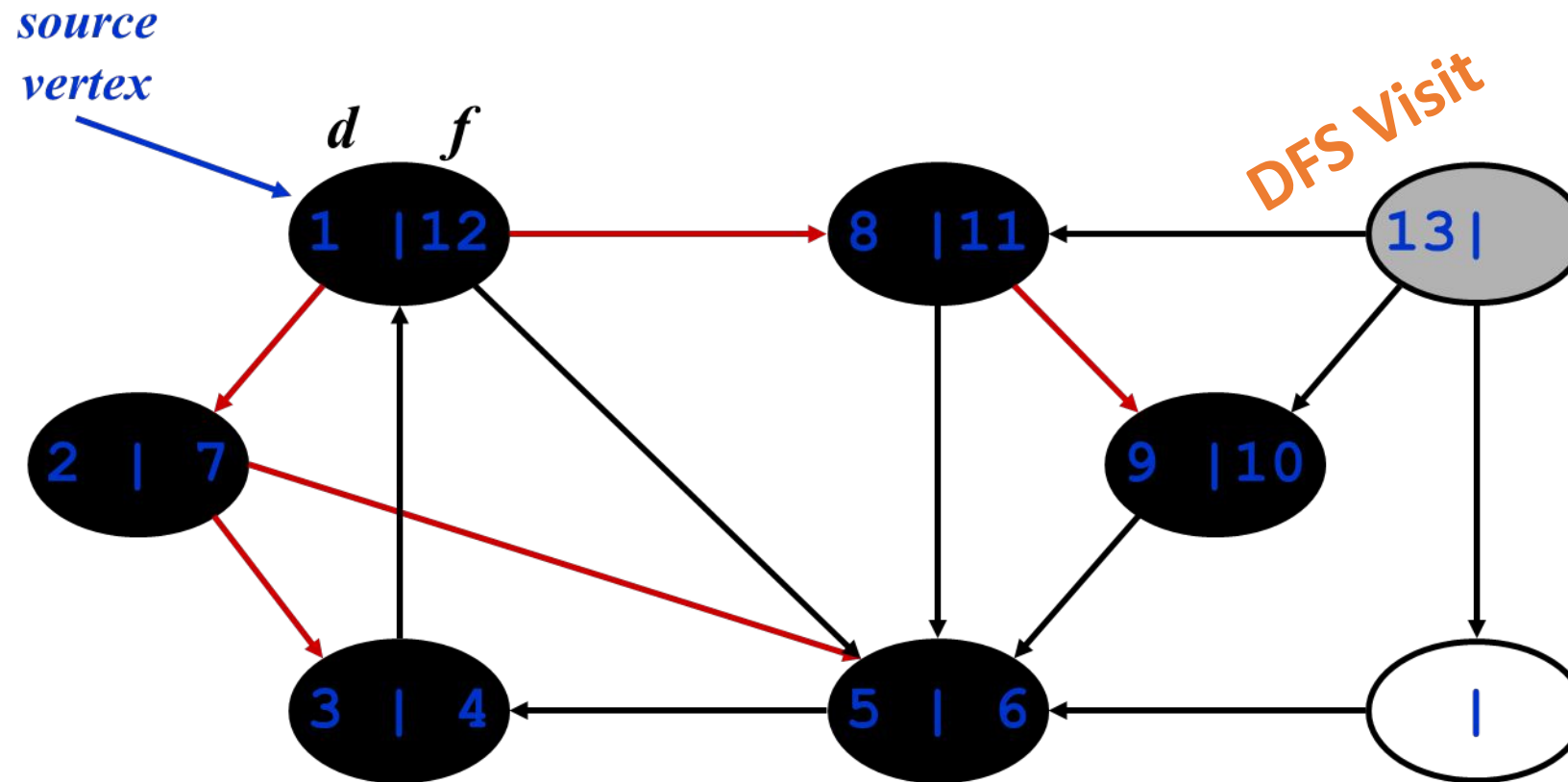
DFS Simulation

- Recall these two functions:

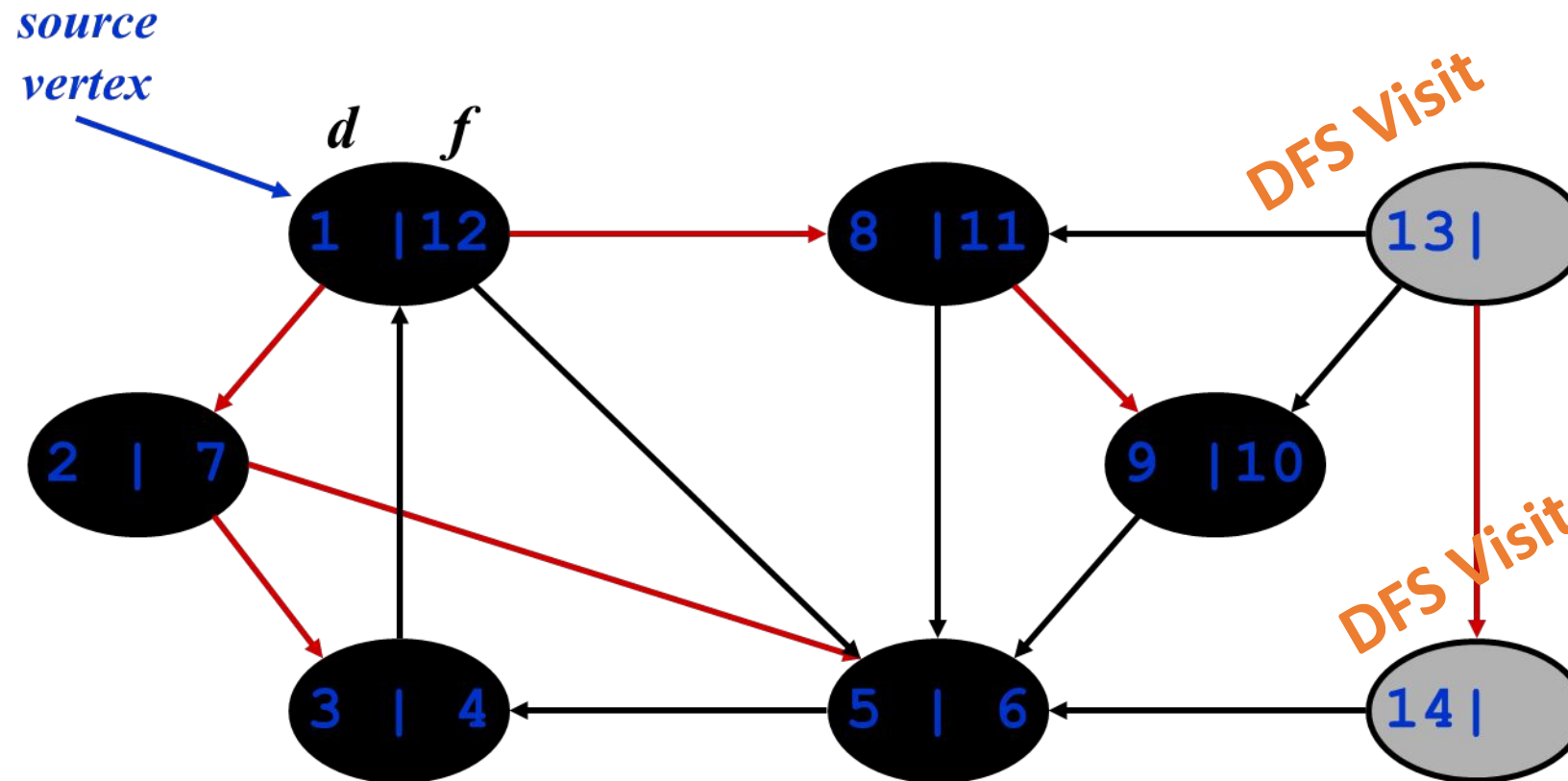
```
DFS (G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(v);
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```

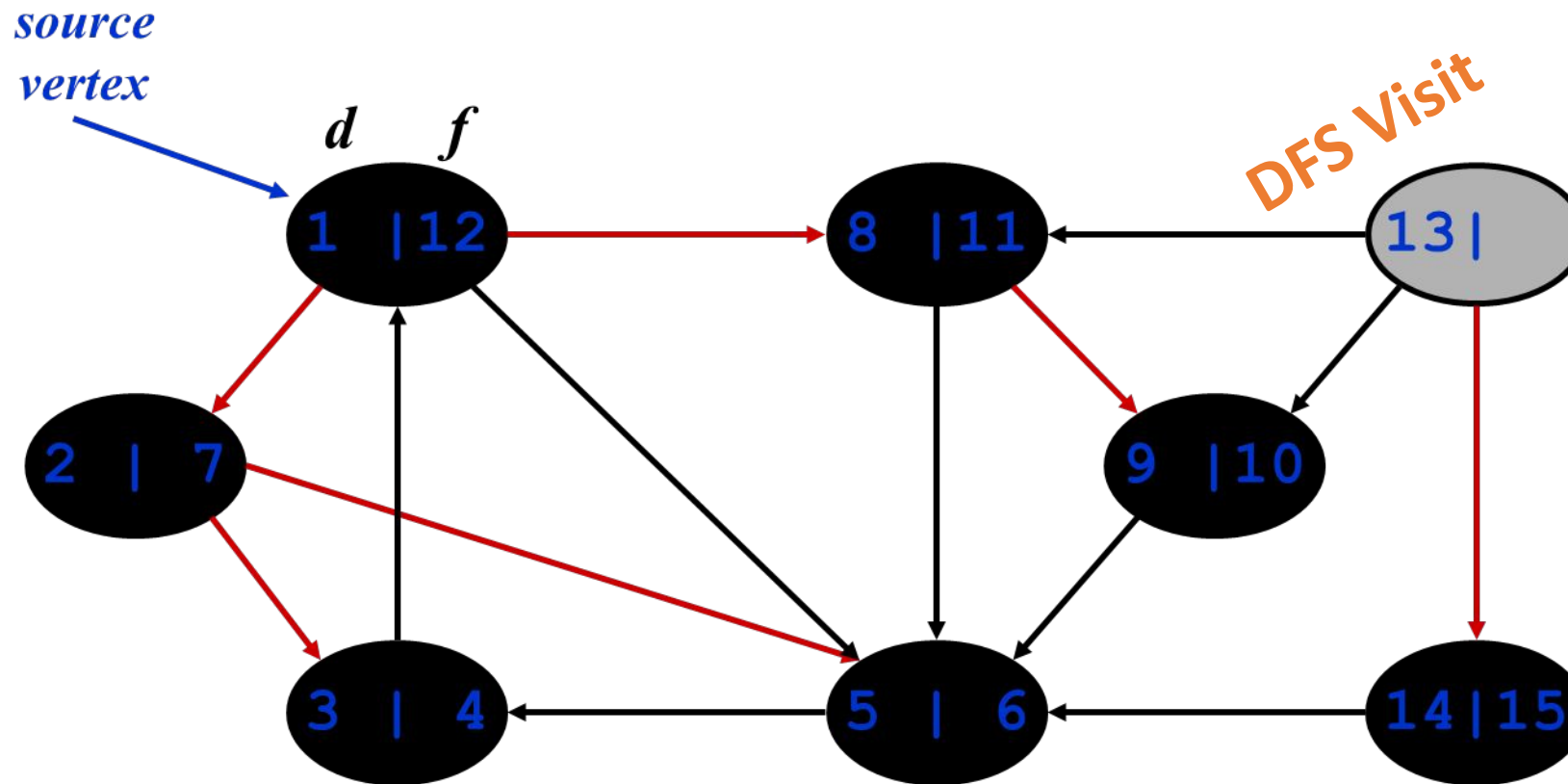
DFS Simulation



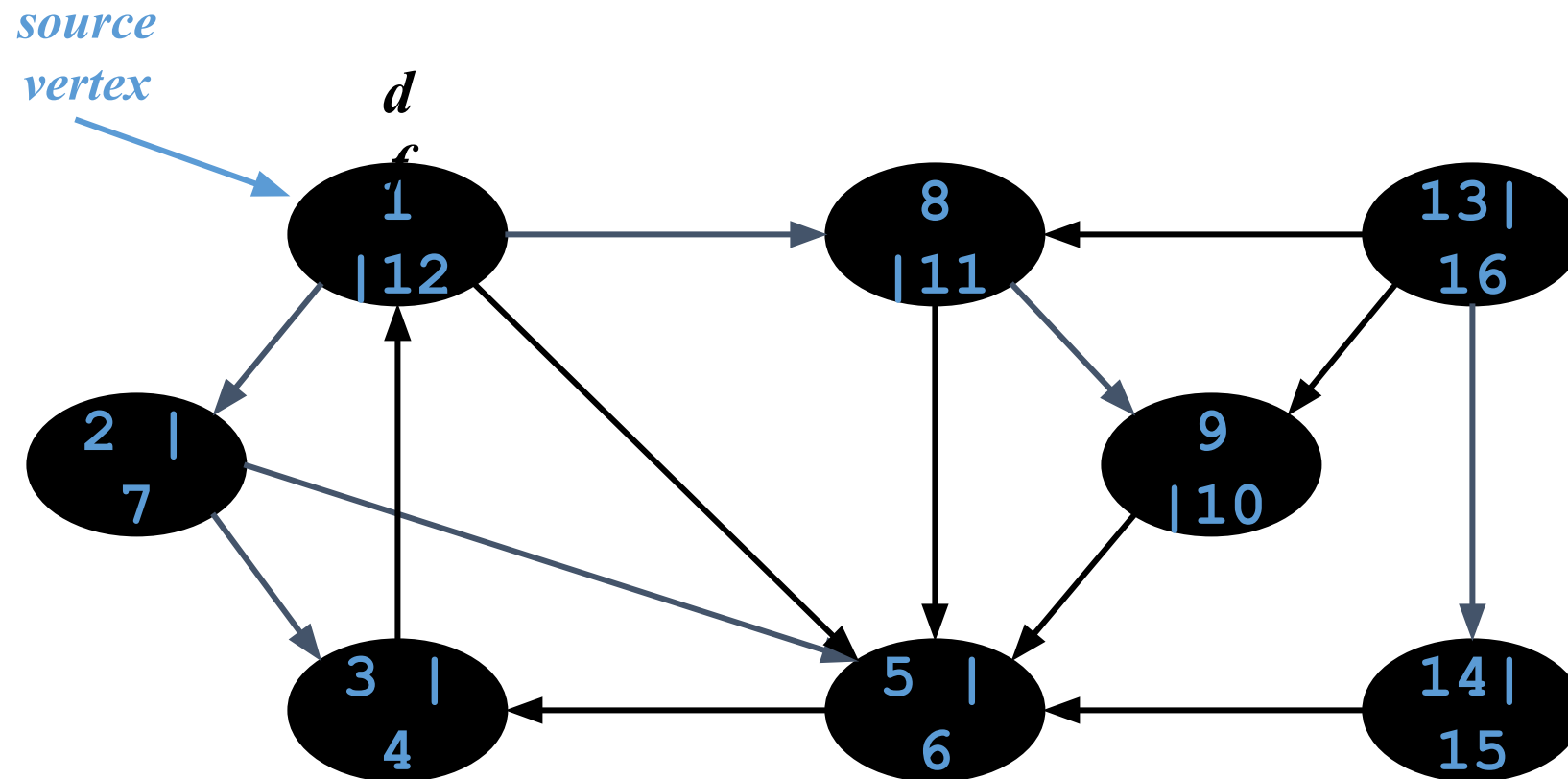
DFS Simulation



DFS Simulation

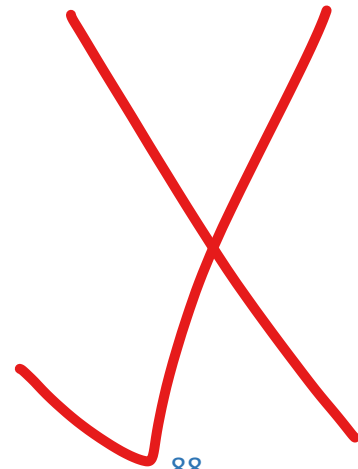
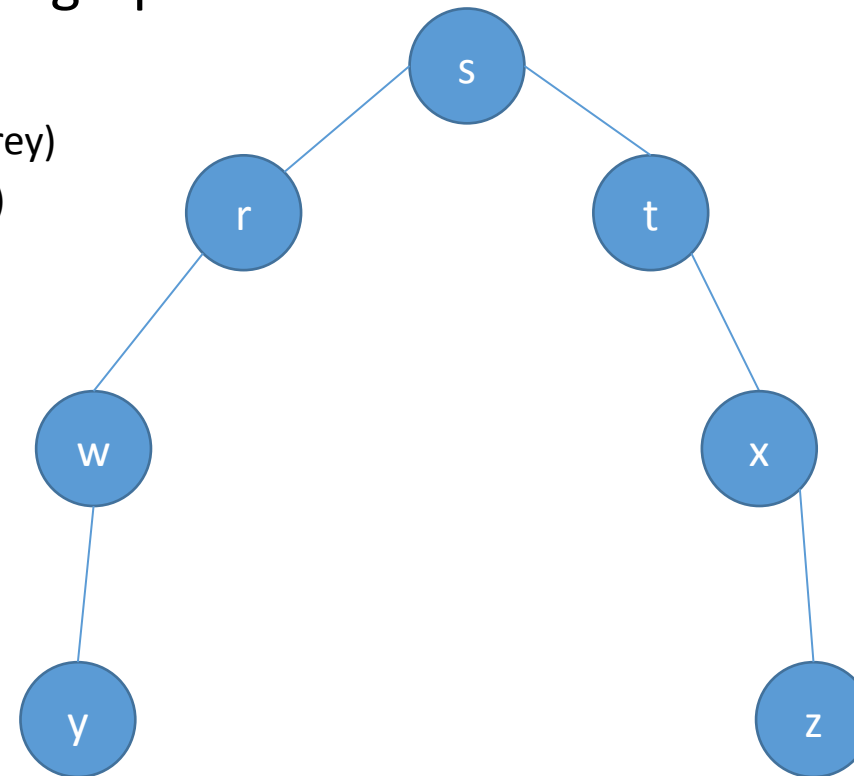


DFS Simulation



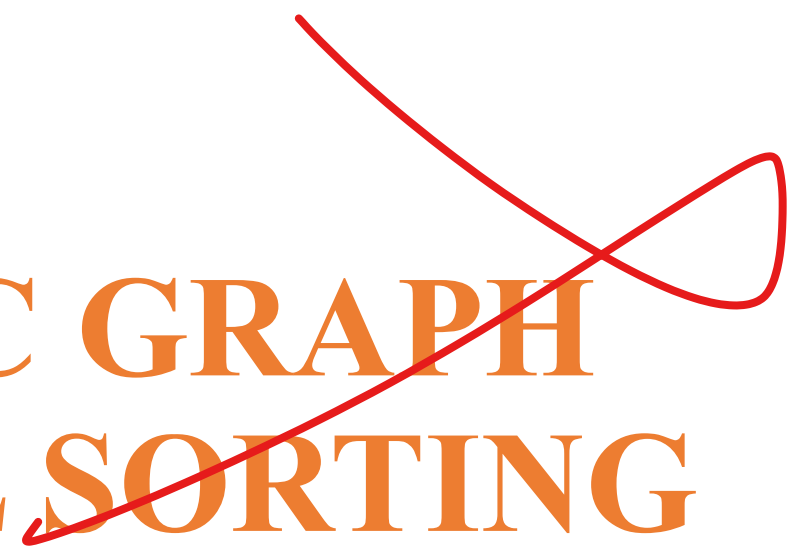
DFS Simulation

- Could we achieve the same result using a stack based implementation?
 - Let us try both approaches with this graph
 - Attributes:
 - Discovery Time (Time when vertex is colored grey)
 - Finish Time (Time when vertex is colored black)
 - Color





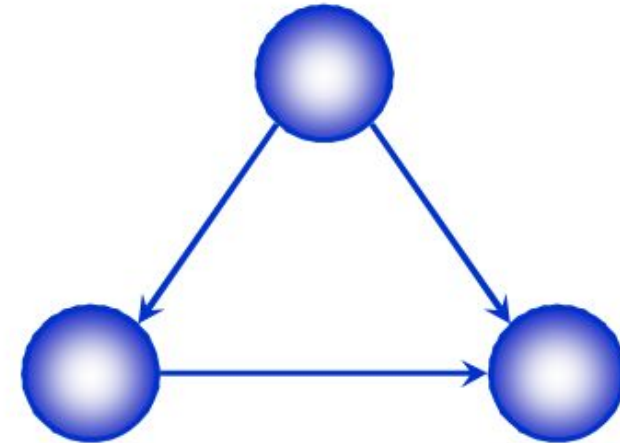
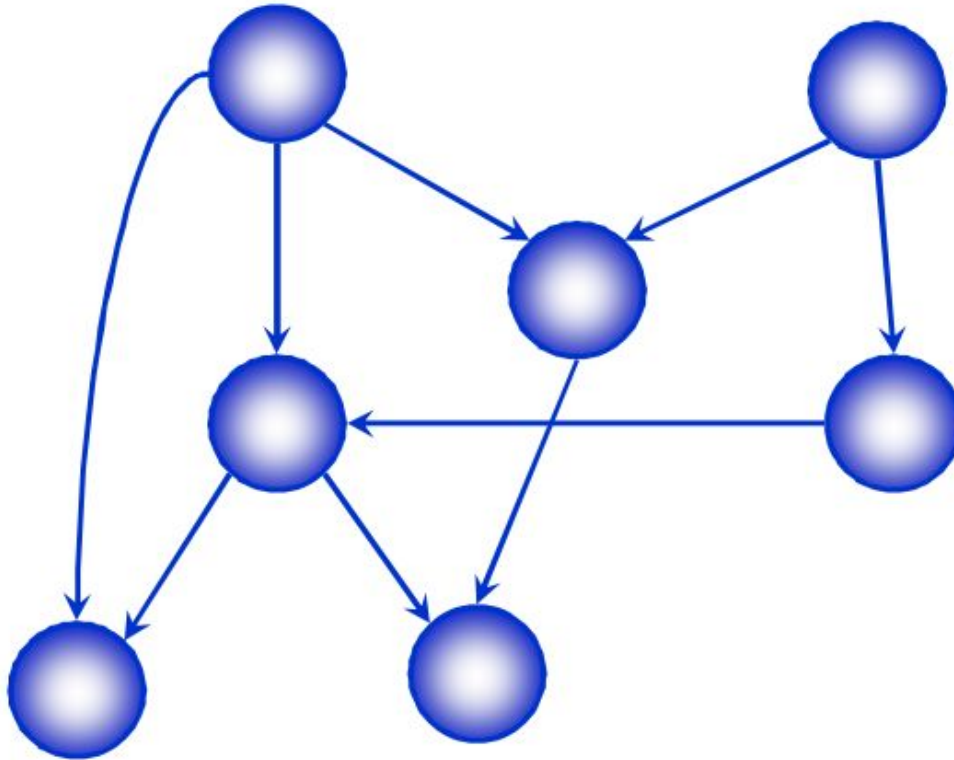
DIRECTED ACYCLIC GRAPH AND TOPOLOGICAL SORTING



Directed Acyclic Graphs



- A directed acyclic graph or DAG is a directed graph with no directed cycles

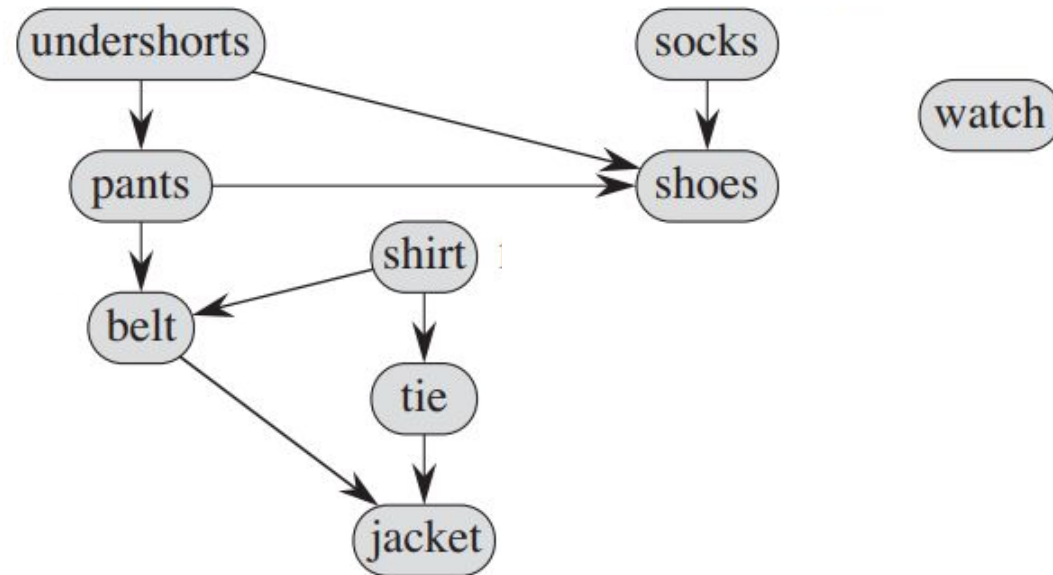


Topological Sort

- A topological sort of a Directed Acyclic Graph $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge (u, v) then u appears before v in the ordering.
- If the graph contains a cycle, then no linear ordering is possible.
- We can view a topological sort of a graph as an ordering of its vertices along a horizontal line so that all directed edges go from left to right.
- Topological sorting is thus different from the usual kind of “sorting” we studied earlier.
- It is useful for sorting events

Topological Sort

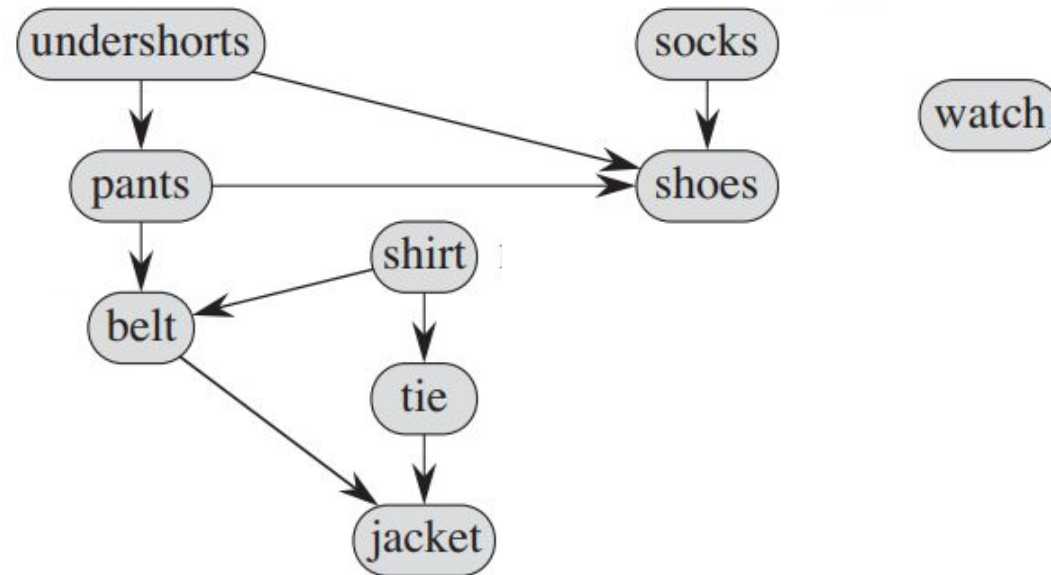
- Reference: Cormen et. al., Introduction to Algorithms, Figure 22.7



How can you store the attributes of these Vertices without a vertex number?

Topological Sort

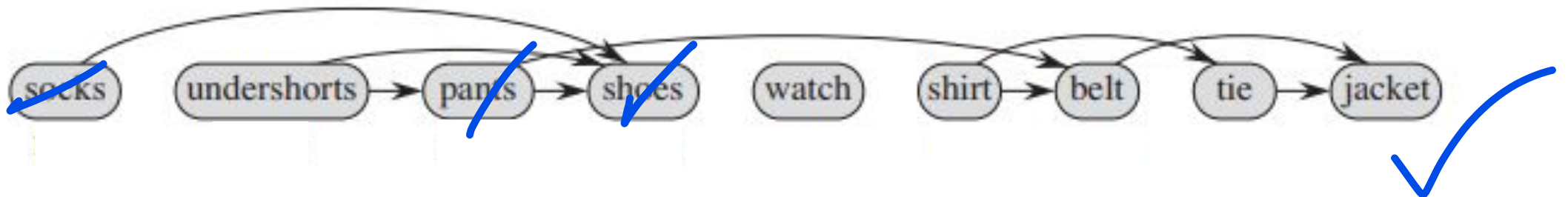
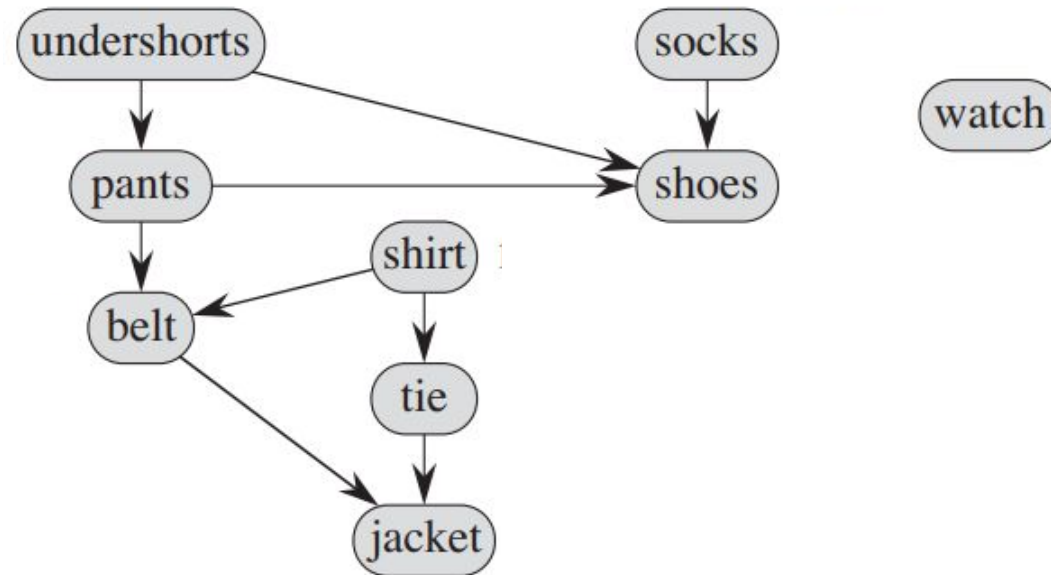
- Reference: Cormen et. al., Introduction to Algorithms, Figure 22.7



How can you sort this graph?

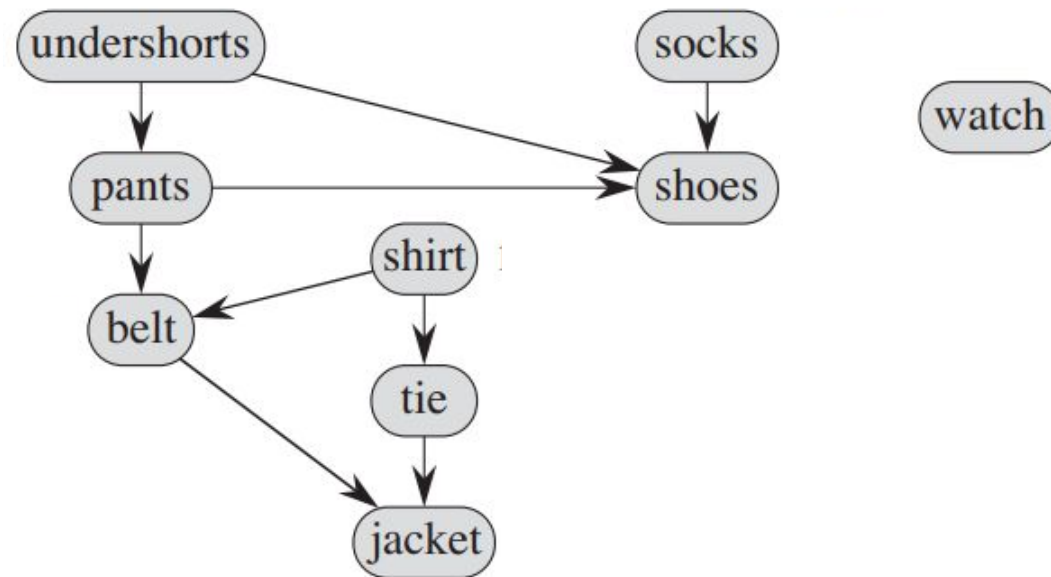
Topological Sort

- Reference: Cormen et. al., Introduction to Algorithms, Figure 22.7



Topological Sort

- Reference: Cormen et. al., Introduction to Algorithms, Figure 22.7



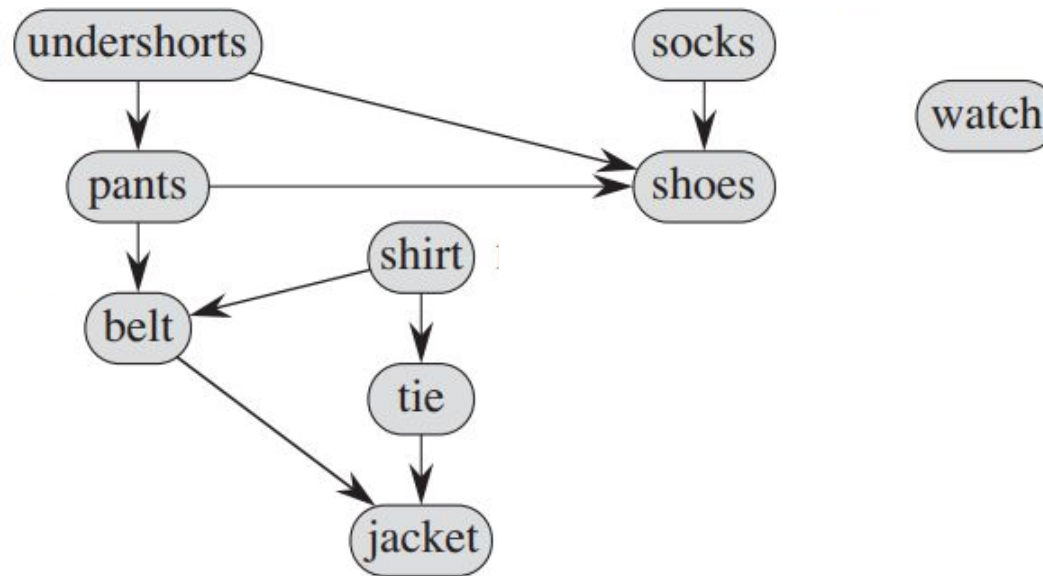
Topological Sort

- Why do we need Directed Acyclic Graphs for Topological Sorting to work?
 - **Circular Dependency** would arise instead of **linear dependency**

Topological Sort - Algorithm

TOPOLOGICAL-SORT(G)

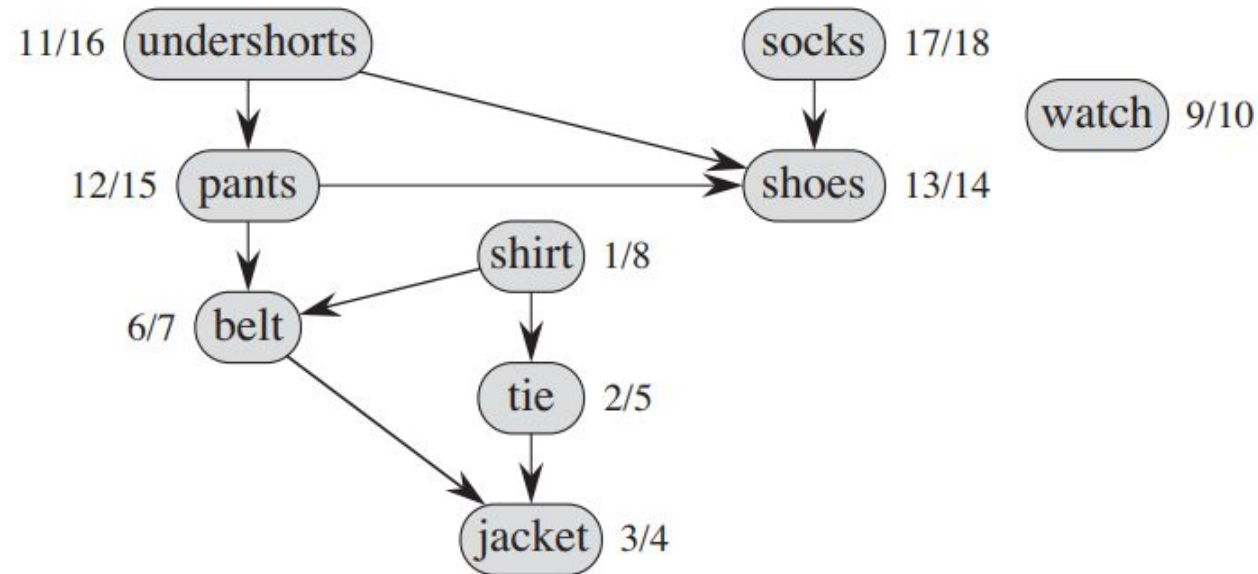
- 1 call **DFS(G)** to compute finishing times $v.f$ for each vertex v
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices



Topological Sort - Algorithm

TOPOLOGICAL-SORT(G)

- 1 call DFS(G) to compute finishing times $v.f$ for each vertex v
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices

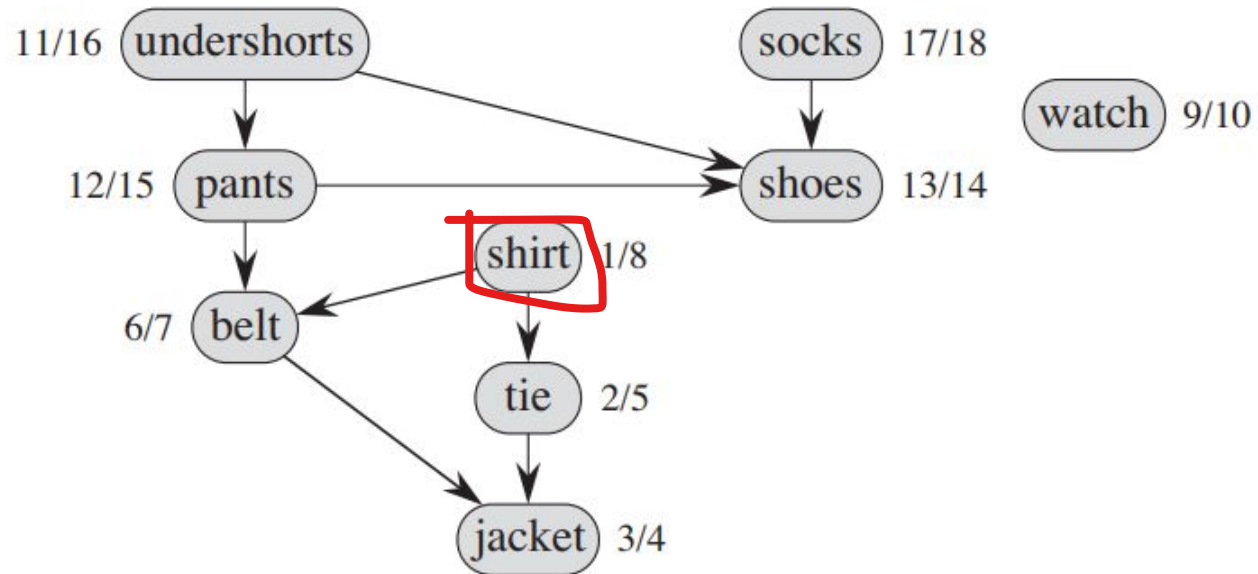


Topological Sort - Algorithm

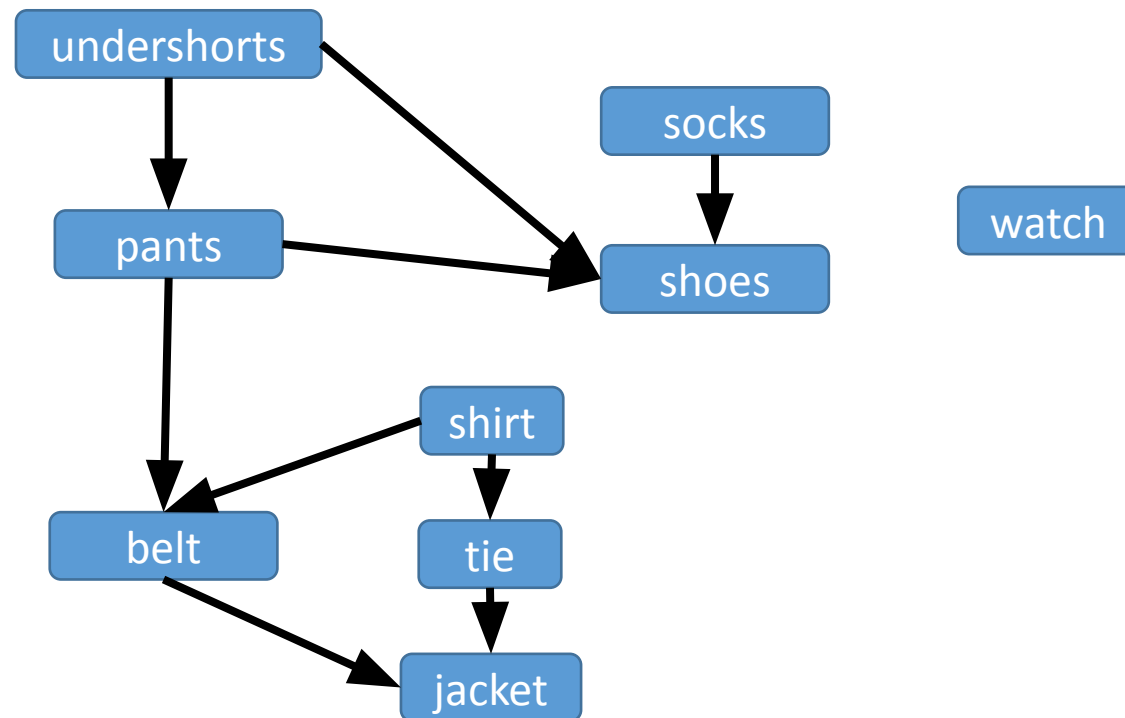
TOPOLOGICAL-SORT(G)

- 1 call DFS(G) to compute finishing times $v.f$ for each vertex v
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices

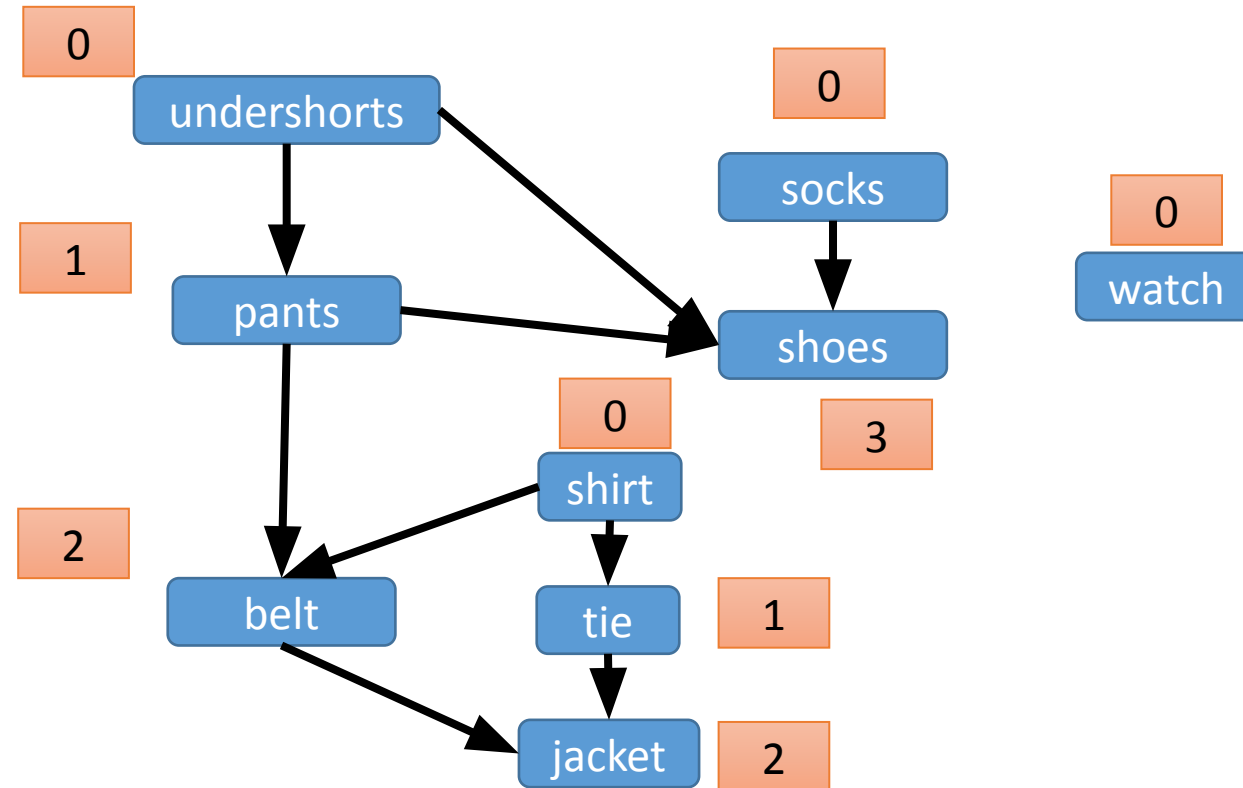
Time Complexity : $O(V+E)$ – Same as DFS



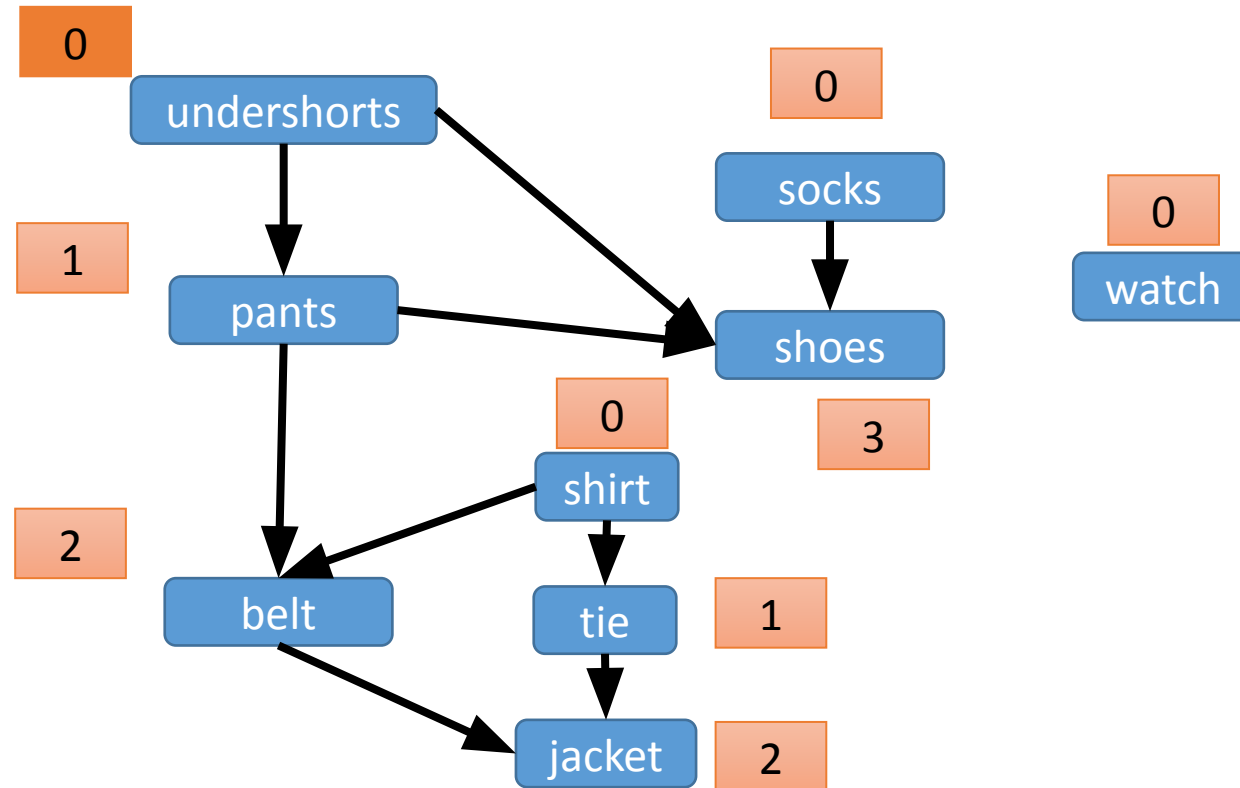
Topological Sort Algorithm (In Degree Method)



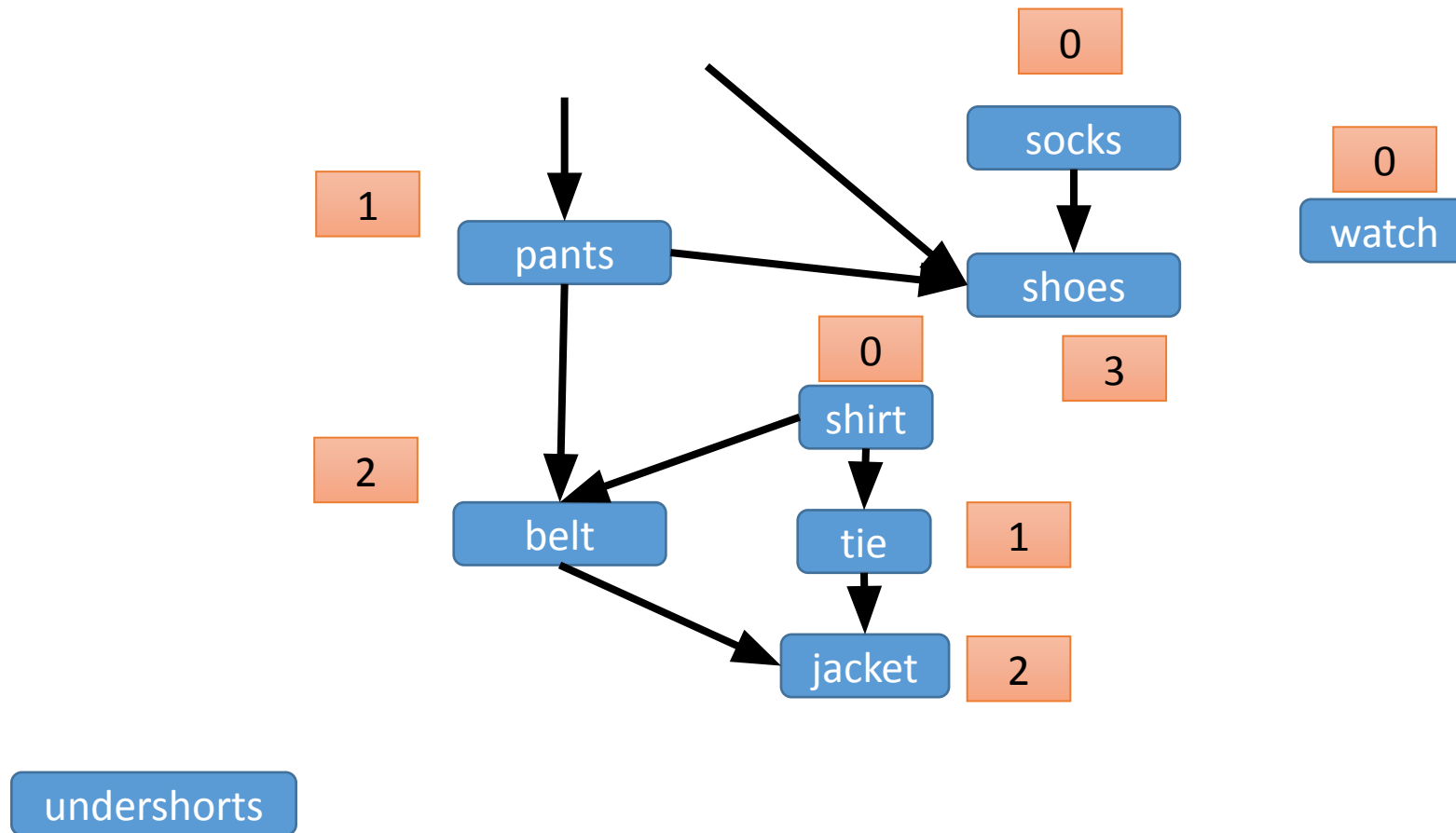
Topological Sort Algorithm (In Degree Method)



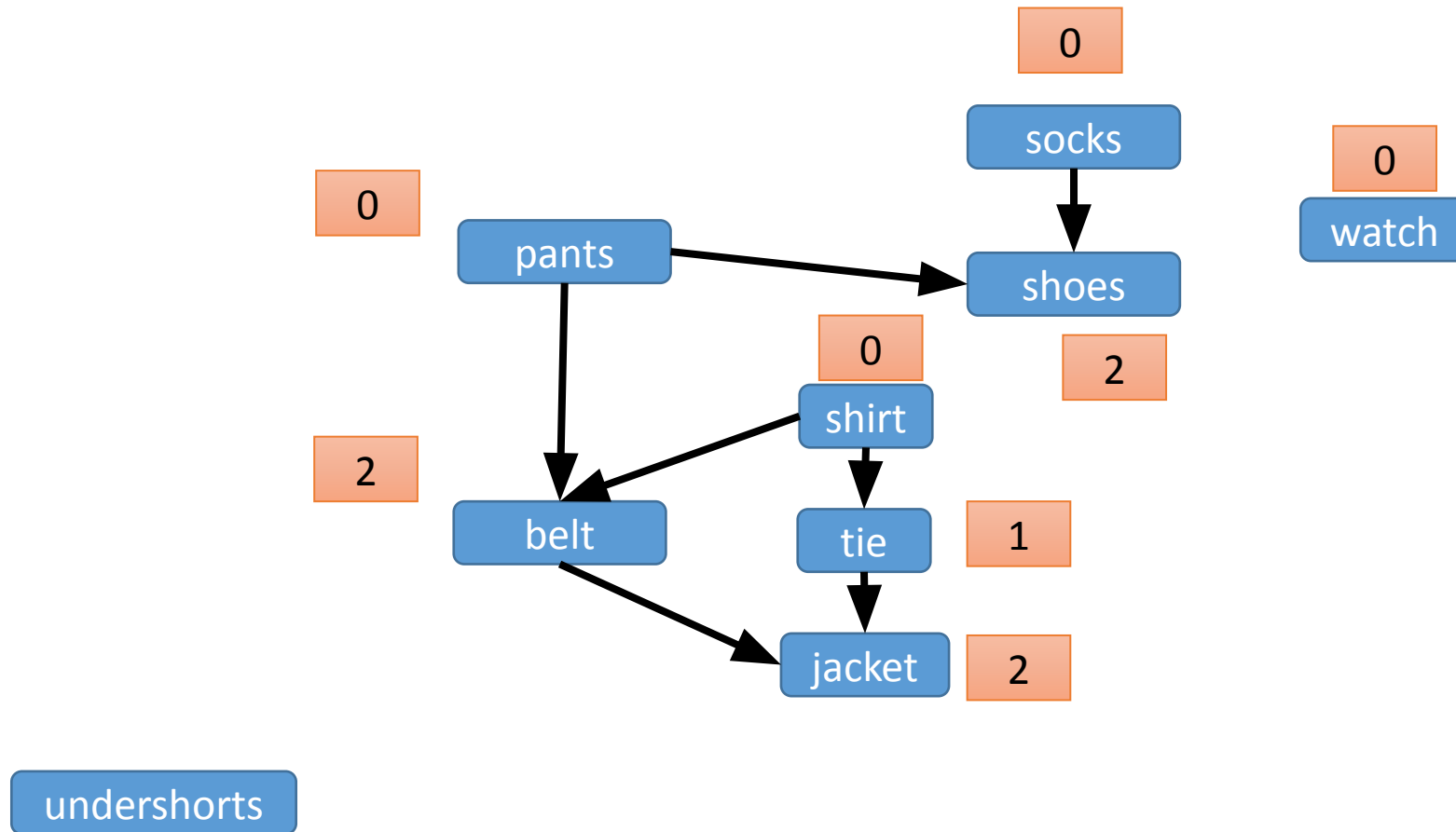
Topological Sort Algorithm (In Degree Method)



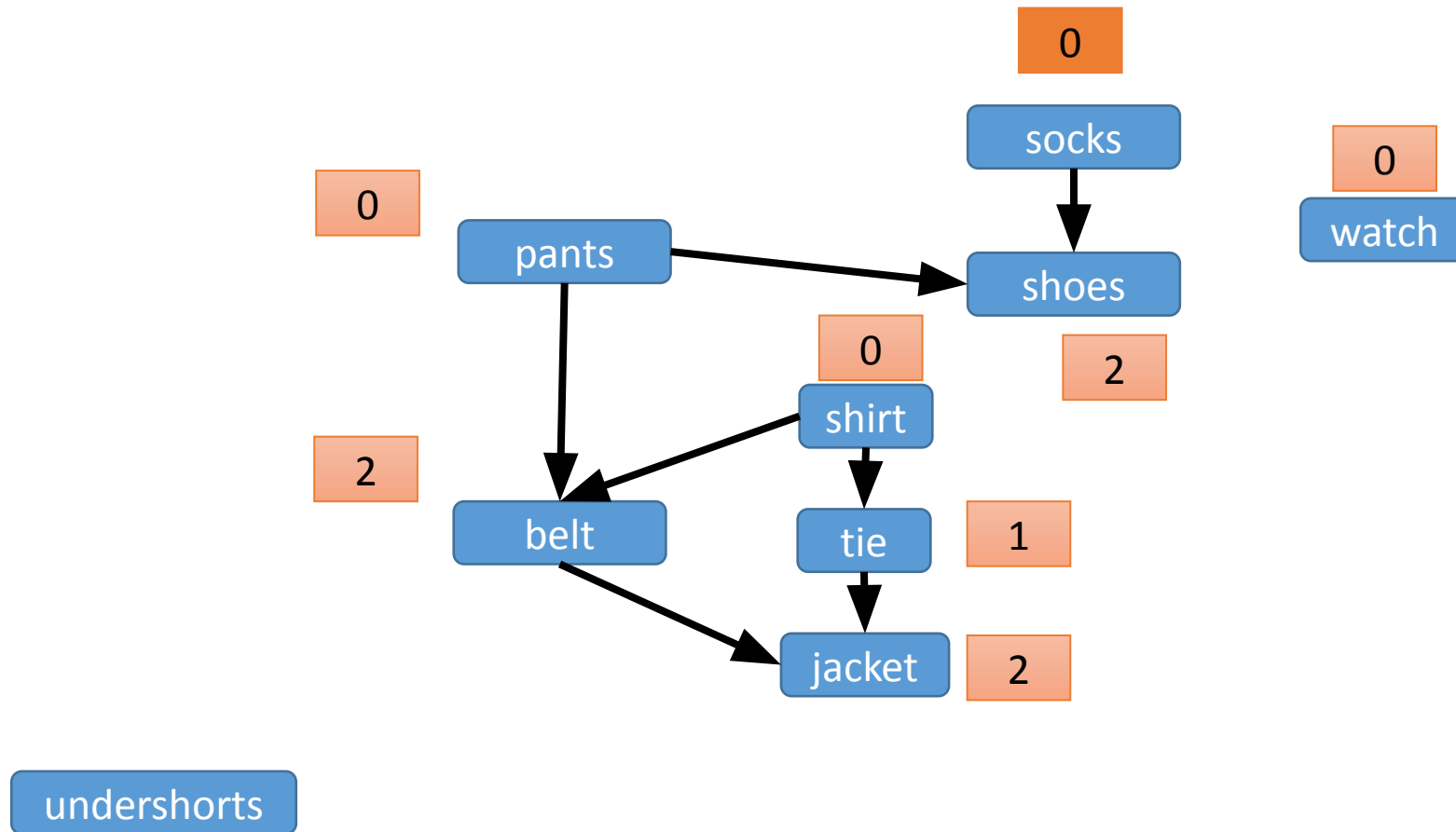
Topological Sort Algorithm (In Degree Method)



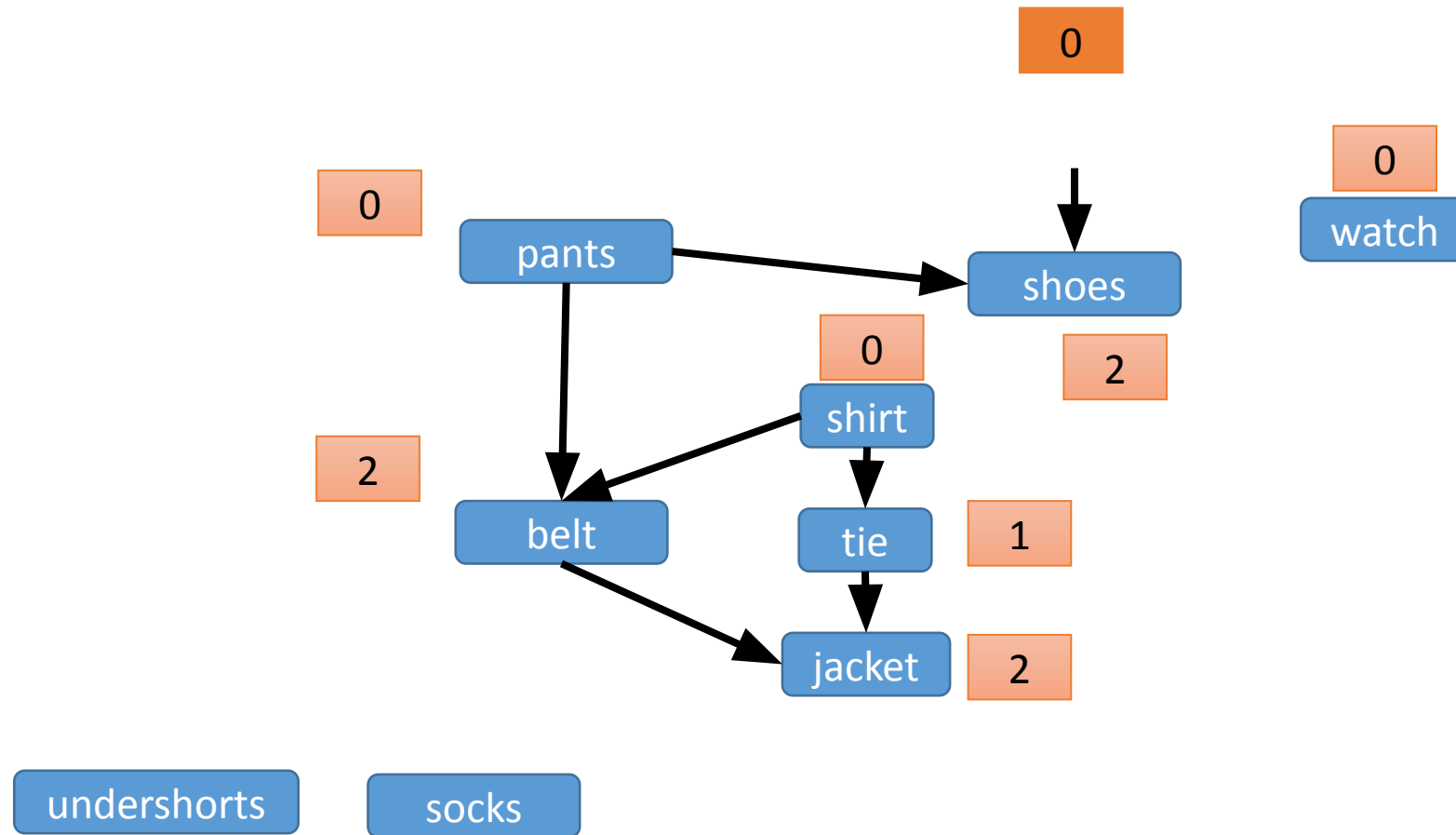
Topological Sort Algorithm (In Degree Method)



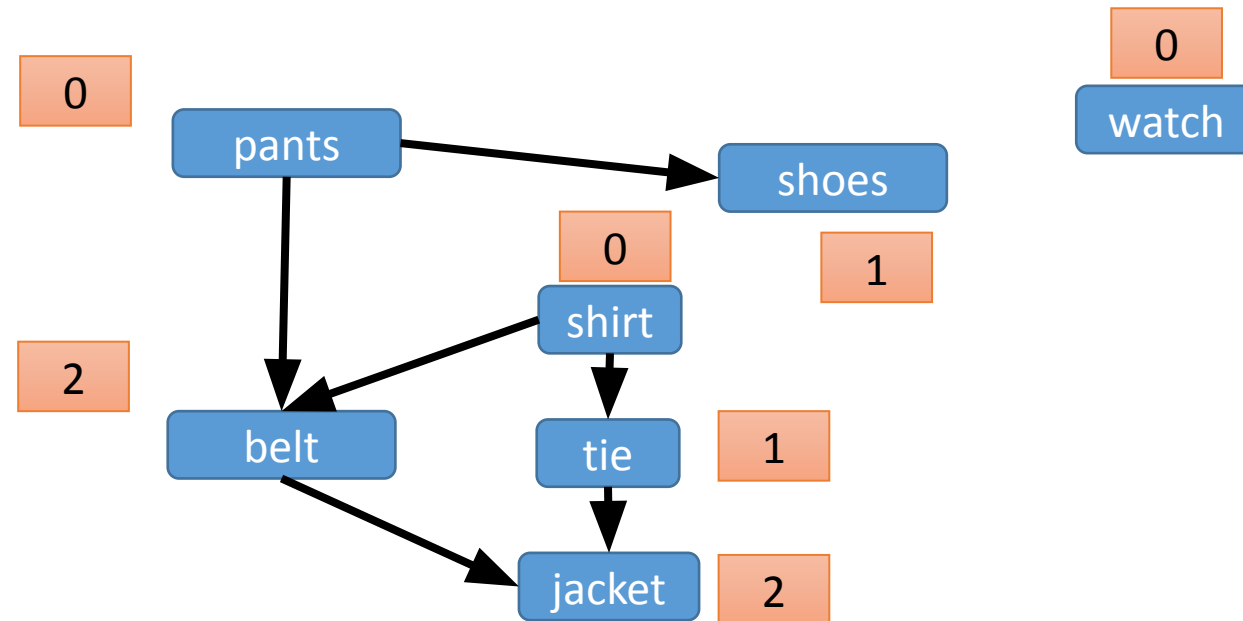
Topological Sort Algorithm (In Degree Method)



Topological Sort Algorithm (In Degree Method)



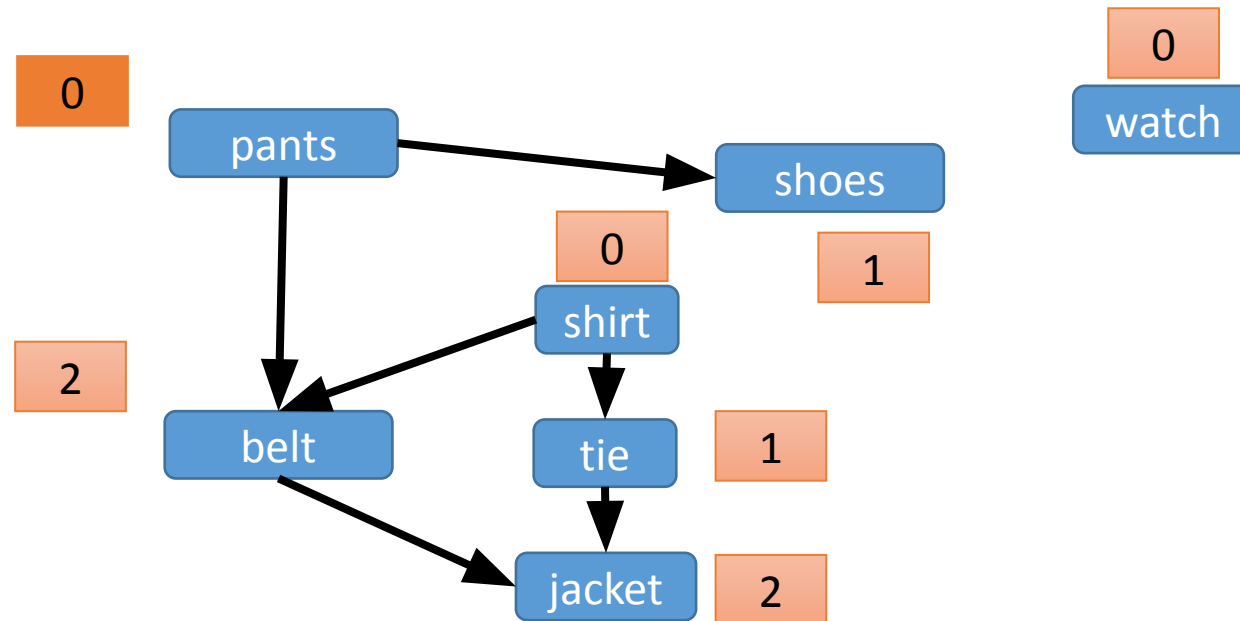
Topological Sort Algorithm (In Degree Method)



undershorts

socks

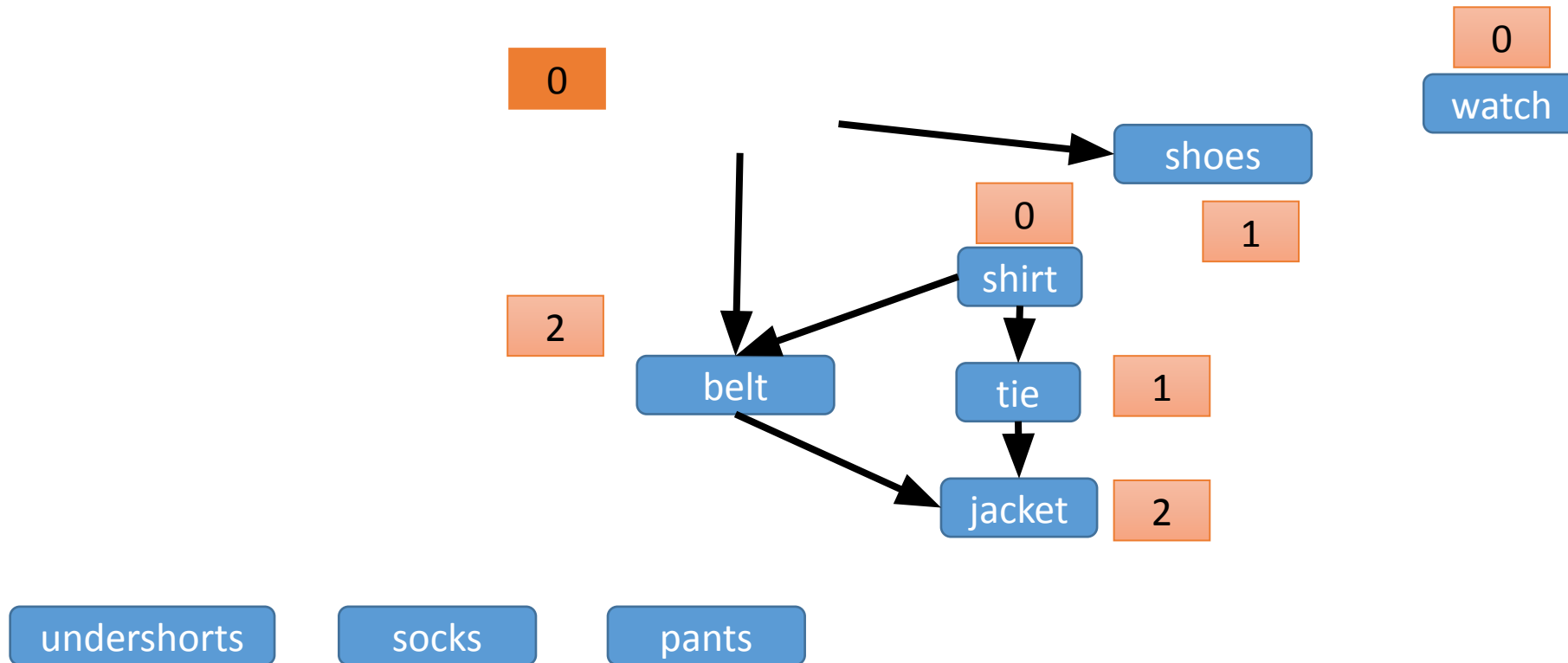
Topological Sort Algorithm (In Degree Method)



undershorts

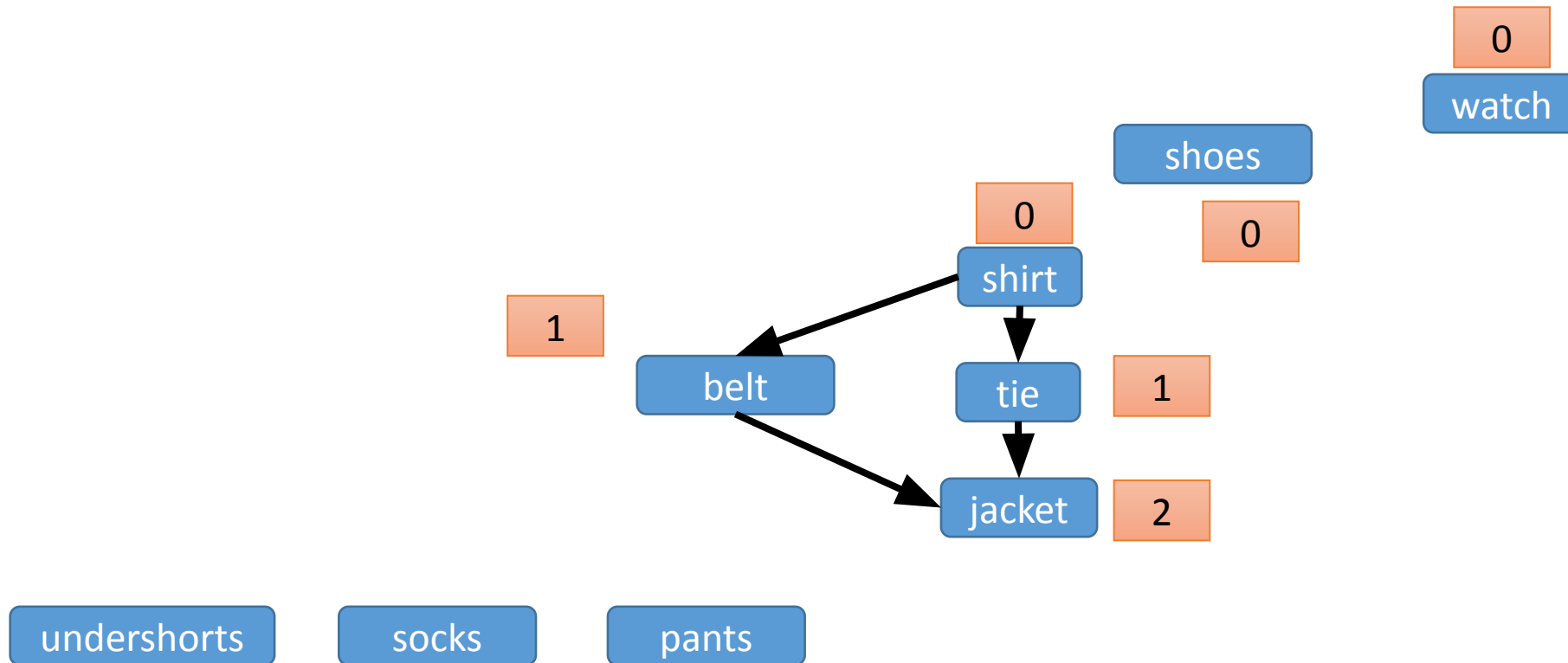
socks

Topological Sort Algorithm (In Degree Method)



Topological Sort Algorithm (In Degree Method)

Keep on going like this until every vertex is removed and inserted in to the tail of the list





Thank You

